Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Duy Khanh Tran

Master's Thesis
Espoo, November 5, 2026

Supervisor: Prof. Petri Vuorimaa
Major: Computer Science

Abstract

A user-interface (UI) transition graph describes the screens of an application and the transitions a user can cause between them. For single-page web applications they are usually built by running the application, though routing frameworks now declare routes in the source. How well static extraction works on real applications, and why it fails, has not been measured construct by construct.

This thesis builds uigraph, a static extractor for Angular, and makes that measurement with it on six open-source applications. uigraph reads nested and parameterised route trees, which the stated restrictions of an earlier Angular extraction exclude in every one of them. Five methods check the graphs. Each construct the extractor does not recover is assigned to a loss category; under the order of the loss rules, the largest is navigation that belongs to no single screen, such as a link in a shared header. For this loss the thesis proposes scope attribution, a dataflow analysis that computes the set of screens from which each navigation site can fire.

Three results stand out. First, almost no navigation can be extracted as a deterministic transition, mostly because the applications' own guards and branches decide it. The corpus yields 159 such edges from 25 constructs. Most of them come from one toolbar whose screen set is untested, and most rest on an assumption about route order that the extractor does not read. Second, almost every cause of loss traced in the source is a construct a static reader can read, so the loss was the tool's; the final version reads most of them, and two lie beyond any static reader of a route table. Third, scope attribution recovers 66.1 % of the constructs the loss rules assign to it, most of them on one application and almost all as possible rather than certain transitions. Unlike the mechanisms of earlier work, it places navigation inside services: 20 of 48 such constructs. Against screen sets read from the sources of two applications, it finds every pair and none outside them on one; on the other, recall off the shell rises from 43.2 % to 90.9 % and precision falls to 82.0 %. Both references apply the analysis's own relations, and the first was partly completed from the graph's output and guided two repairs. Mutation testing on three applications shows that the graph responds to injected changes in constructs it already represents; its harness was corrected after its misses were read, so the test is not independent.

The conclusion is that a declared route table is a usable starting point for static navigation extraction, but not a sufficient one. The tool was developed on this corpus, so every figure is in-sample.

Keywords: static analysis; UI transition graph; single-page application; Angular; dataflow analysis; program analysis; software testing

Use of AI tools

Aalto University asks that the use of artificial intelligence (AI) tools in a thesis be declared. An AI coding assistant was used in this work in the ways set out below, and I am responsible for the whole of the result.

The extractor built in this thesis (uigraph) and the evaluation harness were written with the help of an AI coding assistant. The result tables and plots of Chapter 5 are generated from the harness's result files, except the two tables marked as compiled by hand. So are the figures the text states for the final version. All of these can be reproduced from the commits recorded with them. Four kinds of figure cannot:

- those of superseded instruments, copied by hand from the outputs of those runs (Section 4.3.2);
- a few counts taken at named earlier commits whose output was not committed;
- a few counts from the history of the structural reference and of the instruments, stated in words;
- the corpus-selection counts of Section 4.1, read from the candidate repositories.

The diagrams in Chapters 1 to 4 were drawn by hand.

The assistant was also used to read the application sources, in three roles. As a reviewer it read the constructs behind an edge and reported where the extractor's reading disagreed with them. In red-team panels it attacked the extractor, the harness and this text. Several of the demotion rules of Section 5.5 were written in response to its objections, among them the rule by which the extractor decides which guards a navigation runs. Every defect it reported in either role was reproduced against the code before anything was changed, and the corrections that followed are listed in Section 4.3.2. Its readings are also part of the evaluation itself. The two readings of a development version's must tier in `results-must/review.json` and the reading of the final tier in `results-must/tier-reading.json` were made by the assistant. Chapter 5 says what the reading of the final tier can and cannot show, and Table 4.5 what the review of the development version's tier found. The two readings are two runs of one model, so their agreement measures consistency, not correctness.

The assistant was also used during the literature search, to locate candidate sources and to summarise them. No source is cited on the strength of such a summary.

Every entry in the bibliography was checked field by field against the publisher page, Crossref metadata, arXiv metadata or DBLP before it was cited, and sources that could not be checked in this way are kept in a separate block in the bibliography file and are not cited anywhere in the text.

Finally, the assistant was used to draft and edit prose, which I then revised. The research questions and the interpretation of the results are my own; the analysis and the evaluation protocol were revised in the course of the work in response to the reviews and panels described above, as Section 4.3.2 records, and the responsibility for every claim made here is mine.

Abbreviations

AI	artificial intelligence
API	application programming interface
DOM	Document Object Model
GUI	graphical user interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ICU	International Components for Unicode (message format)
JSON	JavaScript Object Notation
JSP	JavaServer Pages
JSX	JavaScript XML, the markup syntax of React
MVC	model–view–controller
RQ	research question
RxJS	Reactive Extensions for JavaScript
SPA	single-page application
UI	user interface
URL	Uniform Resource Locator

Contents

Abstract **1**

Use of AI tools **2**

Abbreviations **4**

1 Introduction **8**
1.1 Motivation 8
1.2 Research questions and scope 10
1.3 Thesis structure 12

2 Background **14**
2.1 User-interface transition graphs 14
2.2 Static extraction for Android 15
2.3 Navigation models for web applications 16
2.4 Dynamic state models for single-page applications 16
2.5 Static analysis of JavaScript and TypeScript source 18
2.5.1 Analyses over a source-level syntax tree 18
2.5.2 Abstract interpretation of JavaScript 18
2.5.3 What TypeScript's types contribute 19
2.5.4 Frameworks, not the language, are the obstacle 19
2.6 Formal foundations 19
2.6.1 Modal transition systems 20
2.6.2 Soundiness 20
2.6.3 Dataflow analysis and abstract interpretation 20
2.7 Angular routing, dependency injection and reactive state 21
2.7.1 Routes 22
2.7.2 Components and templates 22
2.7.3 Navigation sites 22
2.7.4 Dependency injection 22

2.7.5 Reactive state . . . 24
2.8 Reactive programming and the limits of static dataflow . . . 24
2.8.1 Propagation order and glitch freedom . . . 24
2.8.2 Reactive code and the tools developers have . . . 25
2.9 Mutation testing . . . 25
2.9.1 The coupling effect . . . 26
2.9.2 Mutation for JavaScript and web applications . . . 26
2.9.3 Reporting a mutation experiment . . . 26
2.10 Summary and the gap . . . 27

3 Static navigation analysis and scope attribution 29
3.1 Graph model and soundness . . . 29
3.2 Scope attribution . . . 34
3.2.1 The defect being addressed . . . 35
3.2.2 Scope . . . 35
3.2.3 Input relations . . . 37
3.2.4 Guards as program points . . . 38
3.2.5 Boundary . . . 39
3.2.6 Anchored edges . . . 39
3.3 Implementation . . . 39
3.4 Summary . . . 42

4 Evaluation setup and research material 44
4.1 Corpus selection and validity . . . 44
4.1.1 Selection . . . 44
4.1.2 Validity of an open-source corpus . . . 45
4.2 Metrics . . . 46
4.3 Evaluation methods . . . 50
4.3.1 Preconditions . . . 53
4.3.2 Corrections made to the protocol . . . 53
4.3.3 Reproducibility . . . 60
4.4 Summary . . . 60

5 Results 63
5.1 The two levels of loss (RQ1) . . . 63
5.2 Which failure is worth an analysis (RQ2) . . . 65
5.3 Scope attribution (RQ3) . . . 67
5.3.1 Checking the scopes against the running interface . . . 70

5.3.2 What the attribution discriminates . . . 73
5.3.3 What each mechanism contributes . . . 74
5.3.4 Agreement with a structural reference . . . 75
5.3.5 Agreement with the applications' own suites . . . 76
5.3.6 What scope attribution adds . . . 78
5.4 Change sensitivity (RQ4) . . . 80
5.5 What the losses were (RQ5) . . . 81
5.5.1 What the losses were . . . 82
5.5.2 Construct accounting . . . 84
5.5.3 The final tier and the node set . . . 85
5.6 Dead links, found from the source . . . 86

6 Discussion **88**
6.1 Key findings . . . 88
6.2 Limitations and threats to validity . . . 90
6.3 Declared approximations . . . 97
6.3.1 Why the may tier is incomplete . . . 102
6.4 Transfer to other frameworks . . . 103
6.5 Future work . . . 105
6.6 Summary . . . 106

7 Conclusion **107**

Bibliography **111**

Chapter 1

Introduction

A user interface (UI) transition graph is a model of an application. The nodes are the screens a user can see, and the edges are the transitions a user action can cause between them. For each edge, the graph records the event that triggers it, the condition under which the transition is available, and the screen it leads to. A graph of this kind makes it possible to answer questions that are difficult to answer from the code alone. One example is which screens are reachable from a given screen. Another is which parts of the user interface are affected when a particular component changes.

Transition graphs have been used in automated testing for a long time. In the Android domain they are used for test generation and for guiding exploration [79, 39, 14], and in web testing they form the basis of crawling-based test derivation [50, 51]. They are also useful for change impact analysis [4, 12], because each transition can be traced back to the code that implements it, and from there to the tests that cover it.

This chapter explains why a static transition graph for single-page applications is worth measuring and what is not yet known about it. It then states the research questions and the limits of scope, and ends with the structure of the thesis.

1.1 Motivation

There are two general ways to obtain a transition graph. A dynamic approach executes the application and records the states and transitions it observes. A static approach reads the source code and derives the graph without running anything. Their limitations differ. A dynamically built graph contains only the states the crawler managed to reach. Which states those are depends on the available test data, authentication, and the exploration budget. The graph also has to be rebuilt for every new version. A statically built graph covers everything the source declares. It can be recomputed on each commit without a running system. It is limited, however, to what can be decided from the source, and it must over-approximate wherever a target cannot be resolved.

The two limitations complement each other. Applications of the kind studied here are mostly used signed in. Much of their navigation therefore runs behind an

authentication guard whose outcome only a running system decides. A crawler, for its part, reaches only the states its data and credentials open. A static graph can name the screens and candidate transitions a crawler should look for, and the guards each waits on. A run can confirm which of them occur. This thesis measures the static side; the combination is discussed in Section 6.5 and is not measured.

For Android, static extraction is a well-established technique. The window transition graph work derives windows and the transitions between them from bytecode. It also resolves the control flow of callbacks to the widget that registered each handler [79, 78, 77]. For the web, navigation models have been extracted statically from server-rendered frameworks, in which pages and links are declared in templates and configuration files [62, 9, 69]. Modern single-page applications, however, have almost always been modelled by executing them. Crawljax builds its state flow graph by firing events against the Document Object Model (DOM) of a running application [50]. Later tools in the same line improve the exploration strategy without changing where the model comes from [51, 74]. A survey of web testing covering 2014 to 2025 collects the model-based tools of the past decade [36]. Of the tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application. Static analyses of a modern client-side framework are few. One builds a component dependency graph for React and does not address routing [38]. Three extract navigation, each for a different consumer:

- one from Angular in order to model-check it, accepting only applications that declare no nested routes, no route parameters and no data-bound navigation [56];
- one from Angular sources in order to enumerate the workflows a generated end-to-end suite should cover [64];
- one from the compiled JavaScript bundle of a single-page application in order to check an access-control policy against the screens it can reach [81].

Section 2.4 compares them with the work reported here.

When the dynamic approach was developed, a single-page application had no declarative route table. Its navigation therefore had no explicit representation in the source and was hard to recover from it. The DOM was the observable the tools used (Section 2.4). Client-side routing frameworks changed this. Angular, React Router and Vue Router all declare routes as data in the source code. They connect navigation to those routes through template attributes and service calls. They also attach guards to routes as named functions [27]. The artefact needed for static analysis now exists. How well static extraction works on real applications built with these frameworks, and where it fails, has been measured only against hand-built references on the applications each work evaluated. It has not been measured construct by construct.

A statically extracted graph also has consumers outside model-based testing. Tools that drive a graphical user interface (GUI) with a large language model either explore an application without a model of its navigation structure or build one by

exploring it. GPTDroid treats mobile GUI testing as a question-and-answer loop, passing the current screen to the model and replaying the action it returns. It reports higher activity coverage and more bugs than the best baseline on 93 applications [46]. AutoDroid gives the model application-specific knowledge in addition to its commonsense knowledge, held as a UI transition graph that it builds by automated dynamic exploration offline [73]. LLMDroid uses the model to guide exploration towards coverage the test run has not reached [70]. The approach has been applied to web interfaces with a vision-language model and curiosity-driven exploration [72], and a survey of large language models in software testing covers the wider area [71]. Coding agents working on a repository may also benefit from structure of this kind. The SWE-bench issues typically require coordinated edits across functions, classes and files [31]. Agent performance also depends on the interface through which the repository is navigated [76]. Whether a navigation graph helps such tools is not tested here. These uses motivate the extraction but do not depend on the results of this thesis.

This thesis builds a static extractor for Angular and carries out that measurement with it. It also proposes an analysis for the largest loss it finds under the order of its loss rules. The analysis, scope attribution, is stated as a dataflow analysis over a lattice of screen sets. Nothing in its formulation depends on this extractor.

A method of this kind cannot be assessed without being instantiated, because the questions asked here are empirical. They ask how much of the navigation in real applications it recovers, where it fails, and whether the resulting graph responds to a change in the source. The extractor is *uigraph*, an open-source static analyser for Angular built in this thesis. It is part of the contribution, not an existing tool. It reads nested, parameterised and lazily loaded route trees, which an earlier Angular extraction excludes by its stated restrictions (Section 5.3.6). It also implements scope attribution. Its development versions and the corrections between them are listed in Section 4.3.2. Because the same person built the tool and measured it, on the same corpus, the figures describe what this extractor recovers on these applications. Section 6.2 discusses what that means for them. Chapter 6 separates the limits of the method from those of this implementation.

1.2 Research questions and scope

The thesis addresses five research questions (RQs). This section states what each asks and how it is answered. The exact measures are defined in Section 4.2. The answers are given in Chapter 5.

RQ1. *How much of the navigation in Angular applications can static analysis recover as deterministic transitions?*

This is measured at two levels, because a transition cannot be recovered without the screens it runs between. The first level is node coverage: the share of the screens an application declares that the graph holds a node for. The second is the share of navigation constructs in the source that yield at least one deterministic (*must*) transition.

RQ2. *Where does static extraction fail, and why?*

The observed failures are grouped into defect classes. A class is accepted only if at least two lines of evidence confirm it. At least one of them must come from an evaluation method of Section 4.3. Each construct that is not recovered is assigned to one loss category, so the categories can be compared by size.

RQ3. *How does scope attribution affect the recovered share and precision of static navigation analysis?*

Scope attribution is the analysis this thesis proposes. For a navigation site that belongs to no single screen, it computes the set of screens from which the site can fire. The same extractor is run with the analysis disabled and enabled, so that nothing else differs between the two runs.

RQ4. *Does the extracted graph respond to source changes in the constructs it represents, as change impact analysis requires?*

This is measured with mutation testing: known changes are inserted into the source, and the graph is checked for the expected difference.

RQ5. *Are the losses a property of the applications or of the tool?*

A defect class names where extraction failed, but not whether the navigation was unreadable or merely unread. Each class is therefore traced to its cause in the source, and for each cause the thesis records whether the final version reads it. If it does, the information was in the source and the loss was the tool's. The causes that are still not read, and those beyond a static reader, are named. Every site that still yields no transition must carry a recorded label.

The questions are answered on a corpus of six open-source Angular applications pinned to fixed commits (Chapter 4). Five evaluation methods are used, and RQ3 adds four further measurements; the methods are chosen so that their blind spots differ. The figures in Chapter 5 are those of the final version of the extractor. Its development versions and the corrections between them are listed in Section 4.3.2.

The thesis has the following limits of scope.

- It does not claim completeness. Where a navigation target is computed at run time, the transitions cannot in general be determined statically. The thesis instead states a soundness property over a declared scope and lists what it approximates. This follows the soundiness position (Section 2.6.2). The property is tested, not proved.
- It does not measure the effect of the graph on coding agents or any other consumer.
- The screen sets that scope attribution computes are compared in both directions on two applications only, against references that apply the analysis's own relations.
- uigraph was developed on this corpus, so every figure is in-sample and is not an estimate for other applications.

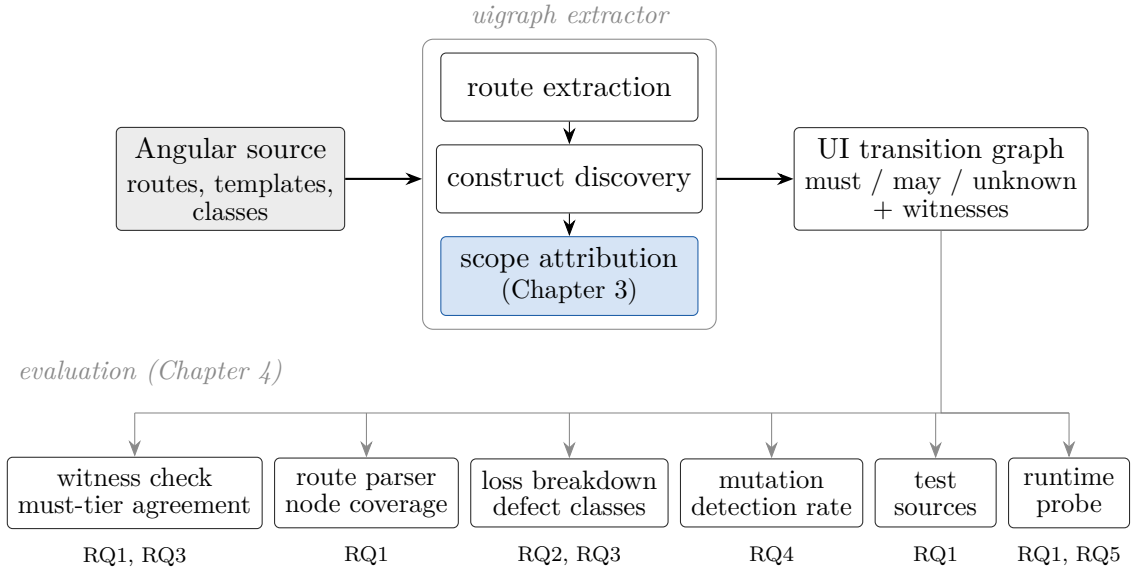


Figure 1.1: Overview of the study: extractor, scope attribution, the main measurements and the research questions they answer.

Note. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. The loss breakdown classifies what the other methods find missing rather than being one of the five methods itself.

- All measurements concern Angular. The graph model does not depend on the framework, but no other framework is evaluated.

1.3 Thesis structure

Figure 1.1 shows how the parts of the study fit together.

Chapter 2 presents the background. It introduces the terminology used for transition graphs in different research communities. It reviews static extraction for Android, navigation models for web applications and dynamic state models for single-page applications. It also describes the Angular mechanisms the analysis has to handle, together with the program analysis ideas it builds on. Chapter 3 presents the graph model and its soundness claim, the scope attribution analysis proposed as a modest extension, and its implementation. The measurement, which is the main contribution, is set up in Chapter 4 and reported in Chapter 5. Chapter 4 describes the evaluation setup: the corpus and why it can be considered valid, the metrics, and the five evaluation methods, and lists in Table 4.5 every correction made to the extractor and the instruments during development. Chapter 5 presents the results for each research question. These are the two levels of loss (RQ1), the defect classes (RQ2), scope attribution and what it recovers (RQ3), change sensitivity (RQ4), and the causes of the losses and the soundness of the must tier (RQ5). The chapter then reports the dead-link candidates the graph finds statically. Each is checked against the route configuration the runtime route oracle recorded. Chapter 6 discusses the results, the limitations of the study, the constructs the analysis approximates, how

the approach transfers to other frameworks, and future work. Chapter 7 concludes the thesis.

Chapter 2

Background

This chapter gives the background needed for the rest of the thesis and ends by stating the gap the thesis addresses. Section 2.1 fixes the terminology. Sections 2.2 to 2.4 review the three lines of prior work closest to this thesis: static extraction for Android, navigation models for web applications, and dynamic state models for single-page applications (SPAs). Section 2.5 reviews source-level static analysis, Section 2.6 the three formal ideas the analysis builds on, and Section 2.7 the Angular mechanisms it has to deal with. Section 2.8 reviews reactive programming and the boundary it puts on a static analysis. Section 2.9 reviews mutation testing, the method Chapter 4 uses to answer RQ4. Section 2.10 states the gap.

2.1 User-interface transition graphs

A UI transition graph is a directed graph in which the nodes represent user-visible states of an application and the edges represent transitions that a user action can cause between those states. Figure 2.1 shows a small example in the model used in this thesis.

Several research communities have studied this kind of graph under different names. In Android static analysis it is a *window transition graph* [79], *activity transition graph* [14] or *screen transition graph* [39]. For Ajax (asynchronous JavaScript and XML) crawlers it is a *state flow graph* over DOM states [50]. In web engineering it is a *navigation model* or *navigation state machine* [62, 18]. This thesis uses the term *UI transition graph* and treats the other terms as equivalent for its purposes, although they differ in what counts as a state and in how the graph is constructed.

More important than the naming is how the graph is obtained: *statically*, from source code or binaries without executing the application, or *dynamically*, by running and observing it. It also matters whether the graph represents *guards*. That is, it matters whether each edge records the condition under which the transition is available, or only that the transition exists. Table 2.1 classifies the work reviewed in the following sections according to both distinctions.

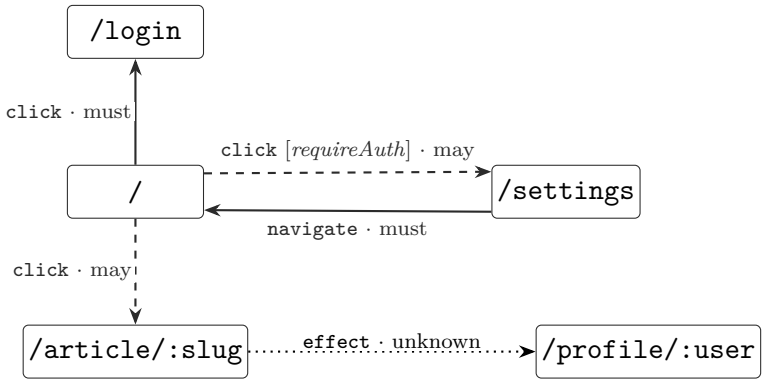


Figure 2.1: A small UI transition graph in the model of this thesis, with must, may and unknown edges.

Note. Each edge carries an event, an optional guard in brackets, and a modality: *must* always fires when its code runs, *may* is conditional (here on a route guard or a partly known target). The dotted *unknown* edge shows the model's treatment of a navigation inside a signal effect, whose firing condition cannot be determined statically; the prototype does not implement it (Section 3.2.5). In the measured graphs the unknown modality marks only edges into a sink for destinations that only the run time knows.

2.2 Static extraction for Android

The largest body of static work on transition graphs concerns Android. Yang et al. introduced the window transition graph. Its nodes are windows (activities, dialogs and menus), and its edges are transitions caused by callbacks that run on the UI thread. The analysis models the stack of active windows, so that the effect of the back button is captured [79]. A journal version extends the work [77]. A related analysis by the same group constructs a callback control-flow graph whose nodes are pairs of a handler and the widget it belongs to [78]. StoryDroid builds an activity transition graph from `startActivity` calls [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration [39]. UI-CTX finds the code context of a widget's behaviour with a backward data-flow analysis from the handler to the call that retrieves the widget [44].

These approaches have in common the way a navigation is attributed to a screen. In Android, an event handler is registered on a widget object, and at run time that object belongs to exactly one window. Attribution can therefore be treated as an object resolution problem: the widget identifier is resolved, usually by constant propagation of the `getId()` call, and the window is then known [78].

The limits of this approach become visible in ProMal. Its authors report that a purely static window transition graph contains infeasible transitions. The cause is that imprecision in the reference analysis makes several widgets aliases of one another, and the over-approximated data flow then attaches a handler to the wrong window. They address this by combining the static analysis with runtime traces and a learned model [45]. Where the owning object cannot be resolved precisely, ProMal therefore turns to dynamic evidence rather than refining the static attribution.

2.3 Navigation models for web applications

Navigation models for the web were first extracted from server-rendered sites, well before single-page applications became common. Ricca and Tonella modelled a web application as a set of pages, frames and forms connected by links and form submissions. Their model includes conditional edges that carry a boolean condition on input values, and was derived semi-automatically with a crawler and annotations by an analyst [62]. WebUml recovers a state diagram from Hypertext Markup Language (HTML), script and server-side pages, deriving guards by control-flow analysis of the server code [9]. A patent assigned to Fujitsu describes a fully static method for extracting a navigation model from Struts and JSP sources [69]. For verification, SiteHopper builds abstracted navigation state machines on the fly for model checking [18].

This work shows that navigation models, including guards, have been extracted statically from web applications. The models reviewed in this section all concern server-rendered frameworks, where a page corresponds to a file and a link to an anchor element. None of them deals with a client-side router, a component tree, or dependency injection. Section 2.4 turns to the work that does.

2.4 Dynamic state models for single-page applications

When web applications moved to Ajax and later to client-side frameworks, work on model construction moved to run time. Crawljax executes user events against a running application, compares the DOM before and after each event, and builds a state flow graph whose nodes are equivalence classes of DOM states [50]. Later work in this line changed the exploration strategy but not the source of the model [51, 74]. The survey by Kertusha et al. reviews web testing research from 2014 to 2025 [36]. Of the model-based tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application.

The reason is largely historical. In an Ajax application there was no declarative route table, and one uniform resource locator (URL) could represent many states, because navigation was implemented by handlers that modified the DOM. Static analysis of JavaScript existed [29], but recovering navigation from it was hard, and nothing in the source represented the routes explicitly. The dynamic approach observed directly what the source represented only implicitly.

Client-side routing frameworks changed this situation. The route table is now a declared data structure in the source. A navigation is an attribute or a method call that names a route. A guard is a named function attached to a route. Four static analyses of such a framework were found during the literature search for this thesis, three of them reading the framework's own sources and one a compiled bundle. Kurant's component dependency graph for React stops at the component rendering graph without addressing routing [38].

The other three do address routing. Oshima and Kishi extract page transitions from an Angular application in order to model-check them with SPIN [56]. Their extraction reads the transition information from the route declarations in `RouterModule`, reads the navigable paths from every component, and resolves the parent–child relationships among the root component, the routed component and the custom child components. That attribution step is scope attribution. A navigable path is attributed to the routed component that contains it, and the root component's own paths are attributed to every page. Section 5.3.3 measures that combination, as the *rooted* level of the analysis presented here (the levels are defined in Section 4.2).

What distinguishes the present work is the input the method accepts and what it claims about its output. The authors state that the applications they handle must not use the `children` attribute or route parameters in the routing declarations. They also state that changes arising from data binding are not treated as page transitions. Those exclusions remove nested route trees and parameterised addresses. How the method treats a navigation whose destination is an expression rather than a literal is not stated in the paper, and it is not assumed here. A screen is identified by the name of its routed component rather than by an address, so path composition does not arise. Guards, lazy loading and named outlets are not part of the model. A navigation site that reaches a screen through a method call or through a dispatch lies outside the attribution mechanism. The model carries no modality, so a navigation that is always available and one that is available only if a guard admits it are represented identically. The evaluation applies the tool to four constructed applications and to two published ones of 1824 and 6893 lines, with six and twenty-three pages. It reports whether the intended model was produced and whether transition faults, injected by hand into constructed variants of the first sample application, were detected. What the extraction loses on an application it cannot fully read is not measured. This is the question this thesis takes as its subject.

Two recent works read a client-side router configuration statically for other ends. Rima et al. parse Angular sources and templates to build a graph of navigation structure, interaction sites and the conditions (guards among them) that decide whether an interaction is executable. They enumerate workflows over that graph and compile each workflow into a browser test. The graph is a coverage denominator for generated end-to-end tests [64]. This account is taken from the publisher's abstract, the operational specification in the project's replication package (`softscanner-sw/softscanner_fa`, `docs/paper/approach.md`), and the anonymised manuscript in the same repository (`docs/paper/main.pdf`). The published chapter was not available to this review and may differ from both documents. Both documents describe attribution beyond containment: the components active at a route are those activated by the route and its ancestor routes, closed under composition, plus the bootstrap components. This covers what the *rooted* and *outlet* levels of Section 5.3.3 place. A handler is followed into helper methods of the same class to depth one and into the first callback of a `subscribe`. The specification excludes router-outlet modelling and provider resolution beyond module and root providers. It does not follow a call into a service, so a navigation inside a service member is not placed on the screens of its callers. Zhang et al. parse the compiled JavaScript bundle rather than the sources, taking declared routes as vertices and navigation calls as edges, and check an access-control

policy against the screens reachable from an authorised one. Their graph is declared an over-approximation and is not narrowed; no navigation site is attributed to the screens it can fire from, and guards are not represented [81]. Rima et al. compare their output with a ground truth built by hand for the applications they study. Neither work classifies the losses by construct or measures them across a corpus. The problem addressed in this thesis is therefore neither new nor settled. Section 2.10 states what none of these works does.

The work reported here is located between these analyses and the dynamic tools described above.

2.5 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 reads Angular source with the TypeScript compiler application programming interface (API), so it works on a source-level abstract syntax tree rather than on a compiled artefact. This section reviews that tooling style, the abstract interpretation work behind lattice-based analysis of JavaScript, what the type annotations of a TypeScript program contribute, and the evidence that frameworks rather than the language limit such analyses in practice.

2.5.1 Analyses over a source-level syntax tree

Analysing the syntax tree of the source is a long-established style, used for example by Baxter et al. to detect code clones in a C system of about 400 000 lines [8]. It keeps the analysis close to what the developer wrote, which matters when the result has to be reported back at a source location. SPOON makes this style into a reusable library for Java [58]. The TypeScript compiler API occupies the same position for TypeScript: it gives a typed syntax tree, a symbol table and a type checker. This is what allows the Angular adapter to resolve a constant reference in a route table or the declared type of an injected field. The cost of the style is that the analysis sees the program as written, including the parts that a compiler would later rewrite or erase.

2.5.2 Abstract interpretation of JavaScript

Lattice-based analysis of JavaScript itself has a formal treatment. Jensen et al. present a static analysis based on abstract interpretation that infers sound type information for the full JavaScript language [29]. It shows that the framework of Section 2.6 applies to JavaScript at the level of individual values, a considerably harder problem than the one addressed here. Scope attribution (Section 3.2.2) uses the same machinery over a much coarser domain than heap objects.

2.5.3 What TypeScript's types contribute

Angular applications are written in TypeScript, so an analysis can read declared types instead of inferring them. The type system is deliberately unsound. Bierman et al. give a precise definition of it [10], and Richards et al. describe the annotations as unsound by design [63]. That deliberate unsoundness means a declared type is evidence about the program rather than a guarantee about its execution.

The empirical picture is consistent with that. Both Flow and TypeScript detected only about 15% of fixed bugs in JavaScript project histories when annotations were added to the buggy versions [25]. Across 604 GitHub projects, TypeScript was associated with better code quality and understandability but no lower bug proneness [11]. The declarations that describe untyped libraries can themselves be wrong: TSCheck found 142 errors in the declarations of 10 libraries [22].

What follows for an analysis of Angular code is a division of labour. A declared type narrows the set of candidate implementations of a call, which is useful and is what the call graph construction in Section 2.7 relies on. It does not establish that the call reaches exactly one of them, so an unresolved or ambiguous type has to widen the result rather than be trusted. In the graph model this appears as the difference between the must tier and the may tier.

2.5.4 Frameworks, not the language, are the obstacle

The practical obstacle to analysing a client-side application is the framework around the code, not the language. Madsen et al. observe that JavaScript applications depend on large frameworks and libraries whose behaviour a static analysis has to know. Hand-written models of them are tedious, incomplete and sometimes wrong, so they infer library behaviour from the way the application uses it [48]. A later analysis by Madsen et al. addresses implicit control flow directly with an *event-based call graph* for Node.js that represents listeners that run when an event fires [49]. An Angular application has the same shape of problem, with the router, template event bindings and observable subscriptions in place of the Node.js event emitter.

Two further static analyses of a framework in the Angular family bear on this work. The one that addresses navigation is Oshima and Kishi's, discussed in Section 2.4. The other, which does not, is Aurebesh, by Ocariza et al. [55]. It detects identifier and type inconsistencies between models, views and controllers of a model–view–controller (MVC) framework. The work concerns AngularJS 1.x, a different framework from the Angular versions this thesis analyses, with a different template language, component model and injector. What the two have in common is the problem of relating an identifier in a template to the code that defines it.

2.6 Formal foundations

The analysis builds on three ideas from the literature: a transition system in which edges carry a modality, a stated position on what soundness can mean for a practical

analysis, and dataflow analysis as the computational framework.

2.6.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required (*must*) or merely permitted (*may*) [42]. The graph model in Chapter 3 adopts this distinction and adds a third label, *unknown*, for edges whose modality could not be determined. The distinction has practical consequences. The thesis makes a soundness claim about the must tier only. The evaluation treats the tiers differently, since the evidence behind a may edge establishes less than the evidence behind a must edge.

With a single edge kind, an analysis that cannot decide whether a transition occurs must either emit the edge and lose precision, or drop it and lose recall. With a modality the edge records how much was established, and a consumer can select the tier it needs. The tiers are reported separately in Chapter 5 for this reason.

2.6.2 Soundiness

Livshits et al. argue that practical static analyses are sound only with respect to a subset of the language. They argue that this is unavoidable in practice. They also argue that the appropriate response is to state explicitly which language features are approximated rather than to claim soundness in general [47]. The thesis states its own position in Section 3.1. The literature reviewed in Sections 2.5 to 2.7 gives the specific reasons why a general claim is not available for this setting. Type annotations in TypeScript are deliberately not guarantees. Framework configuration decides which implementation a call reaches. The firing condition of a reactive computation is not written in the program text.

2.6.3 Dataflow analysis and abstract interpretation

Kildall showed that many program analyses can be stated as the computation of a fixpoint over a lattice of abstract values. Information is combined with a join operation and propagated along the edges of a graph until nothing changes [37]. If the lattice has finite height and the transfer functions are monotone, the computation terminates and its result does not depend on the order in which nodes are visited. Cousot and Cousot gave the general theory behind this approach as abstract interpretation [16], and Nielson et al. give a textbook account [54]. Scope attribution in Chapter 3 is an instance of this framework; its abstract values and join are defined in Section 3.2.2.

A particular instance needs a lattice of bounded height and a propagation graph. Where the propagation graph is incomplete, because a call could not be resolved, the result is incomplete in a way the framework does not detect.

2.7 Angular routing, dependency injection and reactive state

The analysis presented in Chapter 3 operates on Angular applications, and most of its difficulties arise from particular Angular mechanisms. This section introduces those mechanisms and relates each of them to the program analysis problem it creates. The Angular documentation is the reference for the framework details [27]. Listings 2.1 to 2.3 show the constructs discussed below in a small example, which later chapters return to.

Listing 2.1: A route table with a guarded route, nested children, a lazy route table and a constant path.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root template: the header sits next to the outlet.

```
<app-header />                <!-- shell: rendered on every screen -->
<router-outlet />             <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Note. Its links are present on every screen but belong to none of them.

Listing 2.3: Navigation from an injected service.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  private session = inject(SessionStore);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

Note. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

2.7.1 Routes

An Angular application declares its screens as a tree of route objects. Each route has a `path` pattern and, in addition, either a `component` to render, a `redirectTo` target, or a list of `children`. A route that has children but no component acts as a path prefix and does not itself correspond to a screen. Routes can be loaded lazily using `loadChildren` or `loadComponent`, in which case the child routes or the component are defined in another module or file. Routes may also have guards, namely `canMatch`, `canActivate`, `canActivateChild` and `canDeactivate`. Guards are functions or injectable classes that return a boolean, a redirect, or a Promise or Observable of one. A `canMatch` that returns false makes the router try the next matching route rather than cancel the navigation. Routes may also have resolvers, which fetch data the router awaits before it activates the route. Route paths are strings, but they are not necessarily string literals. Applications that keep their route table in one place often write `path: routes.about.path`, and resolving such a path requires following the constant to its definition.

2.7.2 Components and templates

Each screen is rendered by a component, which is a class together with a template. A template can instantiate other components by their selectors, so the components of an application form a nesting graph. A template can also contain a `<router-outlet>` element, which marks the position where the router inserts the component of the currently active child route. Components that are above the outlet in the nesting graph make up the application shell and are present on every screen, whereas components below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make the nesting conditional. Templates can additionally be passed around as `TemplateRef` values, and components can be created programmatically at run time.

2.7.3 Navigation sites

In a template, navigation is expressed with the `routerLink` directive, whose value is either a literal path or an array of path segments. In class code, navigation is performed by calling `Router.navigate` or `Router.navigateByUrl`; the target may be given relative to the current route through the `relativeTo` option. Both kinds of navigation site can appear in any component, in any injectable service, and in guards, interceptors and resolvers. The source code does not state anywhere which screens a given site can be triggered from; this information has to be derived.

2.7.4 Dependency injection

Angular creates services and passes them to the components and services that declare them as dependencies. This is dependency injection, a form of inversion of control: a class does not construct or look up its collaborators, but receives them from a

container [32, 24]. A dependency is requested by type or by an injection token, and the same token can be bound to different implementations in different injector scopes, using `useClass`, `useExisting` or a factory function.

The pattern is widely used outside Angular: Yang, Tempero and Melton detected its use from source code in 34 open-source Java applications [75], and Laigner et al. catalogue 12 dependency injection anti-patterns and detect them with a static analyser [40]. Dependency injection configuration is therefore readable statically; it does not state which binding a given call site will see.

For a static analysis this turns a simple method call into a dispatch problem. A call such as `this.nav.goHome()` can only be linked to the method it invokes after the type of `nav` and the binding of that type in the relevant injector have been resolved. This is the call graph construction problem known from object-oriented languages, where a virtual call must be resolved to its possible targets. Class hierarchy analysis resolves such a call using the declared type and its subclasses [17], rapid type analysis narrows the result to classes that are instantiated [5], and points-to analysis narrows it further [2]; Grove and Chambers place these in a common framework [28]. For JavaScript, Feldthaus et al. show that an approximate call graph is precise enough for practical tools [23].

Dependency injection adds a further difficulty, because the binding of a type to an implementation is not written in ordinary code but in framework configuration. The Java literature has measured what this costs a general analysis and what recovers it. Sridharan et al. report that the heavy use of reflection in frameworks makes static taint analysis ineffective, and that adding support for one framework at a time does not scale. Their tool instead generates a specification of framework behaviour from the application code and its configuration files, which an unmodified taint engine then consumes [67]. Antoniadis et al. capture the concepts that enterprise frameworks share, such as components and entry points, in a declarative form and measure about four times higher completeness [3]. Landman et al. put a bound on what can be hoped for: reflection cannot be ignored in 78% of Java projects, and resolving it requires unsound assumptions [41].

The closest analogue to resolving Angular's injectors is Jasmine, by Chen et al. [13]. Spring's dependency injection and aspect-oriented programming rely on dynamic language features, so a conventional call graph of a Spring application is missing edges. Jasmine extends Soot so that the call graph of a Spring program becomes more complete. The structural problem is the one that occurs in Angular. An injected field has a declared type. The binding of that type lives in framework configuration. An analysis that does not read the configuration cannot connect the call to a body. The difference is in where the configuration lives and how far its effect reaches. Angular binds tokens in provider arrays that belong to a module, a route or a component, so the same token can resolve to different implementations depending on where the injection happens. Resolution is relative to an injector rather than global, which makes the provider hierarchy part of the input to the analysis.

The analysis follows the advice this literature converges on, which is to model the framework explicitly rather than to treat its configuration as ordinary code (Section 3.2.3).

2.7.5 Reactive state

Recent versions of Angular include signals and effects. A signal is a value that changes over time, and an effect is a computation that runs again whenever a signal it read changes [26]. This is an instance of reactive programming, which goes back to functional reactive programming [21] and has been surveyed by Bainomugisha et al. [6]. In reactive systems the dependencies between values are recorded at run time, and the runtime propagates updates without explicit calls in the program text. So when a navigation call is placed inside an effect, the condition under which it fires is a data dependence in a reactive graph and not a control-flow condition. A static analysis can find which signals the effect reads, but in general the source does not determine when the effect runs. The analysis of this thesis does not attempt to bound it. Angular applications also use observables from Reactive Extensions for JavaScript (RxJS) for the same purpose. Section 2.8 reviews what the reactive programming literature establishes about this boundary.

2.8 Reactive programming and the limits of static dataflow

The reactive style gives the runtime, rather than the program text, control over when a computation runs. This section reviews the propagation discipline that reactive runtimes implement, and the evidence about how well reactive code can be reasoned about with tools.

2.8.1 Propagation order and glitch freedom

A reactive program is a graph of values in which an update to one value has to be propagated to the values that depend on it. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language, so that an expression evaluates to a node in a dataflow graph [15]. Their account names the central hazard: a *glitch* occurs when a signal is recomputed before all of the signals it depends on have been brought up to date, which makes an intermediate value observable that no consistent state of the program ever had. Their runtime prevents glitches by updating signals in topological order, which requires the dependency graph to be acyclic. Drechsler et al. carry glitch freedom into a distributed setting [20]. An Angular signal graph, as described in Section 2.7.5, records its dependencies as they are read, so it is the kind of structure that these propagation algorithms are defined over.

The consequence for a static analysis is that the order in which reactive computations run is a property of the graph that the runtime builds, and the graph is built from the reads that actually happen. The set of signals a given effect may read can be over-approximated from the source. The number of times the effect runs, and the state of the application when it runs, depend on which writes occur and in what order, so they cannot in general be determined exactly from the source. A static analysis could still over-approximate them, and the one presented here does not try.

2.8.2 Reactive code and the tools developers have

The empirical literature on reactive programming reports a split between reading code and reasoning about its behaviour. Salvaneschi et al. ran an experiment with 127 subjects comparing reactive programming against object-oriented code using the Observer pattern, and found comprehension significantly better in the reactive version [66]. The same group reports that the tool support is the weak side. Reactive programs, they argue, need debugging concepts of their own, because a conventional debugger shows a call stack while the question is about a dataflow graph. Their Reactive Inspector implements such concepts and outperformed conventional debugging in a controlled experiment [65].

The same picture holds for the Rx family of libraries, which Angular applications use for asynchronous data. Banken et al. report that conventional debuggers have no support for Rx and that developers fall back on inserting console logging. Their RxFiddle visualises the structure and the values of an Rx dataflow, and in an experiment with 111 developers it let them finish debugging tasks faster [7]. Alabor and Stolze interviewed 10 professional engineers and observed 4 of them debugging RxJS code. They found that the engineers rely on manually added trace logging rather than on specialised tools. The study is a workshop paper with a small sample, so it indicates rather than establishes the practice [1]. The surface that a tool would have to cover is large: Zimmerle et al. mined RxJava, RxJS and RxSwift usage on GitHub and Stack Overflow. They found that 95.2% of the Rx operators are used in practice. The stream abstraction itself is the largest of the 23 Stack Overflow topics they identified, at 36.4% of the posts [82].

An analysis that aims to recover navigation from an Angular application inherits this boundary. A navigation placed in an effect or in a subscription has a firing condition that is a data dependence in a reactive graph. The read set is visible in the source. The firing is not determined by it exactly, and this analysis does not approximate it. Chapter 3 therefore specifies the modality *unknown* for such edges and keeps the read set as the symbolic guard. This is the treatment the soundiness position of Section 2.6 prescribes for a construct that cannot be decided. No pass in this version emits that modality for an effect or a subscription. The unknown edges it does emit are those whose destination it cannot resolve to a screen.

2.9 Mutation testing

RQ4 asks whether the extracted graph responds to source changes in the constructs it represents, as change impact analysis requires. The method used to answer it is mutation testing, applied in an unusual direction. The mutants are inserted into the application source. The artefact whose reaction is measured is the extracted graph rather than a test suite. This section reviews the technique, its use on web applications, and the reporting practice that Chapter 4 follows.

2.9.1 The coupling effect

Mutation testing inserts small syntactic changes into a program and measures how many of them a test suite detects. The original argument for it is the *coupling effect* stated by DeMillo et al., namely that tests which uncover simple errors are often also effective at uncovering much more complex errors [19]. Without that assumption a mutant would say nothing about the faults a developer actually writes.

Jia and Harman give the standard survey of the field [30], and Papadakis et al. survey the following decade of work and list the problems that remain open [57]. The evidence that mutants are a usable proxy comes from Just et al., who found with 357 real faults that mutant detection correlates significantly with real-fault detection, independently of code coverage [33].

2.9.2 Mutation for JavaScript and web applications

Mutation analysis has been adapted to JavaScript and to web applications, where the cost of generating and running mutants is a central concern. Mirshokraie et al. guide mutant generation with static and dynamic analysis towards code that is error-prone or likely to affect the output, and propose mutation operators specific to JavaScript and the web [52, 53]. Praphamontripong and Offutt define operators for faults in the way web components interact, in a workshop paper [59].

On the tooling side, Stryker is a mutation testing framework for JavaScript and TypeScript. No peer-reviewed publication describing it was found, and the project does not cite one. Its documentation describes more than 30 supported mutations and a runner-agnostic design [68]. The absence of a paper is the reason the mutation procedure in Chapter 4 is specified in the thesis rather than delegated to a tool's defaults.

2.9.3 Reporting a mutation experiment

Papadakis et al. also give best-practice advice on the use of mutation in empirical studies of testing [57]. Chapter 4 follows it by stating the operator set, how the mutants were produced and sampled, which mutants were excluded and why, and the detection rate for each operator separately instead of one aggregate figure.

This use of mutation puts the thesis in the change impact analysis tradition rather than in test adequacy measurement. Arnold and Bohner define a framework for comparing impact analysis approaches [4], and the collection they edited is the standard reference for the area [12]. Chianti is the classic tool of this kind: it decomposes the difference between two versions of a program into atomic changes and reports, for each affected test, the changes that affect it [61]. Lehnert's taxonomy, a workshop paper, classifies impact analysis approaches in more detail [43]. A transition graph whose edges carry source witnesses supports the same kind of query at the level of screens, and the mutation experiment measures whether the graph changes when the source it was derived from changes.

2.10 Summary and the gap

Static extraction of navigation models from web applications has been directed largely at server-rendered frameworks, in which a page is a file and a link an anchor; client-side routing frameworks have mostly been modelled dynamically instead. The literature search for this thesis found three static extractions of a client-side navigation graph, two from a framework's own sources and one from a compiled bundle, each building its graph for a different consumer. Oshima and Kishi's is the one that fixes the terms of the problem addressed here (Section 2.4). It does so, however, only for applications that declare no nested routes, no route parameters and no data-bound navigation. Its attribution is computed over component containment and the root component's own paths. It puts no modality on the transitions. It makes no measurement of what the extraction fails to read. The other two put the graph to work without narrowing it. One uses it as the coverage denominator of a generated end-to-end suite [64]. The other uses it as the reachability relation of an access-control check, declared an over-approximation [81]. Rima et al. already read nested and lazily loaded route trees and attach guards to edges as constraints. What this review found no earlier work combining is attribution that follows calls across classes and into services under their provider bindings, a modality on each transition that separates what is unconditionally available from what a guard or an unresolved scope makes uncertain, and a measurement of the resulting loss, classified by construct, on applications large enough for the difficult constructs to occur. The Android literature is the nearest static relative. It attributes navigation by resolving the owning object, and when that resolution is imprecise it falls back on runtime traces. A navigation site in a web application frequently has no owning object that could be resolved. In Android the owning object of a handler usually identifies the window, so attribution reduces to object resolution in the common case. Fragments, background tasks and aliased widgets are the exceptions, and the Android work handles them with dynamic evidence. That is why an Android technique cannot simply be reused.

The literature reviewed in Sections 2.5 to 2.9 fills in why the problem is open and what is available for addressing it. The analysis machinery is not missing. Abstract interpretation of JavaScript is established [29]. Source-level analysis over a typed syntax tree is a standard tooling style [8, 58]. The Java literature has shown repeatedly that framework behaviour can be modelled explicitly to recover call edges that a general analysis loses [67, 3, 13]. The other peer-reviewed static analysis in the Angular family targets AngularJS 1.x and checks template and controller consistency rather than navigation [55]. The type information that TypeScript adds narrows a dispatch without settling it, because the type system is unsound by design [10, 63]. The reactive literature marks where the boundary of the approach lies. This is because the firing of a reactive computation is decided by a graph the runtime builds and not by the program text [15, 7]. Mutation testing supplies a method for asking whether an extracted model reacts to source changes [30, 57]. For a statically extracted navigation graph of a client-side framework, it has been applied only to faults injected by hand into constructed applications [56]. An operator-level mutation study of such a graph was not found in this review.

Table 2.1: Prior work closest to this thesis, by target, extraction mode, guard representation and evaluation basis.

Work	Target	Static	Guards	Evaluation basis
Yang et al. [79, 77]	Android windows	yes	no	20 apps; manual on 6
Yang et al. [78]	Android callbacks	yes	context	manual vs. dynamic
StoryDroid [14]	Android activities	yes	no	hand-built, 10 apps
GoalExplorer [39]	Android screens	hybrid	no	manual, 36 apps
ProMal [45]	Android windows	hybrid	no	40 apps, F1
UI-CTX [44]	Android widgets	yes	branches	2000 labelled widgets
Ricca and Tonella [62]	server-rendered pages	semi	yes	manual
WebUml [9]	server-rendered pages	hybrid	yes	case study
Tkachuk and Rajan [69]	Struts, JavaServer Pages (JSP)	yes	outcome	not stated
Crawljax [50]	Ajax DOM states	no	no	manual
FeedEx, FRAGGEN [51, 74]	DOM states	no	no	coverage
Kurant [38]	React components	yes	n/a	F1, no routing
Oshima and Kishi [56]	Angular pages	yes	no	6 apps, faults
Rima et al. [64]	Angular workflows	yes	conditions	7 apps, tests run
Zhang et al. [81]	SPA routes, from bundles	yes	no	4 apps, vs. ZAP

Note. Evaluation basis is a reference, a metric or the tool compared with, as each work states it.

Chapter 3

Static navigation analysis and scope attribution

This chapter defines the graph model (Section 3.1) with its one soundness claim, then scope attribution, the analysis proposed in answer to RQ3 (Section 3.2), and the extractor that implements part of it (Section 3.3).

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, namely the first and the last component of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge. That edge carries the strongest modality among them, with* M *ordered* $unknown < may < must$*. So* E *holds at most one modality for each* (s, σ, g, s')*. The paragraph on witnesses below says how the constructs are traced.*

Nodes outside the model. The extractor emits two node kinds that are not screens, and the edge relation does not range over them. An *unplaced origin* stands for a component, with the navigation sites in it, that the analysis could not place under any screen. A *dynamic target* stands for a destination the extractor records as

Table 3.1: What the published graphs are made of: nodes by kind and edges by modality.

	nodes			edges		
Application	screens	anchors	other	must	may	unknown
realworld	10	2	0	9	56	0
alfresco	27	1	26	0	36	19
ghostfolio	84	6	96	0	266	137
k8s dashboard	67	1	76	125	85	103
bitwarden	98	4	147	1	434	143
mifos	606	14	195	24	14018	197
All	892	28	540	159	14895	599

Note. An anchor stands for a whole subtree of screens; *other* counts unplaced origins and dynamic targets together. These are the attributed graphs; the node column of Table 5.2 counts the graphs of the same snapshot without attribution, which hold no anchors and fewer *other* nodes.

unresolved. Both are kept for two reasons. An unplaced site is a loss that Chapter 5 has to count. A destination the analysis cannot name is one the run time could still take. Table 3.1 reports how much of each graph they are. 540 of the 1460 nodes in the corpus are of these two kinds, 37.0 % of them. 599 of the 15653 edges reach a dynamic target. A further 73 edges leave an unplaced origin and are held at *may*. They therefore sit inside a tier that Chapter 5 reports, although their source is not a screen. An edge into a dynamic target asserts that its source screen has an outgoing navigation whose destination the analysis could not name, and nothing about which screen is reached. Claim 3.1 and must conversion (RQ1) range over the edges whose source and target are both screens.

Modalities. A *must* edge is claimed to fire for every input on which the enclosing code executes. A *may* edge is possible but conditional. *Unknown* marks an edge whose modality could not be determined, and carries the destinations the analysis cannot name. In the measured graphs it labels only edges into the `u_` sink of Section 5.5, which records that a screen is left by an event whose target the extractor records as unresolved. The pinned extractor produces 599 such edges of the 15653 in the corpus, in 5 of the 6 applications. Carrying this modality and reaching a node that is not a screen coincide on every corpus edge, and the census refuses to report a graph in which they come apart. Must conversion (RQ1) and Claim 3.1 are defined over the other two tiers, so the unknown tier is counted in the edge totals and in no rate. The model also assigns *unknown* to navigations inside signal effects (Section 3.2.5); no pass of this version emits one.

The artefact also records a numeric *confidence* on each edge (1 for a must edge, at most 0.6 for a may edge, 0 for an edge into a dynamic target). It is not part of the model, and every figure is reported over the modalities.

Guards are stored as source text and are never evaluated by the tool, which keeps G opaque: the model does not need to represent application state.

Screens and routes. The set S is derived from the route configuration, and not

every route contributes a screen. Write *Routes* for the set of routes the application declares and *component*(r) for the component that route r renders. Then

$$S \;=\; \{\, \mathit{screen}(r) \;\mid\; r \in \mathit{Routes} \text{ and } \mathit{component}(r) \text{ is defined} \,\},$$

where *screen*(r) is the node for route r, identified by the full path that concatenates the paths of r and of its ancestors. Two kinds of route contribute no element of S:

- A route that declares `children` but no `component` is only a path prefix (Section 2.7). Its path is carried into the full paths of its children.
- A route whose only effect is a `redirectTo` renders nothing.

In Angular a redirecting route is only a redirect: the router refuses `redirectTo` together with `children` or a `component`. This was read off the router's configuration validator in Angular 18.2.14, the vendored version, and is not true of every router: Vue Router applies a redirect only when its record is the last one matched, so a redirecting parent does render its component for a matching child (Section 6.4).

Angular also permits two component-bearing routes with the same full path, told apart at run time by `pathMatch` or a `canMatch` guard. The model assumes this does not occur: the extractor keeps the route declared first and drops the later ones, silently for Angular and with a note for Vue and React (Section 6.4). Two routes with different paths that render the same component give two screens, which is the case that makes the attribution of Section 3.2 necessary.

Witnesses. Every edge of static or runtime origin carries a witness: the file and line of a construct that produced it, or an observation identifier. Where several constructs produce the same edge, one is kept as the witness and the others are recorded as merged witnesses (Section 4.2). No edge is added to the base graph without a witness; manual edges stay in a separate overlay. The first evaluation method checks this mechanically.

The soundness claim. The soundiness position that frames such claims is argued in Section 2.6.2. The thesis claims only the following.

Claim 3.1 (Soundness of the must tier)**.** *Every must edge corresponds to a navigation that occurs, for every input on which the enclosing code executes, over the declared route set, under the route-order assumption of Section 6.3, and for a site whose component is not placed by a join over an ambiguous selector, within the following execution model: the event is a plain primary activation that navigates the current browsing context, no other listener the event reaches starts a navigation, and no call the reader cannot resolve to a body in the application, nor any work the handler schedules for later, starts one.*

The execution model is part of the claim and not a detail of the implementation. Section 6.3 lists the cases it excludes: a call into a library that cancels the navigation by starting another, a second listener on the element or an ancestor, and an activation with a modifier key or into another browsing context. On such an execution a must edge can name a navigation that does not complete, and the claim says nothing about it.

The route-order assumption of Claim 3.1 carries much of the tier: without it the must tier holds 33 of its 159 edges, on 22 of 25 constructs.

What the evaluation tests. The evaluation tests a necessary condition of this claim rather than the claim itself. The distinction decides how the figures of Chapter 5 may be read. The witness check of Section 4.3 re-reads the source at every location an edge cites and asks whether the construct found there names the destination the edge names. A must edge cannot satisfy the claim if the construct that makes it must names a different destination, so a failure of the check refutes the claim there. Agreement does not establish it. The other constructs a must edge cites are evidence for may and are not judged against the claim.

Demotion conditions. Whether the navigation occurs *for every input on which the enclosing code executes* is decided by the rules that place an edge in the must tier instead of the may tier. Table 3.2 lists every condition that demotes a site to may.

Table 3.2: Conditions that demote a navigation from the must tier to the may tier. Every condition yields may.

Group	Condition	Example construct
Guards and resolvers	The navigation runs a guard or resolver, whatever it would decide	`canActivate` on a route it enters; a resolver, which demotes as an `await` does
	The guard list cannot be brought back to an array	a guard list built at run time
Target and source addresses	The target is the address the source already displays	navigating to the current URL, which the router ignores or follows only when a parameter or the query differs (held at may as a matter of policy, the safe side)
	The source is never displayed at its own address	a redirect answers the source's address
	Two declared routes, or a route and a redirect, can answer the address	two routes matching one path
	The base a relative target resolves against cannot be read	a relative target with an unreadable base
	The URL names an auxiliary outlet	a URL with a named-outlet segment, whose route the adapter does not read
Conditional execution	The site is under a template control-flow construct	a control-flow block, a structural directive, a container that renders lazily
	The site is in a template fragment that need not render	`<ng-template>`, an International Components for Unicode (ICU) case
	The site is under a branch, loop or fallback in code	a branch, a loop, a `catch` clause, an optional call, a default value, a callback
	The site follows an exit in its block	after an `await`, an early `return` or `throw`, a `break` out of its block
	No handler calls the enclosing function unconditionally	a module-level function, or a member of its class

Group	Condition	Example construct
	The component is mounted conditionally	a component under a template condition on that screen; a route component behind a conditional outlet
A later navigation may cancel it	Code that runs after the site, before the event is handled, may navigate	the rest of the expression holding the navigation (the body of a call it is an argument of, a right operand, a branch of a conditional whose condition it is) the statements after it in its block and in every block around it, and a `finally` block around it what runs once its function returns: what each caller in the class does after calling it, handler or not; for a constructor or lifecycle hook, the hooks Angular calls after it

A navigation runs the guards the router runs. These are the `canMatch` of every route its address is matched through. They are also the `canActivate`, `canLoad` and resolvers of every route it enters (`canLoad` is taken as run every time). Next come the `canActivateChild` of every route above those. Last come the `canDeactivate` of every route it leaves. A route the source and the target share is re-run only if its parameters can change or its `runGuardsAndResolvers` policy directs. A shell or anchor edge is judged per screen and drawn per screen where the screens disagree.

A handler is a member that the framework calls itself: a lifecycle hook, or a member that `@HostListener` registers. A handler is also a member that the template binds to an event as a bare call outside any template condition.

The router cancels a navigation when another starts before it has finished. Code that runs after a site may navigate when it holds a navigation call, in a branch or a callback as much as at the top, or a call to a member of the class, a function of the file or a method of an injected service whose body does. An awaited navigation has finished before the rest of its function runs, so what follows it there does not count, but its callers go on as soon as the function first awaits, so what they do still counts. A call the reader cannot resolve to a body in the application is taken not to navigate (Section 6.3).

An exception from an earlier synchronous call is not counted, since every call can throw; an `await` is counted, because a rejected request is an ordinary outcome. The router's fetch of a lazily loaded route is not counted: a failed chunk load is a fault of the deployment.

How far the rules are checked. Unit tests cover each rule in both directions, and a mutation sweep deletes each rule in turn and checks that some test fails (Section 5.5). The tests share the extractor's reading of each construct, and no second classifier re-derives them. The constructs of the final must tier were read against the source by the assistant the declaration describes, with the extractor's decisions in view; this checks that each decision can be defended and is not an

independent second reading (Section 5.5). Some defects in the condition rules were found by an earlier review of a development version's tier and not by the witness check (Section 4.3.2). What the witness check establishes is therefore narrower than the claim. It establishes that every cited site exists in the source and agrees on its target. The stronger half of the claim rests on the demotion rules and is not independently verified.

Scope of the claim. This is the only soundness property claimed in the thesis, and it holds only relative to the declared route set. No such property is claimed for the may and unknown tiers. The witness of a may edge shows that the corresponding call site exists in the source, not that the transition is ever taken. This difference matters for the evaluation. Whether the target found at a witness location agrees with the target of the edge is a meaningful question for must edges, and for may edges whose target is a literal. It is not a meaningful question for edges the extractor produced by over-approximation. Treating it as one leads to spurious failures (Section 4.3.2, Table 4.5).

The may tier as an upper bound. As in a modal transition system [42], the may tier is read as an over-approximation. The must and may edges together are meant to contain every transition the application can take between its declared screens. An excess may edge, one the application never takes, is admitted by this reading and costs only precision. A transition the application takes and the graph lacks is the defect. No completeness theorem is claimed. The end-to-end check (Section 5.3.5) measures how far the graph falls short of the transitions the suites take; the structural references (Section 5.3.4) measure only how far it falls short of the analysis's own relations. Section 6.3.1 sorts what is still missing by whether it lies inside the static boundary.

3.2 Scope attribution

What is specified and what is implemented. Scope attribution is specified here as a fixpoint over four *input relations*:

- component nesting (N): which component's template instantiates which;
- router outlets: which components lie on the path from the root to an outlet, and below a route's component;
- the call graph with dependency injection (CG);
- dispatch (D): the relations that carry NgRx effects (NgRx is the Angular state-management library) and router event subscriptions.

The fixpoint also uses two derived relations. *Mem* links each component to its members. T links each callable that holds a navigation site to the entry guards that navigation runs. The prototype implements three of the four input relations: nesting, outlets and the call graph. The dispatch relation D is specified but not implemented. The guard relation T is not implemented either, and a stand-in replaces

it (Section 3.3). The evaluation of Chapter 4 therefore measures this three-relation prototype and not the full model.

Three further parts belong to the model only: guards that return a `UrlTree`, the injector hierarchy, and navigation inside signal effects and observable subscriptions, which the model labels *unknown* (Section 3.2.5). The extractor approximates or omits them as Section 6.3 lists. A site that no implemented relation reaches contributes no edge and is counted in the loss category "Attribution, service" of Table 4.3.

3.2.1 The defect being addressed

Without scope attribution, a navigation site is attributed to a screen only if it occurs directly in the component that the route for that screen renders. Navigation sites anywhere else are attributed to no screen. This one cause explains three symptoms:

- shared components contribute no edges, because the application shell places the header next to the router outlet and not inside the subtree of any screen;
- a component mounted at several routes contributes one edge instead of one per route;
- navigations from injected services yield no edges.

These are the symptoms of the baseline, the extractor with the attribution pass disabled (Section 3.3). On it no application is without edges.

3.2.2 Scope

This subsection is the definition of scope attribution that the rest of the thesis points to.

The lattice. Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is an element of the powerset $\mathcal{P}(S)$. The abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y \;=\; X \cup Y,$$

the powerset lattice over the screens. Its least element is the empty scope $\bot = \varnothing$, a site that fires from no screen at all. Its greatest element is $\top = S$, every screen.

True, computed and reference scope. Two different things are called a scope, and the rest of this thesis keeps them apart.

- $\mathit{scope}^*(x)$ is the *true scope* of a site x, the set of screens from which x really can fire. It is a property of the application and not of any analysis.
- $\mathit{scope}(x)$ is the *computed scope*, the set of screens the extractor emits edges from for x. In the model it is the fixpoint of Section 3.2.3. The prototype widens it with fallbacks that are not constraints of the model (Section 3.3).

A screen in $\mathit{scope}^*(x) \setminus \mathit{scope}(x)$ is a scope too small, and a screen in $\mathit{scope}(x) \setminus \mathit{scope}^*(x)$ is a scope too large. No true scope is established anywhere in this thesis. For two applications a third set is read off the sources by hand: the *structural reference* $\mathit{scope}^R(x)$. It applies the analysis's own relations without its implementation (Section 5.3.4). Agreement between *scope* and scope^R shows that the relations are implemented as stated. It does not show that either equals scope^*. Everywhere else the thesis reports the computed scope and says so.

The empty scope. The empty computed scope is likewise two situations, and conflating them would hide a defect inside a property of the application.

- A site is *unreachable* when $\mathit{scope}^*(x) = \varnothing$: the code exists and no screen can reach it, so contributing no edge is the correct answer.
- A site is *unresolved* when $\mathit{scope}(x) = \varnothing$ because no rule of the analysis reaches it (a navigation in a service nothing is known to inject, or behind a construct the reader could not follow). The true scope is simply unknown.

Both yield $\bot$, so $\bot$ alone does not say which holds. The extractor therefore does not let an unresolved site vanish silently. A site in a component that no rule places is anchored at an unplaced origin, a node that carries the component's file. The site's edges are held at *may*. A site in any other component whose scope is empty contributes no edge. The extractor records a note naming its file and the reason. Either trace makes the unresolved sites countable in Chapter 5 rather than indistinguishable from unreachable code.

Named elements. Three elements of the domain are named, because attribution anchors edges at them. Write $r' \succeq r$ for the reflexive ancestor relation on routes: r' is r itself or an ancestor of r in the route tree. The three elements are

$$\{\mathit{screen}(r)\}, \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

and they form the chain

$$\bot \;\sqsubseteq\; \{\mathit{screen}(r)\} \;\sqsubseteq\; \mathit{subtree}(r) \;\sqsubseteq\; \top.$$

The chain is stated for a route with a component; for a prefix route only $\mathit{subtree}(r)$ is defined.

These names are a representation device for anchoring edges, not a restriction of the domain: the join of two scopes is their union, so a component mounted at two unrelated leaf routes has the scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, none of the three named elements. The names matter for storage, where an edge anchored at $\top$ or at $\mathit{subtree}(r)$ is kept once rather than once per screen. Table 3.3 assigns a scope to each Angular construct.

Computing the scopes. The row for services is why a simple traversal of the component tree is not enough. A service has no scope of its own: its scope is the join of the scopes of the places that call it, which may themselves be services. Attribution is therefore computed as a fixpoint over the relations of Section 3.2.3. The outlet relation gives the initial scopes. Scopes are propagated along the nesting relation

Table 3.3: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$\mathit{subtree}(r)$
component mounted at routes $r_1, \ldots, r_k$	$\mathit{subtree}(r_1) \sqcup \cdots \sqcup \mathit{subtree}(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
member (method, property initialiser, lifecycle hook) of a component	the scope of the component
method of an injectable service	join of the scopes of its callers
`canActivate`, `canActivateChild` or `canMatch` on route r	join of the scopes of the navigation sites with a known target in $\mathit{subtree}(r)$, an approximation of the screens a navigation into r starts from
`canDeactivate` on route r	$\mathit{subtree}(r)$
HTTP interceptor	$\top$
NgRx effect for action a	join of the scopes of every dispatcher of a

N. They pass from each component to its own members along Mem. They pass along the call graph CG and along the dispatch relation D. They pass from each navigation site to the entry guards of the routes its target lies under along T. This is a monotone dataflow problem in the sense of Kildall [37, 54], and two properties of it do separate work.

- Finite height gives termination. The height of $\mathcal{P}(S)$ is $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$.
- Monotonicity gives the result. The iteration reaches the least solution of the constraint system, independent of the order in which nodes leave the worklist. The transfer functions are monotone because the only operation they apply to a scope is a join, and union is monotone in both arguments.

The relations may contain cycles, since mutually recursive templates are legal, and the worklist handles them. The restrictions, fallbacks and stand-ins of Section 3.3 are computed outside the worklist.

3.2.3 Input relations

Four relations, in order of increasing difficulty, provide the input to the fixpoint.

- *Component nesting.* $A \to B$ when A's template instantiates B's selector,

resolved within the correct import scope. Matching selector text globally is unsound, since two components may share a selector string in different scopes.

- *Router outlets.* Components on the path from the root to the outlet form the shell and take $\top$. Components below the component of route r take its scope $\mathit{subtree}(r)$. A named outlet carries a second mount, which makes auxiliary routes representable without separate treatment.

- *Call graph with dependency injection.* A call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17], with Angular's provider declarations as the source of bindings, as recommended for framework-heavy code [3]. Where a token admits several implementations the analysis joins over all of them, which is one source of *may* modality.

- *Dispatch relations.* A dispatch is treated as a call-graph edge from every dispatcher to every handler, which admits NgRx effects and router event subscriptions under the same fixpoint.

Algorithm 1 gives the computation, and its propagation steps are the transfer-function applications bounded above. A step adds a screen to a scope only when an edge of N, Mem, CG, D or T carries it there, and scopes only grow. What the algorithm computes is therefore the least solution of the constraint system that Table 3.3 states. In the model a component that is both mounted by a route and rendered in the template of another component holds the join of the two scopes. Section 3.3 reports where the prototype departs from this. It does not propagate into a component that a route mounts. It also adds fallbacks that are not constraints of the table, so its result is not the least solution of this system or of any restriction of it.

3.2.4 Guards as program points

A guard that returns a `UrlTree` is itself a navigation, typically to a login screen. Its source is not the guarded route. A `canActivate` or `canMatch` guard on r runs while the router leaves the screen the navigation started from. The redirect edge therefore leaves that screen, the origin of an edge into $\mathit{subtree}(r)$. Only a `canDeactivate` guard redirects from a screen of $\mathit{subtree}(r)$ itself. Treating guards only as labels on inbound edges misses this edge class entirely.

Further, the guard of an edge in the model is the conjunction of the structural conditions on the path from screen to call site and the guards the router runs on that navigation (Section 3.1). The prototype does not materialise that conjunction in the measured graph: a structural condition or an enclosing `if` demotes the edge to may but is not written into its guard, so the guard of a screen-to-screen edge holds only the router's guards. The evaluated graph therefore records that a navigation is conditional without recording the condition. Only an optional mode that no table of Chapter 5 uses keeps the enclosing condition.

3.2.5 Boundary

A navigation inside a signal effect has a firing condition that is a data dependence on a reactive graph. As Section 2.7 notes, only the read set is statically visible. The model gives such edges the modality *unknown* and records the set of signals read by the effect as the symbolic guard. No pass in this version emits one. The analysis does not try to determine when these edges fire. It marks this limit explicitly instead of giving a result that looks more precise than it is.

3.2.6 Anchored edges

If the result of attribution were stored explicitly, a single link in the application shell would produce one edge for every screen; for the largest application in the corpus the bound is the product of its screens (Table 5.2) and its 1343 constructs. Anchoring removes the factor that comes from the source scope. Each edge is stored once, anchored at its scope, and the set of edges leaving a screen is computed when requested:

$$\mathit{edgesFrom}(r) \;=\; E_{\top} \;\cup\; \bigcup_{r' \succeq r} E_{\mathit{subtree}(r')} \;\cup\; E_{\{\mathit{screen}(r)\}},$$

where E_X is the set of edges anchored at scope X. The term for $r' = r$ contributes $E_{\mathit{subtree}(r)}$ itself. The query is made for a screen, so $\mathit{component}(r)$ is defined, while r' ranges over all ancestors of r, prefix routes included. An edge whose scope is some other screen set is stored at each of its screens and reaches $\mathit{edgesFrom}(r)$ through the $E_{\{\mathit{screen}(r)\}}$ term.

Claim 3.2 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set.*

The claim is tested rather than proved: Section 5.3 compares the two sets on the corpus and on fixtures.

Relative targets. A site attributed to several screens fires on each of them, but a relative target does not resolve against each of them. Angular resolves a `routerLink` without a leading slash, and a `Router.navigate` call relative to the injected route, against the route that renders the component holding the site. A component's template renders on every screen below its route, so a link in it that fires on a child screen still resolves against the component's own route. Write $\mathit{holder}(s, c)$ for the nearest route at or above screen s whose component is c. A relative target at a site in component c that fires on s resolves against the path of $\mathit{holder}(s, c)$. Where no route on that chain renders c, as for a component nested in a template, a full target's base is not read and its edge is held at may (Table 3.2); a target prefix falls back to the address of s, and its edges are may edges as every prefix edge is.

3.3 Implementation

The extractor, *uigraph*, was built for this thesis and is available as open source. It consists of a framework-independent core, which defines the graph model and the

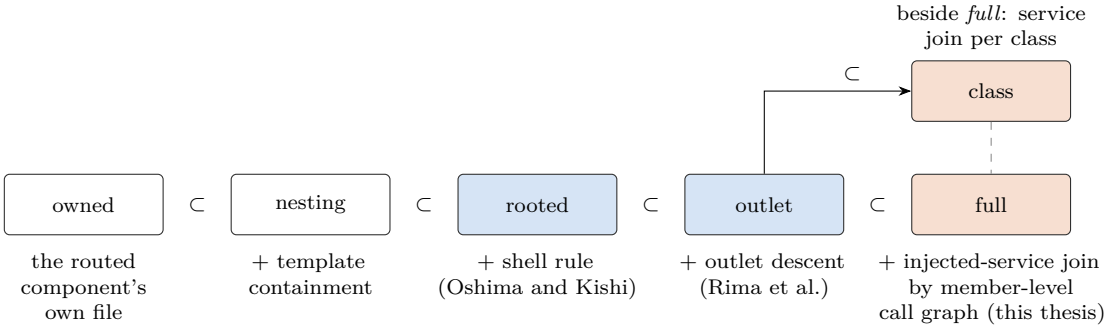


Figure 3.1: Attribution levels, from the ownership and containment baselines to the service join.

Note. Blue: the levels that reimplement the two earlier Angular extractions compared in Section 5.3.3. White: the ownership and containment baselines, the first being the direct ownership of the Android literature. Orange: what only this thesis adds; *class* is the per-class variant of the service join.

operations over graphs, and one adapter per framework that maps source constructs onto the shared model. The Angular adapter reads route configuration, templates and class code with the TypeScript compiler API and emits screens and edges for `routerLink` attributes and `Router.navigate` calls, each with the rule identifier and source location that produced it as witness.

Scope attribution is a separate pass in the Angular adapter, enabled with the option `--attribution`. With the option off, the extractor produces exactly the baseline edge set, as checked on every application in the corpus. The pass implements component nesting, router outlets and the call graph; D and T are not implemented, and stand-ins replace them (below).

The pass is switched on cumulatively in the evaluation (Section 5.3.3), at the levels of Figure 3.1. Each level adds to the one before it, and the figure marks which levels stand for earlier work and which only this thesis adds.

Component nesting. Every component in the application, excluding test hosts and stories, is read with its element selectors and template. An edge $A \to B$ is added when A's template contains an element with B's selector. When a selector belongs to more than one component, the candidates are narrowed to those that A's file imports. If none is imported, the analysis joins over all of them and records a note. Edges from such a join are not capped at *may*, so a site in one candidate can be placed on a screen that renders another. Claim 3.1 excludes such edges (Section 6.3). The corpus has such joins in k8sdash and bitwarden. No must edge of the final graphs cites a construct placed through one. An attribute selector is matched the same way, so a directive has a scope as a component does.

Router outlets. The route tree is recorded during route extraction. A route component's template is rendered on its own screen and on every descendant screen, so its navigation sites get the scope $\mathit{subtree}(r)$. A component is the application shell when it is not routed, is not placed on a screen by the nesting relation and holds the primary `<router-outlet>`. Everything it renders gets the scope $\top$. A component that is not routed and not referenced, but whose subtree contains the outlet, is most

likely mounted by a route the extractor cannot see, for example through a helper function that wraps a route table. It is also treated as shell. Its edges are capped at *may*, so that Claim 3.1 is not put at risk by a scope that may be too large.

Departures from Algorithm 1. The pass departs from the algorithm in two respects.

The first is that it does not propagate a scope into a component that a route mounts. A routed component keeps $subtree(r)$ and does not also receive the scope of a component whose template renders it. Traversal does pass through a routed component, so a component nested beneath it still receives the scopes of the components above. Taken alone, this restriction can only shrink a routed component's scope, never enlarge it. Two other rules can enlarge a scope beyond the model's:

- components that nothing references and no route renders, but whose subtree holds the primary outlet, are treated as shell and given $\top$, with their edges capped at may;
- an ambiguous selector that no candidate's import resolves joins over all candidates, which Claim 3.1 excludes.

A scope that is too small loses edges and cannot invent them, which is why the restriction cannot put Claim 3.1 at risk. It costs recall. The constructs it strands are counted in Chapter 5.

The second departure is in how the call graph of Section 3.2.3 is read. The pass ranges over the members of classes, and a member of a component runs on that component's screens. A call `this.f.g()` is resolved through the declared type of `f` (a constructor parameter, typed field, `inject(T)` field or `@Inject(TOKEN)` parameter). The call resolves to:

- the class that type names;
- the classes a provider binds it to with `useClass` or `useExisting`; or
- for an abstract class no provider binds, its subclasses in the application.

This is class hierarchy analysis [17] restricted to the declared type. Bindings are read across the whole application rather than relative to the injector that resolves them, so where two injectors bind a token differently the join covers both. This can only widen a scope. A factory or value provider is not read. A call on a type declared outside the application contributes no edge.

The seeded call-graph iteration on its own reaches the least solution of its constraints. The scopes emitted for service sites add two fallbacks to it:

- a member no caller reaches falls back to the join over every component that injects its class, the per-class approximation that the *class* level of the ablation applies to every service (Section 5.3.3);
- a member the iteration does reach receives more screens where a caller the graph does not see reaches it: the join of that caller's class is added, or this class's own join if nothing places the caller.

Neither fallback is a constraint of Table 3.3. The first is not monotone in the call graph, since adding a call edge can narrow a scope. The emitted service scopes are therefore the least call-graph solution widened by these fallbacks, and not the least solution of the constraints in Table 3.3. Service sites are held at *may* either way, because the calls on the way carry conditions the analysis does not follow.

Anchored edges. Edges with scope $\top$ are stored once at a node of kind `scope`. Edges whose screen set is exactly *subtree*(r) for a screen r with at least one descendant are stored once at a scope node listing the members of that subtree. A prefix route that is not itself a screen is not anchored. Other screen sets are stored per screen. The core provides the query *edgesFrom*(r) and a function that materialises all anchored edges.

Stand-ins for D and T. A navigation in an NgRx effect has no scope from the model itself, because D is not implemented. Two approximations stand in where the model has a relation the pass does not implement. Section 5.3.3 withholds them below the *full* level of Figure 3.1, so no lower level counts their sites.

- *Guards (for T).* A guard is given the screens of the routes that name it, as a bare identifier, in `canActivate`, `canActivateChild`, `canMatch` or `canDeactivate`: the screen of each such route, not *subtree*(r), and nothing for a componentless prefix. For a `canDeactivate` guard that is narrower than the model's scope. For an entry guard it is not the model's rule, since the redirect leaves the screen a navigation starts from. The stand-in can therefore be too small where navigations into the route start elsewhere, and too large where they do not start at the route itself.

- *Effects (for D).* An effect, being registered for the lifetime of the application, is given $\top$.

Edges from both are capped at *may*. A class the call graph does not reach and nothing injects still has no scope. The evaluation counts those in the loss category "Attribution, service" of Table 4.3. One change was made to the baseline itself: for components with an inline template, the witness now points at the construct's own line rather than the template's first line; on the corpus the baseline graphs with and without this change are identical. Unit tests cover each case above, including that the baseline is unchanged when the option is off.

3.4 Summary

The chapter defined a graph of screens and edges with three confidence labels and one soundness claim for the must tier, relative to the declared route set, the route-order assumption and an execution model; the evaluation tests only a necessary condition of it. Scope attribution assigns each navigation site a set of screens as a least fixpoint over four input relations. The prototype implements three of them, replaces the guard relation by a stand-in, and widens the scopes with fallbacks that are not part of the model. Chapter 4 describes how the analysis is measured.

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$.

Require: routes R with route tree; for each route r its entry guards $\mathit{entry}(r)$, named in `canActivate`, `canActivateChild` or `canMatch`, and its `canDeactivate` guards $\mathit{leave}(r)$; components C; nesting $N \subseteq C \times C$; HTTP interceptors I; call graph CG; dispatch relation D; guard relation T, linking a callable x to $g \in \mathit{entry}(r)$ when x holds a navigation site whose target lies in $\mathit{subtree}(r)$

Ensure: $\mathit{scope}(x) \in \mathcal{P}(S)$ for every component and callable x; the callables include the members of components and of services

for all x **do** $\mathit{scope}(x) \leftarrow \varnothing$
end for
for all $r \in R$ and c the component of r or a layout wrapping it **do**
    $\mathit{scope}(c) \leftarrow \mathit{scope}(c) \cup \mathit{subtree}(r)$ ▷ own screen and outlet descendants
end for
for all c on the nesting path from the root component to the holder of the root-level primary outlet **do**
    $\mathit{scope}(c) \leftarrow S$ ▷ application shell, $\top$
end for
for all $r \in R$ and $g \in \mathit{leave}(r)$ **do**
    $\mathit{scope}(g) \leftarrow \mathit{scope}(g) \cup \mathit{subtree}(r)$ ▷ *model only*: runs as the user leaves r
end for
for all HTTP interceptor $i \in I$ **do** ▷ *model only*: registered for the application
    $\mathit{scope}(i) \leftarrow S$
end for
$W \leftarrow$ every x with $\mathit{scope}(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        Propagate(x, y)
    end for
    for all members m of x, if x is a component **do** ▷ a member runs where its component is shown
        Propagate(x, m)
    end for
    for all y with $(x, y) \in CG \cup D \cup T$ **do** ▷ CG: member level, bindings read application-wide; D, T: *model only*
        Propagate(x, y)
    end for
end while
procedure Propagate(x, y)
    if $\mathit{scope}(x) \not\subseteq \mathit{scope}(y)$ **then**
        $\mathit{scope}(y) \leftarrow \mathit{scope}(y) \cup \mathit{scope}(x)$; add y to W
    end if
end procedure

Note. The relations marked *model only* belong to the full analysis and are not implemented in the prototype evaluated in Chapter 5; the prototype's other departures, among them the stand-ins for guards and effects and the rule for routed components, are listed in Section 3.3.

Chapter 4

Evaluation setup and research material

This chapter describes how the analysis of Chapter 3 is evaluated: the corpus (Section 4.1), every ratio and measure (Section 4.2), and the five evaluation methods with what each can and cannot show (Section 4.3). Chapter 5 reports the results.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria: a real application rather than a library or template; a nested, lazily loaded route tree; guards present; end-to-end test configuration for a browser driver; retrievable history in which tests and source change together; and an active repository. Six candidates were retained (Table 4.1), each fixed to a specific commit so that the measurements can be repeated on the same code.

Criterion (4) was checked as configuration, not as a usable suite. At the pinned commits only alfresco has specifications the test-source cross-check can read, so that method covers alfresco alone. Ghostfolio and bitwarden carry no browser-test configuration at all, so for them the criterion was not met. It was therefore not applied uniformly: `gravitee` was rejected for a thin UI suite while two applications with none were kept.

Four candidates were rejected; the counts were read from the repositories and are not generated. `angular/components` is a component library with no product router. `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none the user interface. `gravitee` declares 618 routes but has 14 UI specifications. `SAP/spartacus` has a large Cypress suite, but its routing is configuration-driven in a way the extractor does not support; that reason is tool support, which is not one of the six criteria, so this rejection rests on a ground added after screening began.

Route-tree size and end-to-end test density were inversely related: no application provides both a large route tree and execution-based ground truth. The methods that need test suites are applied only to the applications that have them, and the other methods to all six.

Table 4.1: The corpus: commit, route count and role in the study of each application.

Application	Commit	Routes	Role in the study
gothinkster/angular-realworld-example-app	`dd99ed2c`	14	size floor; all literal paths
alfresco/alfresco-content-app	`b88d47f4`	≥57	87 Playwright specs including a navigation suite
ghostfolio/ghostfolio	`9884fa0b`	≥135	constant-valued route paths; self-hostable
kubernetes/dashboard	`0ba796d9`	140	mid-size control
bitwarden/clients (web)	`e40ce6f0`	≥168	many guards, guard factories among them
openMF/web-app	`7f781796`	≥909	size ceiling

Note. Routes are the route entries reached by the independent route parser of Section 4.3, as it stands in the final version (Section 5.5). A count marked ≥ is a lower bound: the parser records a blind spot for that application, so the entries behind it are not in the figure; Table 5.3 gives how many blind spots each application has. The marks are on alfresco, ghostfolio, bitwarden, mifos, on these grounds: children not resolvable to a route array; matcher route: path not statically knowable; spread route source: not followed.

4.1.2 Validity of an open-source corpus

Kalliamvakou et al. show that many GitHub repositories are inactive, personal or not software projects, and recommend filtering candidates by explicit criteria and reporting them [34, 35], which the selection above does. The remaining threats are external validity (open-source applications may differ from proprietary ones) and a possible selection bias, since I selected the corpus and wrote the extractor. The selection protocol is therefore reported in full: the criteria were fixed before any results were obtained, but one candidate was rejected on tool support and two retained applications do not meet the test-suite criterion. The corpus spans nearly two orders of magnitude in routes, and the commits and harness are released so the measurement can be repeated.

None of this addresses a further threat. uigraph was developed on this corpus and is measured on it (Section 6.2), so the figures are fitted to it. A case study on a proprietary Angular application, reporting only aggregate figures, is planned as the most direct response (Chapter 6). There is no standard benchmark corpus for single-page applications either [36], so the corpus and its manifest are an output of the thesis.

4.2 Metrics

Each research question is answered with one main measure. Table 4.2 lists every ratio the thesis reports, and the paragraphs after it define the terms the table uses.

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code, in the application source excluding tests. Constructs are found by a separately written parser that shares no code with the extractor. Table 4.5 qualifies what that independence covers: for the route-path forms added during development, the two resolve the same constructs by the same rules, and three of the independent parser's corrections were made because it disagreed with the extractor.

Must conversion (RQ1). Must conversion is the fraction of in-scope constructs that are witnessed by at least one must edge. Three rules make it precise.

- A construct is matched to a witness on its own line or on a line either side, to absorb offsets in the extractor's locations. Two constructs on adjacent lines can therefore both be credited by one witness, so the ratio is an upper bound at line granularity.
- Query-only navigations and the two exclusions of Table 4.3 are left out of the denominator as well as the numerator.
- When several constructs produce the same edge, the extractor keeps the most certain as the witness and records the others as merged witnesses, each with the modality it earns on its own. A construct counts as converted only where it earns must itself.

The other ratios. More than one of these measures has been called coverage in the literature this thesis builds on, but they are not interchangeable: on this corpus they differ by more than an order of magnitude (99.0 % of sites accounted for against 1.4 % converted to must). Node coverage rises if less of an application is read, so the blind-spot count is printed beside it. The recovered share widens the must-conversion numerator to an edge of any tier. The accounted share alone measures whether the losses are declared, not how much is recovered. Coverage without a qualifier means node coverage; the gap in recovered constructs is called the recovery gap, and the loss tables measure it.

Agreement between a scope and the rendered interface (RQ3). The measures above cannot be asked the question the contribution turns on: is the set of screens that attribution computes for a site the set that site can really fire from? The *rendered-interface check*, run on the harness of the runtime route oracle (Section 4.3), answers one direction. The harness navigates to each screen the graph holds, reads the in-app links the DOM carries and compares them with the edges the graph's per-screen query yields there.

A link rendered at a screen where the graph has no such edge is unambiguous: the site was attributed to too few screens or never read. An edge the graph has and the DOM does not show is weaker, since a template may render links conditionally and

this harness signs in to nothing; where a target could not be resolved to one address, every address the static prefix admits is recorded at reduced confidence, which is not a claim that the screen links there. This direction is therefore judged only over predictions that name a destination exactly, and their number is published. Three bounds apply. Only template-sourced sites are judged, since a `Router.navigate` call in an event handler leaves nothing in the DOM. A screen whose guard redirects to a sign-in page without a backend renders another screen's links and cannot be judged; such screens are counted apart, and on this corpus they are most screens (Section 5.3.1). Ghostfolio's per-locale build does not fit the harness, so it is measured by every other method and by none of this one.

The ablation and the structural reference (RQ3). Two measurements ask whether the attribution computes a set of screens that agrees more closely with a structural reference than a simpler analysis does, or only a larger one.

The ablation withholds the mechanisms of Section 3.2 one at a time and extracts the whole corpus at each level:

- *Owned*: a site belongs to the screen whose routed component's own file holds it, the direct ownership of the Android literature.
- *Nesting*: adds component containment through template element selectors.
- *Rooted*: adds the shell rule, so a component the root renders is attributed to every screen. Oshima and Kishi assemble their page information from the root component as well (Section 2.4), so this level measures how far the prior work reaches here.
- *Outlet*: adds router-outlet descent.
- *Full*: adds the member-level call graph, with its fallback join and the stand-ins for guards and effects.

Each level adds to the one before it, so the *full* step is not one mechanism alone. A sixth level, *class*, runs beside the nested ones and replaces the call graph with the per-class rule, which joins a navigating service over every component that injects its class. The harness checks on the real graphs that the levels nest: a (site, screen) pair present at one level and absent at the next is a defect in the switch, and no table is written.

The structural reference is scored in both directions, which no other method allows. For one application, the screens each navigation site fires from were read off the sources by hand, with a derivation per site, in `route-oracle/ground-truth/realworld.json`. It is the structural reference of Section 3.2.2, called the ground truth below. A pair the reference has and a level does not is a scope too small by the reference; the converse is a scope too large. Four limits apply.

- On realworld it is one application, small enough to read exhaustively. A second, alfresco, is read under the same rules (Section 5.3.4).

- It is the author's reading, so it is not independent of the tool's author. It is published with its reasoning so a reader can check it.
- Its rules are the analysis's own relations, and a template condition or guard that admits a screen only in one sign-in state does not narrow a scope under them. The ground truth therefore tests whether the implementation applies the relations correctly, not whether they match what the running application renders.
- It is checked in one direction only: the scorer reports every site the graph places that the ground truth omits.

The end-to-end navigation check (RQ3). The structural reference applies the analysis's own relations, so it cannot show that they miss a navigation the application performs. This check takes its reference from execution. Two applications ship an end-to-end suite that can be run here. The first, realworld, runs against the public demonstration backend, which is not pinned. The second, mifos, runs against a local Fineract server with the image pinned by digest. A small module added to each application logs every navigation the router completes while the suite runs, with the route patterns of the screen left and the screen reached.

An observed pair is such a source and target pattern for a navigation the application itself triggered; back-button navigations and the first screen after a full page load are excluded. A pair is *contained* if any edge, must or may, joins the two screens once scope nodes are expanded to their members, with parameter names ignored. An edge to an unknown target never counts. A hit is any such edge, not the edge of the site the test used, so the check does not test the scope of an individual site. Redirected navigations are compared on the address they asked for, as preregistered. The scorer's first departure from this rule is listed in Table 4.5, and its figure is not used.

A missed pair is classified from the graph alone, by rules over the graph so that no assignment is made by hand. The classes are tested in this order, and the first match wins:

- *redirect*: the navigation was redirected and the graph holds the pair on the address it landed on (the preregistered redirect class);
- *base*: the target is reached from a screen above the source but not from the source, as a relative link resolved against the wrong route appears;
- *dynamic*: the source screen's routed component, or one above it, holds a site whose target the extractor records as unresolved;
- *other*: none of these.

The base and dynamic classes were defined after the misses had been read. Because of the order, the dynamic count is what is left after base. On mifos every miss met the dynamic test whatever the order, so that count said little alone; Section 5.3.5 names the sites, which were repaired after the check.

The counting rules, three validity checks and predictions for each application were written down before any log was read, except for that departure. The checks are that the instrumented and the plain suite pass and fail the same tests, that at least ten distinct pairs are observed, and that a second instrumented run observes the same pair set. The check measures recall on the transitions the suite exercises and nothing else. The graph is an over-approximation, so that recall is containment, and its weight depends on how many (screen, screen) pairs the graph holds, which is reported beside it. A graph edge no test takes is not an error, so the check cannot measure precision, and one run cannot refute a must edge.

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct that is in scope, cited by no must or may edge and not accounted for is *lost*. It is assigned to exactly one loss category by the first rule in Table 4.3 that applies. The categories therefore partition the lost constructs. Named outlets are removed before the assignment, and guards are measured outside it. The categories Constant target, Node missing and Other were not put through the admission rule that maps a category to a defect class (Chapter 5).

Guard defects do not appear in the table, since a wrong guard mislabels an edge rather than removing it. No method here checks a guard label directly; guard handling is exercised only by the guard-removal mutation operator and by the source review of Section 5.5. The guard defect was found by red-team review of the graph and the source, not by any check above.

The nested branch decides a category from the *owning* route component, so a construct can be assigned to a node category and still be converted once attribution gives it a scope through the shell or a subtree (Section 5.3). The size of *other* counts constructs left for inspection by hand, not one cause.

Change in recovery and must-tier precision (RQ3). Must conversion and the loss breakdown are measured on one commit of the extractor, run twice on identical inputs with the attribution pass disabled and enabled. With the pass off the extractor reproduces the baseline edge set exactly on every application (Section 3.3).

Must-tier precision is the fraction of must edges that meet two conditions. The witness line contains a construct of the claimed kind with the claimed target, *and* the destination node sits at an address the independent route set declares; otherwise a node minted at an address the application never mounts would pass.

An edge whose destination the check cannot read is neither a pass nor a failure, so precision is stated as bounds: the share of must edges that pass and the share that do not fail. RQ3 is answered positively only if the recovered share and must conversion, over the same construct denominator, do not fall, the recovered share rises, and must-tier precision does not fall. This is read as no detected failure in either run, not as a precision that did not fall where the check reads only part of the tier. The bounds of the two runs cover different sets of must edges, so they are not compared as rates.

Detection rate (RQ4). The detection rate is the fraction of injected source changes that produce the predicted change in the graph, reported per mutation operator with its n. A change whose construct the graph does not contain is excluded

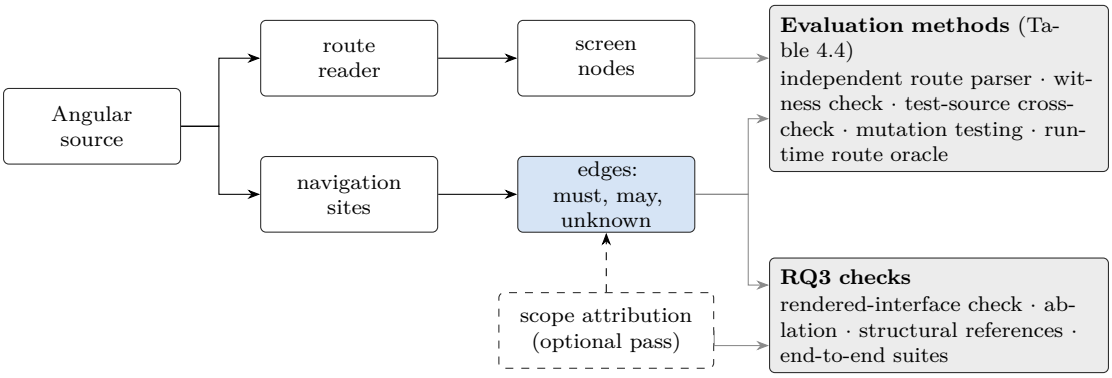


Figure 4.1: What is evaluated: the extractor, scope attribution and the evaluation methods that check the resulting graph.

Note. The extractor reads route tables into screen nodes and navigation sites into edges, tiered must, may or unknown by the demotion rules; scope attribution, an optional pass, places sites that belong to no single screen. The five methods of Table 4.4 check nodes and edges against independent readings; the RQ3 checks examine the attribution.

and not counted as a miss, since it can only repeat a recovery gap that the loss tables measure. The rate is therefore conditional on recovery, and the number of excluded mutants is reported alongside it.

Cause trace (RQ5). Each defect class of Table 5.4 is traced to its cause in the source, with one of three outcomes: the final version reads the cause (the loss was the tool's and not the application's), it does not yet read it although a static reader could, or the cause lies beyond a static reader of a route table. The result is a list of causes by outcome, not a ratio. Every site that yields no transition still carries a recorded reason (Section 5.5).

4.3 Evaluation methods

Figure 4.1 shows where the methods act on the extraction pipeline. Five methods are used; RQ3 adds four further checks (the rendered-interface check, the ablation, the structural reference and the end-to-end check), which Table 4.4 does not list. None establishes correctness alone, so Table 4.4 states what each can show and where it is blind. Figure 4.2 shows which part of the graph each method checks and where its ground truth comes from.

Four of the five read the application's sources, and two readers of the same text can be wrong in the same way. The fifth, the runtime route oracle, was added for that reason.

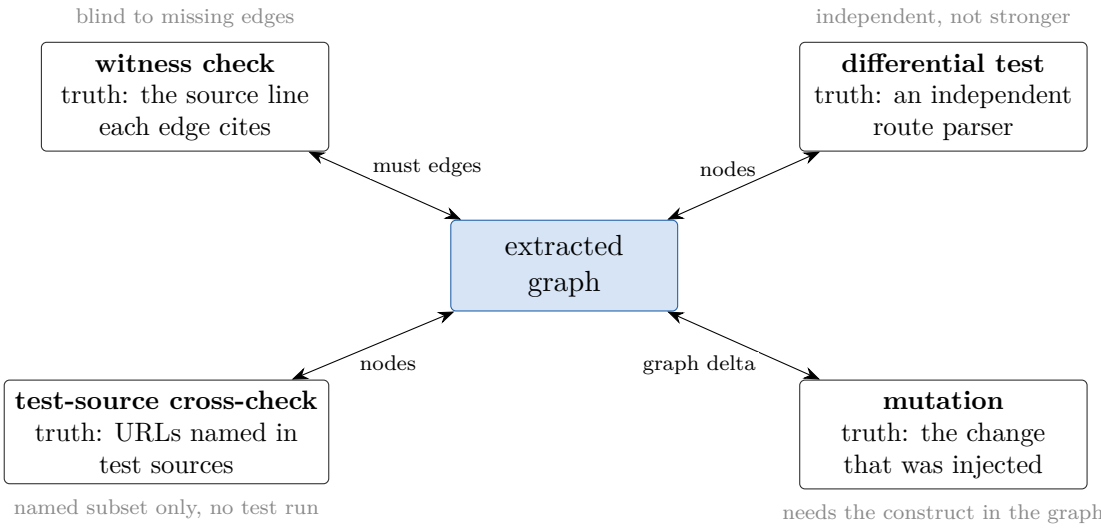


Figure 4.2: The four evaluation methods that read the applications' sources, the graph part each checks, and its ground truth.

Note. The fifth method, the runtime route probe, is not drawn because its ground truth is the running application rather than any part of the graph. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.4.

Table 4.4: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-tier precision. For every must edge, the witness file is re-parsed at the claimed line, the construct kind and target are compared, and the destination node's address is tested against the independent route set.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists. Where both tools read a table at the wrong prefix, they agree on a wrong address (Section 5.5.3). Target agreement is literal, so an edge to the screen a redirect lands on is reported as a failure.
Differential test	Disagreements with a separately written route-configuration parser.	Diagnostic, not a metric. The independent route parser is independent, not stronger, and for the path forms added during development (Table 4.5) it is no longer independent of the extractor at all.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Test-source cross-check	Node-level recall against a fourth reader of the source. The URL literals in the application's end-to-end specifications are collected; a URL its own test suite names is an address its authors expect to exist, and the extractor is asked whether it minted a node there.	Reads test source text and runs nothing, so a collected string is an address a test *names*, not one a test was observed to reach. Tested subset only. Textual extraction under-samples click-driven navigation. A direct `goto` is not a UI transition, so visit pairs are not counted as edges.
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a recovery gap and is excluded, as is a guard mutant on a screen that no edge enters. A component several routes render maps to every screen those routes declare, which errs towards scoring a mutant rather than dropping it. Detection is conditional on recovery, and the attributed and unattributed graphs do not exclude the same mutants, so they are also compared over the mutants both can score, paired by operator, file, line and subject. The sample is drawn from the graph, so it is not independent of the artefact under test.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Runtime route oracle	Node-level soundness against the framework itself. Each application is built in its development configuration, loaded in a headless browser, and asked for the route configuration the Angular router holds, with every lazily loaded subtree expanded through the application's own `loadChildren`. The addresses the extractor mints are compared with the addresses the framework mounts. Nothing re-reads the route files. Which addresses count as mounted is itself a reading, so the harness navigates the running application to a list of addresses and records what the recogniser made of each, before any `canActivate` guard runs and after the `canLoad` and `canMatch` guards that take part in matching.	Requires the application to build and boot without its server. It reads the configuration and not the rendered screen, so a route that a guard always refuses still counts as mounted. An address claimed through a `UrlMatcher` cannot be decided from the configuration. A workspace application that composes the measured library mounts the shell's own routes as well, so part of any difference is scope and not reading. Its result files record no extractor commit or runtime version (Section 4.3.3).

4.3.1 Preconditions

Two properties of the extractor were checked before any measurement. *Determinism*: two extractions from the same unchanged source give identical nodes and edges. *No accumulated state*: extracting into an existing database gives the same graph as extracting after deleting it. Without them, differences seen in mutation testing could not be told from noise.

4.3.2 Corrections made to the protocol

The final figures are those of the final version of the protocol. uigraph was developed within this thesis, and three versions are named: the *early development version* (commit f17a302), before the corrections the evaluation prompted; the *intermediate version* (extractor eb9dab80, with a must tier of 20 edges, tables as printed at 7d109c7); and the *final version*, which the chapters present. Of the intermediate version only the must-tier figures are cited, as printed; the runs behind the printed tables were lost, and an archived re-run of the same extractor gave a different loss

ledger. The figures of superseded instruments cannot be regenerated with the shipped harness, and none is cited as a result. A surprising figure is checked against its inputs before it is written down, since such a figure has more than once proved a property of the measurement and not of the extractor.

The mutation harness was corrected during development after its misses had been read, so the final run is not an independent test. The final run used the same seed as the run whose misses were read but a partly different sample. 46 of its 73 mutants are at the same location (operator, file and line) as one of the 70 of that run, though the destinations may differ. 6 of the 9 misses that were read are at no location of the final sample.

Table 4.5 lists every correction, one row each, in the order the chapters record them; the full record with commit hashes is in the accompanying repository. In the first column, *figure* marks a correction made after an implausible or unwelcome figure was seen, *panel* one from a red-team panel, *review* one from a source review made with the assistant, *harness* one that changed the mutation harness, and *end-to-end check* a gap in the extractor that the suites of Section 5.3.5 exposed, repaired afterwards with a test for each. The reviews and panels were made with the AI assistant that the declaration at the front of this thesis describes, and every defect they reported was reproduced against the code before it was repaired.

Table 4.5: The corrections made to the instruments.

Found by	What was wrong	What changed	Effect on the figures
Protocol and harness			
One application's recall fell when the rule was tightened.	Test-source cross-check: a URL parameter in a test could match a literal route segment.	A parameter no longer matches a literal segment; a negative test is excluded.	The earlier recall cannot be regenerated; end-to-end recall there is now 71.4 %.
Figure: inspection of the flagged edges.	The first witness check showed a low precision for one application; all flagged edges were may edges from one over-approximated call site.	No extractor change. Over-approximated sites are reported apart.	The low figure described the measurement; the must tier had no failing edge.
Harness: not stated beyond one application's misses.	A mutant whose target is absent from the baseline graph repeats a recovery gap counted elsewhere.	Such mutants form a separate category.	Most of that application's misses left the detection rate.
Figure: a construct count that tripled.	The construct denominator counted the substring `routerLink`, which also matches `routerLinkActive`.	Constructs are counted by the independent route parser, so ratio and loss breakdown share one definition.	alfresco: the independent route parser finds 34; the substring count was about three times larger.

Found by	What was wrong	What changed	Effect on the figures
Figure: coverage reached 100 per cent (mifos).	The first independent route parser skipped wrapper calls, did not follow a lazily loaded module to its routing module, and dropped the prefix of lazy tables.	It reads wrapper calls, follows the routing module, and keeps the prefix.	mifos has 610 declared screens, 58 with a node; the extractor was the same in both runs.
Figure: a ratio that rose implausibly after attribution (mifos).	Must conversion was first must edges divided by constructs.	Computed per construct; edges whose screen set is a subtree are stored once (Section 3.2).	The rise came from how edges were stored, not from evidence. Definition: Section 4.2.
Harness, *review*, *figure*: a review of the harness, and a detection rate that moved by more than twenty points.	The mutation harness patched the first textual match, judged detection on node and edge sets alone, and retargeted only the head of a commands array.	Patches at the construct's offset; detection also on the witness set; retargeting points at another declared route; guard mutants on a screen no edge enters are excluded; a rename counts when the renamed route appears.	realworld's detection rose sharply. Same seed, not the same sample: 70 mutants earlier, 73 in the final run, 46 at one location (operator, file, line; destinations may differ); 6 of 9 earlier misses are not in the final sample. In the earlier run six paired disagreements favoured attribution; five were retarget mutants whose operator replaced only the head of the commands array, and with it corrected the baseline detects them too.
Harness: reading the graph at the sites the rule let through.	The guard exclusion rule located a route by the nearest path literal before `canActivate`, which fails for children, empty paths, constants and template literals.	The route is found by brace-matching over source with comments and strings blanked; a route yielding no screen is excluded.	Four unobservable guard mutants had been scored as misses and explained as a defect of the graph; they are now excluded. The superseded run sampled 70 mutants.
Harness: a change in the extractor, not a figure.	Removed guards were misjudged: a child of a guarded parent lost the parent's guard, and a set comparison read three identical guard texts as unchanged.	A removal is detected when some edge loses a guard, excluded when no baseline edge carries it; guards are counted.	The sampled mutants are unchanged; how a sampled mutant is scored changes.
A table script stopped.	The construct denominator took constructs from commented-out links and code comments.	Comments are blanked before parsing, with the compiler's scanner for TypeScript.	One construct leaves the denominator in each of bitwarden and mifos; nothing else changes.

Found by	What was wrong	What changed	Effect on the figures
The extractor and the deterministic tier			
Panel: red-team panels reading graph and source after commit f17a302; the witness check missed it.	A guard on a route a wrapper function returns was not read, so everything below was published as unconditional.	The guard set of a screen also reads routes a function returns.	At f17a302: 1024 must edges with 26 witness failures, read from that commit's results. Reading the guard cut mifos to 1 on the intermediate version.
Panel, as above.	A `canActivate` element that is a call, such as a guard factory, was skipped.	An element the reader cannot name gates the edge anyway.	bitwarden's deterministic edges into its root no longer read as unconditional.
Correcting the witness check itself.	It opened only the first of several constructs yielding one edge; later citations in bitwarden targeted `/vault`, which the extractor had not read.	It opens every cited site. An edge took the weakest modality of its constructs (replaced below).	Later citations are 12.1 % of cited sites, 54.9 % in the largest application. The intermediate tier held 20 must edges, 20/20 passing, 0 failing.
Review, then *panel*: a review of every construct the intermediate tier cites (20 edges, 30 distinct constructs, cited 36 times). The witness check cannot find this: it compares destinations, not whether a construct runs every time.	The demotion rules were incomplete: a link under `*ngIf`, in `@if`, a callback, a `catch`, or after `await` or an early return was must.	Demotion under every condition of Section 3.1; the panels added members no handler calls unconditionally, components under a template condition, ICU cases and addresses two routes can answer; a later review added code after a navigation that may start another.	32 of the 36 citations ran under such a condition.
Review, as above.	A guard on an empty-path child without a component was not read, and `canMatch`, `canActivateChild` and `canLoad` not at all.	Such a guard gates the screen above; the other three kinds are read.	10 of the intermediate tier's edges, all of bitwarden's, are guarded.
Panel: a red-team panel.	The repair above gated an edge on every guard of its destination, demoting navigations the router lets through.	The extractor computes the guards the router runs for each source and destination: levels left, matched, entered, kept.	Navigations between screens under a shared guarded parent pass; mifos holds 24 must edges again.
Review, as above.	A navigation to the address its source already displays was treated as occurring on every execution.	Such an edge is drawn per screen: must from the others, a may self-loop at the destination.	1 of the intermediate tier's edges is such a self-loop.

Found by	What was wrong	What changed	Effect on the figures
Reading the edges that survived.	The weakest-modality rule discarded a certainty the edge had (realworld's Global Feed link runs whenever its page does).	An edge takes the modality of its most certain construct, its witness; other constructs record their own, and the witness check judges the must tier only on constructs that earn must.	The final tier is 159 edges on 25 constructs, a different set from the intermediate tier, not a smaller one. Superseded tables counted every cited construct as converted; current ones count a construct only where it earns must.
The independent route parser and the denominators			
The disagreements of the first node audit; each gap had scored the extractor against paths the independent route parser had not read.	It did not read a path written through a constant, an enum or `$localize`, a first-party import through `baseUrl`, or a named route array.	It reads all three forms.	Before the last, it saw almost none of kubernetes/dashboard's screens and scored nearly every node the extractor minted there as unmatched.
The dead-link corrections.	It read Storybook fixtures as screens, a sibling application's tables via a tsconfig alias as this application's, and a path in an `Object.freeze` table as unresolvable.	It sets Storybook aside, ignores the sibling's tables, reads the frozen table. Dropping the frozen routes as unreadable was considered and rejected: they were readable one call away.	Declared sets moved in 5 of 6 applications; in 1 of them the count stayed the same.
Fixing the provenance of the comparison.	A file from an interim revision of the parser, dropped into the table directory, was read as the earlier parser's output; the script could not detect this, and an absent file read as a denominator that did not move.	The earlier parser's own committed output is read from the repository.	The bitwarden decomposition built on that file is withdrawn.
Resolving the first node audit's disagreement.	The first-audit extractor reached route tables without their parent prefix and minted a node at `*` per catch-all.	The loss-list repairs: a table not reached, a path or destination named and not written (Section 5.2).	Of 143 nodes with no declared route, 138 were the tail of a declared route and 5 catch-alls. The final version has 2 nodes matching no declared screen, 2 at `*`.

Found by	What was wrong	What changed	Effect on the figures
Not stated; the check was made after the graph had been consulted.	The grounds that exclude a construct from the denominator were tested after the graph had been consulted.	The grounds are decided from the construct's own text first; the comparison refuses to print if a construct changes ground between runs.	The after column had been over more sites than the before column; both are now over the same 1775 sites.
Findings of the runtime route oracle			
The oracle returned two alfresco addresses the framework answers with its catch-all; both static readers read the subtree alike, so 100 per cent coverage hid it.	A route with a named outlet could take the primary address from a sibling, and routes beneath it were minted at the primary address.	Outlet context is carried down the walk; no screen is minted beneath a named outlet.	The extractor declares 0 screens at addresses the framework mounts nothing at. What remains limits recall.
The oracle against the route set as the static readers held it; both readers had arrived at the defect independently.	A lazy route naming a module through a barrel was followed one hop only, and the table scanned at the root.	Both follow the chain to the table, bounded, refusing at each hop a file the route sits in, a file another route loads, and a file whose tables are registered at the root.	10 addresses in alfresco, bitwarden were wrong, the two alfresco ones among them. Node coverage had been 100 per cent against a denominator with the same wrong addresses.
Instrumenting the predicate (commit `d40c8dc`).	The test keeping a lazy route from adopting the application's root table read `RouterModule.forRoot`, so it fired nowhere in the corpus.	Both tools mark root tables per array, from the argument of either registration API.	A guard that never fires had been credited with a property nothing showed it had.
The dead-link census			
Refutation attempts on the dead-link finding.	An earlier correction followed only the redirecting kind of catch-all route.	Both kinds are followed, the edge is held at may, and where the catch-all sent the URL is recorded.	The correction held for the application it was written against and failed for the two needing the other kind.
Refutation attempts.	Resolving a target and calling a link dead were conflated; bitwarden's authentication paths went through an unread enum, so the wildcard forwarded them all.	The enum and wrapper call are read; each application notes when its route set is a lower bound; each caveated candidate is put to the framework's route configuration.	The wildcard had raised that application's recovered figure. Withdrawn: 24 (the framework serves a screen), 19 (the matching route places none), 0 (a redirect answers); 0 keep the caveat.

Found by	What was wrong	What changed	Effect on the figures
Refutation attempts.	Unreadable routes were taken as undeclared: a spread-built array, ghostfolio's `matcher`, alfresco's tsconfig `paths` alias.	Both tools record the holes they cannot read; no undeclared claim is counted inside one.	ghostfolio's `/admin/users` had been reported as a confirmed dead link, and two live alfresco links as dead; all three constructs are read now.
Refutation attempts.	The route-set test read only the first quoted segment of a commands array, testing a prefix as the target.	The whole array is read as a pattern, computed segments as holes, matched pattern to pattern.	It cleared alfresco's two candidates and found further sites in mifos; the census holds 27 candidate sites.
Mutation sweep and end-to-end check			
Mutation sweep of the demotion rules, at commit c8eec9b.	On its first run in the final revision the sweep found rules the router fixes had added without a test.	Tests were written for most survivors; the rest were retired as equivalent, since no input tells them from their absence.	20 of 164 mutants survived; 18 now have tests, 2 were retired. At f26a302 every one of 174 deletions fails some test. The tests exposed a relative link that climbs above the root.
Panel: a review panel checking the scorer against the preregistration.	The end-to-end scorer compared a redirected navigation on the landed address, against the preregistration.	The scorer applies the preregistered rule.	mifos then scored 12 of 22 pairs, 5 of them redirected; the landed-target figure had been 15 of 22.
End-to-end check: mifos's suite missed transitions to action screens.	A template literal inside a commands array (`navigate([`actions/${name}`])`) was read as a variable, so its target was recorded as unresolved.	Its static text is read, each segment holding an interpolation a parameter.	The client, group and savings views reach their action screens.
Figure: after that repair the re-measured mifos graph joined every pair of screens.	A target prefix was cut at its first parameter, so the notifications tray's prefix, which opens with two parameters, was empty and the shell site reached every route.	Only trailing parameters are dropped from a target prefix; a parameter inside it matches any one segment.	The tray reaches the account screens its targets name; the mifos graph holds 163138 of 367236 possible pairs, against 28185 before the repairs.
End-to-end check: the same.	`relativeTo` passed in an options variable was not evaluated, so the loan view's actions were resolved from the root.	A `const` initialised with an object literal is read as that literal.	The loan view reaches its action screens.

Found by	What was wrong	What changed	Effect on the figures
End-to-end check: mifos's suite missed tab transitions.	A relative target was resolved against the screen it fires on; a link in a parent's template, rendered with each child tab, named nothing mounted.	It is resolved against the route that renders the component holding it (Section 3.2.6); the witness check applies the same rule (harness 1.10.0).	With the two rows above, mifos holds 20 of 22 suite pairs, against 12 before; every figure that depends on the extractor was re-measured.

Note. Found by names the method as the main text does; *figure*, *panel*, *review*, *harness* and *end-to-end check* are explained in the text above.

Several corrections in the protocol and harness group were found by a figure that looked implausibly good or bad (marked *figure*); each proved a property of the measurement and not of the extractor.

4.3.3 Reproducibility

Every figure is generated by scripts at one commit of the extractor and written into the thesis as a macro. `REPRODUCE.md` in the repository lists the scripts and the commit each table comes from. The version evaluated throughout is commit `f26a302`, whose extractor last changed at `2aac284`, with harness 1.10.0 and mutation harness 1.9.1; the demotion-rule sweep and the may-reason census were recorded at the same commit. The end-to-end figure before the repairs of Table 4.5 comes from the graph of the previously evaluated commit, `c42f220`, kept beside the results. Two groups are historical and were not re-measured: the rendered-interface check (Table 5.11), and the runtime route oracle, run on 29 and 30 October against a graph whose extractor commit was not recorded and whose node total is its only link to the final graph.

4.4 Summary

The corpus is six applications at pinned commits, selected by explicit criteria that were not applied uniformly, and uigraph was developed on it. Every ratio has one name and one definition in Table 4.2. The five methods are each blind where the others see; the reproducibility arrangements cover the first four fully and the fifth, the runtime route oracle, only in part. The corrections that led to the final protocol are listed in Table 4.5.

Table 4.2: The ratios and measures of the thesis, with numerator, denominator and research question.

Name	Numerator	Denominator	RQ	Reported in
Node coverage	declared screens that have a node	screens the independent route parser declares	RQ1	5.1
Must conversion	in-scope constructs witnessed by a must edge	in-scope constructs	RQ1	5.1
Recovered share	in-scope constructs witnessed by an edge of any tier	in-scope constructs	RQ1, RQ3	5.3, 5.5
Accounted share	recovered constructs plus in-scope constructs recorded as dynamic or unplaced	in-scope constructs	RQ2	5.2
Must-tier precision	must edges whose witness passes (lower bound) or does not fail (upper bound)	all must edges	RQ3	5.5.3
Node-level recall	distinct URLs named in an application's end-to-end test sources that have a node	distinct URLs so named	RQ1	5.1
Structural-reference recall and precision	(site, screen) pairs the reference and a level share	pairs in the reference (recall) or pairs the level places (precision)	RQ3	5.3.4
End-to-end containment	observed screen pairs joined by an edge of the graph	observed screen pairs	RQ3	5.3.5
Scope width	mean screens a site is attributed to	the same when every site is assigned to every screen	RQ3	5.3.2
Detection rate	injected changes that produce the predicted change in the graph	injected changes the graph can score (excluded mutants reported)	RQ4	5.4
Cause trace	defect classes traced to a cause in the source, by kind (read by the final version, still not read, beyond a static reader); no ratio	defect classes	RQ5	5.5

Note. Each measure has one name, used throughout. A construct is *in scope* when it is neither a query-only navigation nor excluded by the first rows of Table 4.3. Node coverage is the only measure called coverage in this thesis.

Table 4.3: Rules that classify a construct, in the order the harness tests them.

Category	The construct ...
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
No commands	names no destination at all
Converted, must or may	is cited as witness by an edge of that tier
Accounted, dynamic	is cited by an edge into a sink: the extractor records its destination as unresolved
Accounted, unplaced	is recorded by the extractor as a site it read but could not place
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

Note. The first three rows take a construct out of the denominator, the next counts it as recovered in the tier that cites it, and the two after that as accounted for. Only a construct that reaches the rows below is lost. The indented rules are the nested decision taken when the construct's owning route component has no node.

Chapter 5

Results

This chapter reports what the harness of Section 4.3 measured on the corpus of Section 4.1, one research question at a time. Section 5.1 answers RQ1, Section 5.2 RQ2, Section 5.3 RQ3 and Section 5.4 RQ4. Section 5.5 asks what the losses were and what the cause of each says about the claim (RQ5). Section 5.6 reports the dead links the method found as a by-product.

The numerical tables are generated from the harness result files; summaries compiled by hand are marked as such. The main tables describe the final extractor snapshot, with scope attribution off (the baseline) and on. The development versions and the corrections between them are listed in Section 4.3.2.

Two conditions hold for every figure below. The source trees are extracted without their dependencies installed, and the corpus was selected with the tool in view. Section 6.2 discusses both. The second makes the figures likely to be optimistic for Angular applications in general.

5.1 The two levels of loss (RQ1)

RQ1 is answered at two levels. The first is whether the graph holds the screens the application declares. The second is how much of the application's navigation becomes a deterministic transition between them (the ratios are defined in Section 4.2). The first is a precondition of the second, because a transition cannot start or end at a screen the graph does not have. It is not a numerical bound on the second: one ratio counts screens and the other counts constructs, so a missing screen costs only the constructs that name it.

Table 5.2 gives the second measure on the final snapshot. The constructs that earn must number 21 in the whole corpus without attribution and 25 with it. They lie in 4 applications with attribution, and the other 2 have none. Edge counts must be read per tier and not summed as distinct constructs, because one unresolved site can produce many over-approximated may edges.

Why the tier is small is measured directly. A measurement switch of the final version records, at every may witness, each condition that held it at may. It records them at the extractor's own decision points. With the switch off the output is unchanged, which was checked on all six applications (Section 4.3.3). Table 5.1

Table 5.1: Why the constructs that yield only may edges are may, by family of reason.

			Any, by application					
Family	Any	Sole	realworld	alfresco	ghostfolio	kdash	bitwarden	mifos
async	332	10	4	0	15	5	61	247
branch	230	9	4	4	17	1	85	119
template	740	31	30	3	34	7	29	637
guard	1475	174	16	9	98	11	202	1139
reader	127	1	1	1	0	1	3	121
attribution	194	0	1	4	9	3	95	82
target	640	4	13	3	28	5	97	494
same screen	184	0	4	3	15	1	6	155

Note. Any counts the constructs with at least one reason of the family, *Sole* those whose every may witness has that one reason only. The application's own decisions are the async, branch, template and guard families; reader, attribution and target are the extractor's uncertainty; same screen is neither.

Table 5.2: Must conversion per application, baseline.

Application	Constructs	In scope	Nodes	Must	May	Must conversion
realworld	40	39	10	1	24	2.6%
alfresco	34	27	37	0	7	0.0%
ghostfolio	198	186	112	0	60	0.0%
k8s dashboard	72	68	72	0	7	0.0%
bitwarden	131	112	113	1	89	0.9%
mifos	1343	1343	681	15	5923	1.4%

Note. Must conversion is the fraction of in-scope constructs whose own witness earns must; the edge columns count edges. *Nodes* counts every node of the baseline graph: the screens of Table 5.3 together with its unplaced origins and dynamic targets (Table 3.1 counts the attributed graphs, which add anchors and more of the latter). The baseline and attributed graphs are extracted from the same final snapshot on identical corpus inputs.

groups the reasons of the 1607 constructs that yield may edges and no must edge. 1582 of them sit behind at least one decision of the application itself that the extractor reads: a guard or resolver, a branch, a callback or a template condition. Only 25 are held at may by the extractor's own uncertainty alone, so a better reader could lift at most those to must. The small must tier is therefore mostly a property of how these applications navigate, not of the extractor. One limit applies: a guard that always lets the navigation through still counts as a guard, so the census says the application decides, not that the navigation is conditional at run time. The shares are also mostly mifos's: it holds 1240 of the 1607 constructs.

Table 5.3 gives the first measure. It compares the nodes the extractor mints with the screens the independent route parser finds declared. Against that denominator node extraction is complete: on all six applications every declared screen has a node.

The two readers still disagree about a handful of addresses. Two qualifications

Table 5.3: Node extraction against the independent route parser, in both directions.

Application	Declared	Nodes	Matched	Coverage	Unmatched	Blind spots
realworld	10	10	10	100.0%	0	0
alfresco	26	27	26	100.0%	1	10
ghostfolio	84	84	84	100.0%	0	2
k8s dashboard	67	67	67	100.0%	0	0
bitwarden	98	98	98	100.0%	0	1
mifos	605	606	605	100.0%	1	1
All		892			2	

Note. Declared screens that have a matching node, and nodes for which the independent route parser holds no declared screen. The blind-spot column counts route entries the parser could not follow (an unfound lazy route array, an unresolvable path expression, a custom matcher, or a route source spread from a call it does not follow), which marks where either direction is incomplete. The node set is the same with scope attribution off and on, since attribution mints no nodes.

apply.

- The denominator is the screens a *static reader* can enumerate. Against the addresses the running framework mounts, the same node set is 892 of 1146 (Section 5.5.3). Almost all of the difference is one construct in one application that the extractor declares unread.
- The two static readers reached their agreement partly by being corrected against each other, so it is weaker evidence than if they had stayed independent. The runtime method is the external check, and it finds no node at an address where the framework mounts nothing (Section 5.5.3 gives its provenance).

Completeness is a claim about the screens the readers can *see*. The blind-spot column marks where it is incomplete but does not bound it, because one route entry neither reader can resolve can hide many screens (Section 5.5.3).

5.2 Which failure is worth an analysis (RQ2)

Table 5.4 lists the defect classes and the evidence for each, separating the named evaluation methods from the other observations. The attribution class leans most on observations, and it is the class the analysis of Chapter 3 addresses. Its observations are not weak in themselves, but they are not independent instruments.

The five classes were found during development, and each has at least two confirmations. The quantitative breakdown revealed a sixth candidate: a node that exists but is not linked to the component that renders it. Only the comparison between the independent route parser and the graph confirms it, so it is reported as a candidate. It is empty on the final version.

Table 5.5 and Figure 5.1 assign every navigation construct to one outcome by the rules of Table 4.3. Of the 1818 constructs in the corpus, they show how many

Table 5.4: Defect classes and the evidence for each.

Class	Evidence	Methods	All
Attribution: navigation outside route components is invisible	mutants excluded on the baseline graph; multi-mount edge absent; manual audit; an application with nodes and no edges on a development version (k8s dashboard; Section 4.3.2)	1	4
Lazy module boundary not followed	independent route parser; test-source cross-check; mutation	3	3
Guards unreliable in both directions	red-team panels and a source survey of the guard factory and the call-form `canActivate` (Section 5.5.1)	1	2
Named router outlets not implemented	test-source cross-check; source survey	1	2
Constant-valued route paths not resolved	independent route parser; node-count collapse	1	2
Candidate: node not linked to its component	independent route parser against graph	1	1

Note. A class is admitted on two lines of evidence, at least one from a named evaluation method; the methods column counts only those. This summary is compiled by hand; it is not harness output.

convert onto each tier and why the rest do not. A construct counts on the must tier only when its own witness earns that modality. The rows are loss categories, not the defect classes of Table 5.4, and guard defects are measured separately (Section 4.2).

Under the order in which the loss rules test a construct, attribution is the largest loss category in every application. Of the 1818 constructs, the two attribution categories hold 454 between them, and no other loss category holds a tenth as many. The constructs whose destination the extractor records as unresolved are counted apart and come to about a third as many.

This ranking depends on the order in which the loss rules test a construct. Ownership is tested before the target, so a construct outside a route component is filed as attribution whatever its target. Of the attribution constructs, 101 have a target that the loss rules' literal test does not accept. If the target were tested first, the target category would be larger than attribution on alfresco and k8s dashboard.

Constant route paths, lazy module boundaries and nodes with no link to the component that renders them are empty in Table 5.5, because the final extractor reads their causes (Section 5.5.1). The ranking justifies spending an analysis on attribution: once the extractor reads what the applications declare, the navigation it still cannot place is mostly navigation with no single owning screen, under the order caveat above.

The categories also interact, in both directions. A construct in a shared component used only inside screens with no node is counted as attribution, although attribution alone cannot recover it. A construct whose own route component has no node is counted as a node defect, although attribution could reach it through a shell or subtree scope. The 454 attribution constructs are therefore neither a floor nor a ceiling on what scope attribution recovers (Section 5.3).

Table 5.5: Outcome of every navigation construct, baseline.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	1	0	0	0	1	19	21
Converted, may	16	3	55	4	53	979	1110
Attribution, shared component	21	10	63	51	37	224	406
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	0	0	0
Constant path	0	0	0	0	0	0	0
Component unlinked	0	0	0	0	0	0	0
Constant target	0	0	0	0	0	1	1
Node missing, other	0	2	3	1	0	34	40
Other	0	0	6	0	2	3	11
Dynamic destination	0	4	50	7	4	73	138
Source the analysis cannot place	0	0	0	0	0	0	0
Query only (not a transition)	1	0	11	4	19	0	35
Excluded: names no destination	0	0	1	0	0	0	1
Excluded: navigates a named outlet	0	7	0	0	0	0	7
All constructs	40	34	198	72	131	1343	1818

Note. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos. All rows are measured on the final snapshot with attribution disabled.

Table 5.6 reports the witness check on the final attributed tier and the test-source cross-check. Section 5.5 treats the must tier separately. 636 edges in 3 applications have their only witness outside the application directory; both tools read those files, so their agreement there is not independent evidence.

5.3 Scope attribution (RQ3)

The analysis under evaluation is the one Chapter 3 states: scopes in Section 3.2.2, the rules that compute them in Section 3.2 and the anchored representation in Section 3.2.6.

Three quantities decide the answer: how much navigation the pass recovers, whether it costs the deterministic tier anything, and what it still cannot place. The third shows where the analysis as implemented stops.

Table 5.7 and Figure 5.2 compare the extractor with and without scope attribution on identical inputs. The comparison is made construct by construct. A construct is *recovered* if it was lost to attribution in the baseline and is witnessed by an edge after attribution.

Attribution recovers 300 of the 454 constructs the loss rules assign to it. This is 66.1 % of them, which this thesis calls the attribution recovery. Most of them are on

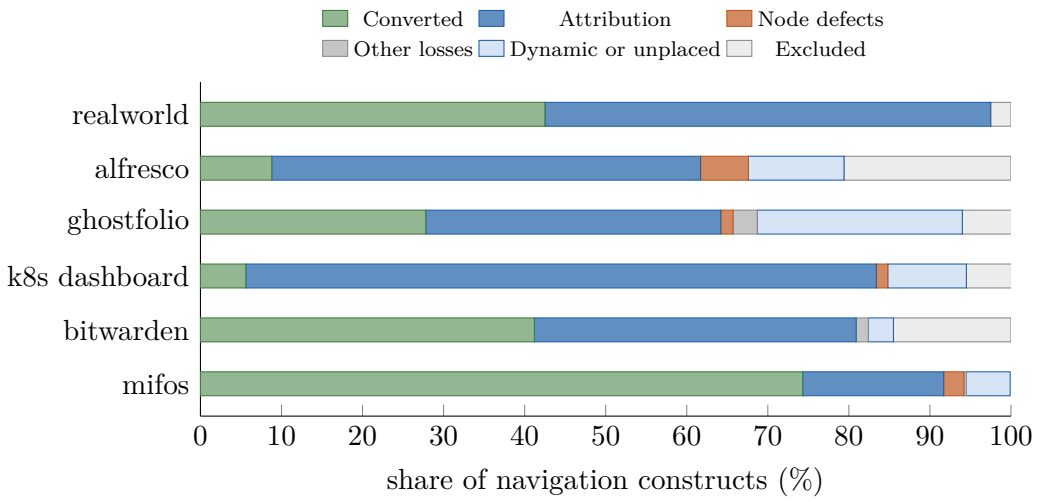


Figure 5.1: What happens to the navigation constructs of each application without the attribution pass.

Note. Attribution losses are the largest category in every application; the lazy-boundary, constant-path and component-unlinked node classes are empty on this instrument, and the remaining one holds few constructs, most of them in mifos. Exact counts per class are in Table 5.5. *Converted* counts constructs witnessed at must or may, as that table does; the must conversion of Figure 5.2 counts must only.

mifos. Without mifos the share is 37.7 %, and 4 of the six applications recover under half. It takes the graphs from 6266 edges to 15653.

Measured by RQ1's must conversion, the gain is far smaller. Of the recovered constructs, 2 earn must and 298 earn may, and 2 of the six applications gain any must conversion at all. The must constructs rise from 21 to 25. Two of the rise are those recoveries, both on realworld. The other two are k8s dashboard constructs that the baseline already converted at may and attribution raises to must. The recovery count does not include them.

The two are different measures of recovery, and both are reported. A gain in represented navigation need not be a gain in deterministic transitions when targets or execution conditions remain uncertain. Counted in edges, the must tier grows from 17 to 159. The edges added are the links of two application shells, drawn once for each screen the shell renders (Section 5.5).

Following each construct by its identifier across the two runs finds 0 that leave the converted categories, so no must conversion of the baseline was lost. This measures recovery of represented constructs and says little about must-tier precision. The evidence for that tier, and the edges outside it, are treated in Section 5.5.3.

The anchored representation was checked on the same graphs. Claim 3.2 requires the expansion of the anchored edge set to equal the materialised edge set, so the anchored edges are materialised independently: each scope-anchored edge is fanned over the screens its anchor stands for, with its own deduplication. The two agree on every application, and Table 5.8 is not produced otherwise. The table is also the only measurement of what the representation saves. On mifos the materialised edge

Table 5.6: Results of the witness check on the final attributed tier, and of the test-source cross-check.

Method	Result	n
Witness check, final attributed must tier	0 detected failures; 62 edges have a destination the checker cannot read. Unread edges are not passes, and target agreement does not establish Claim 3.1.	159
Test-source cross-check, alfresco	10 of 14 URLs the specifications name have a node; one negative test excluded.	14

Note. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

Table 5.7: Before and after scope attribution on the final evaluated snapshot.

	Attribution losses, recovered			Must conversion (RQ1)			Must, read and passed	
Application	Lost	must	may	Before	After	pp	Before	After
realworld	22	2	20	2.6%	7.7%	+5.1	1/1	9/9
alfresco	18	0	4	0.0%	0.0%	+0.0	–	–
ghostfolio	72	0	32	0.0%	0.0%	+0.0	–	–
k8s dashboard	56	0	8	0.0%	2.9%	+2.9	–	63/125
bitwarden	52	0	17	0.9%	0.9%	+0.0	1/1	1/1
mifos	234	0	217	1.4%	1.4%	+0.0	15/15	24/24
All	454	2	298					

Note. First, the constructs assigned to attribution in the baseline and how many recover onto must or may, the measure of RQ3. Then must conversion (RQ1) in per cent and its change in percentage points, and must-tier edges whose destination the witness check read and passed; a construct's tier is the modality its own witness earns. The remaining must edges are unread destinations, all 62 of them on k8sdash, which are neither passes nor failures; no edge failed. Constructs converted from other baseline categories are in Table 5.9.

set is 12.0 times the size of what is stored.

The check exercises the member walk and the deduplication key. The definition of a scope's membership is shared by both paths and is not tested by it. The claim is therefore supported on 6 applications and on fixtures, not proved. The library's own materialisation function is not used for this check: it is a loop over the query and could only disagree about which nodes it iterates. On 5 of the applications it returns a larger set than either, which is that iteration difference and not a disagreement about the edge set.

Table 5.9 gives the recovery by the loss category each construct held in the final baseline and by the modality its own witness earns. The loss categories are not independent causes: a shared component can be assigned to attribution even when a missing target node prevents its recovery.

Table 5.10 classifies each construct that stays lost.

The two tables close the attribution categories between them. 300 recovered plus 154 remaining is the 454 the rules assigned. The script that produces the tables

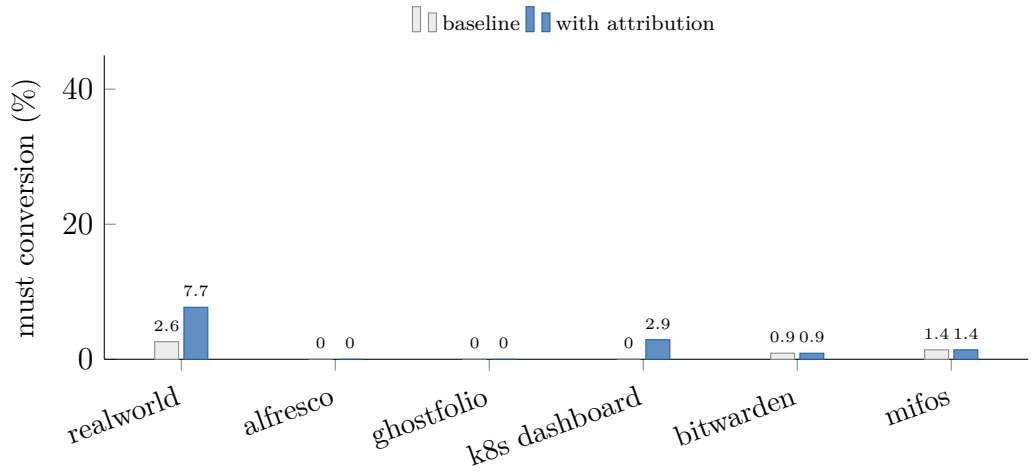


Figure 5.2: Must conversion before and after scope attribution, on identical inputs.

Table 5.8: The anchored edge set against its expansion.

Application	Screens	Anchored	Expanded	Independent
realworld	10	65	97	97
alfresco	27	55	100	100
ghostfolio	84	403	1166	1166
k8sdash	67	313	698	698
bitwarden	98	578	763	763
mifos	606	14239	170772	170772

Note. *Anchored* is what the graph stores, *expanded* what the per-screen query yields, and *independent* a second materialisation written for this check. Claim 3.2 requires the last two to be equal; the check refuses to emit this table if they differ on any application.

asserts this before it writes either table. All but 2 of the remaining constructs carry a reason. Those 2 have a scope and a destination the graph holds, and the diagnosis names no reason they stay lost. The reasons are limitations of the implemented analysis and limits of the source population.

- Some components have no recovered route or template path to a screen. This is an empty computed scope, not proof that the application never renders them.
- Some sites lie beyond the member-level call graph's supported receiver and provider forms.
- Other sites have a scope but a dynamic destination, or a target for which the graph has no node.

5.3.1 Checking the scopes against the running interface

Whether the *set of screens* attribution computes for a site is the set that site fires from is a different question from how much it recovers. The rendered-interface check

Table 5.9: Where recovered constructs came from on the final snapshot.

Baseline class of the construct	must	may	total
Attribution, shared component	2	278	280
Attribution, service	0	20	20
Other, no class rule applied	0	2	2
Out of the attribution classes	2	298	300
Out of the node classes	0	0	0
Out of the remaining classes	0	2	2
All conversions gained	2	300	302

Note. Rows are their baseline loss categories; columns give the modality each construct's own witness earns after attribution. Constructs are followed by identity, not matched by source location.

Table 5.10: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	8	11	5	18	1	43
In a service or guard, not placed	0	6	4	4	7	7	28
Scope found, target screen has no node	0	0	23	2	6	3	34
Scope found, target fully dynamic	0	0	2	37	2	6	47
Scope found, not explained	0	0	0	0	2	0	2

of Section 4.2 answers one direction. Table 5.11 retains the recorded observations of an earlier graph. The browser experiment was not repeated and its predictions were not recomputed for the final call graph, so this section documents what that earlier graph missed.

Scopes too small. Of the 33 (screen, target pattern) pairs that the running applications render on the judged screens, 29 are predicted by the computed scope (87.9 %). Behind these predictions stand 67 raw anchors. On realworld, which renders most of its screens without a backend, every rendered pair on every judged screen is predicted. The pairs that are not predicted are all bitwarden's.

The unit of the check is the screen and the destination, not the site, so a confirmed pair shows that the screen's outgoing destinations cover what it renders, not that each site behind them was attributed to that screen. An unpredicted pair is not affected by this pooling, so the 4 are direct evidence of scopes that are too small.

The evidence is limited in size. It covers 30 screens of 806, because 776 redirect to a sign-in page when there is no backend behind them, and only 5 of the six applications are measured (ghostfolio's per-locale bundles do not fit the harness's single served directory). The check settles one direction of a question, on part of the corpus.

Table 5.11: Recorded scope predictions of an earlier graph against the rendered interface, on the 5 applications the harness could serve.

Application	Screens	Judged	Pairs rendered	Predicted by the scope	Not predicted	Raw anchors
realworld	10	7	25	25	0	59
alfresco	26	3	0	0	0	0
k8s dashboard	67	1	0	0	0	0
bitwarden	98	18	8	4	4	8
mifos	605	1	0	0	0	0
All	806	30	33	29	4	67

Note. These predictions were not recomputed for the final call graph. *Judged* counts screens that rendered themselves rather than redirecting to a sign-in page; the rest cannot be judged from what rendered, because a redirected screen shows another screen's links. *Not predicted* is the only column that counts against the analysis; the other direction is reported in the text, because it can be judged only where the graph names a destination exactly.

The 4 unpredicted links share one cause, and all four are in bitwarden: its `/login` screen renders links to `/hint` and `/signup`, and two further screens render a link to `/signup`. The graph holds those edges, with the right witness, but anchors them at an origin node rather than at a screen, because the component that renders them lives in a workspace library outside the application directory and attribution computed the empty scope for it. The failure is a scope the analysis declined to compute, which the running application shows to be non-empty. These sites are the *unplaced* class of Table 5.20.

Scopes too large. This direction does not support a single figure. 80 predictions run this way: the graph holds the edge and the page showed no link. Two of the three causes mixed in that count are not defects. The harness signs in to nothing, so realworld's header hides links behind an authentication condition (interface state), and a tag page navigated to with a placeholder segment lists no articles (data). The third cause is target over-approximation, a property of the destination and not of the scope: a profile link resolves to both `/profile/:username` and `/profile/:username/favorites`, the deeper route recorded as a may edge at half confidence.

Only predictions that name a destination exactly can be refuted by a page not linking there. There are 7 of them across the judged screens, against 102 over-approximated. 0 of the 7 were not rendered. A denominator of 7 supports no rate. The over-attribution direction was therefore not refuted on this corpus, because the graph almost never claims an exact destination at a screen this check can observe.

Table 5.12: The computed attribution against assigning every site to every screen.

Application	Screens	Sites	Attributed pairs	Mean scope	All-screens pairs
realworld	10	39	188	4.82	390
alfresco	26	19	117	6.16	494
ghostfolio	84	189	4104	21.71	15876
k8sdash	67	62	2450	39.52	4154
bitwarden	98	167	1785	10.69	16366
mifos	605	1316	43347	32.94	796180

Note. A site is a source construct a witness cites; an edge anchored at a scope is counted over the screens that scope stands for, since counting it at its anchor would understate the analysis's breadth. *Mean scope* is the attributed pairs per site.

5.3.2 What the attribution discriminates

The rendered-interface check cannot say whether the scopes are too large, so this subsection asks how much narrower the computed attribution is than the widest one there is. Assigning every navigation site to every screen recovers every construct and needs no analysis. It is useless, because the per-screen query would return the whole site list on every screen. The ratio between the two measures what the analysis contributes (Table 5.12).

The analysis is between 1.7 and 18.4 times narrower than that baseline, and the low end of the range is the figure that qualifies RQ3. On k8sdash the average site is attributed to 39.5 of 67 screens, which is 59 % of the application. The per-screen query there excludes about two screens in five and admits the rest.

Two things follow, and the second is the less favourable to the analysis.

1. The trivial baseline is not a competitor for the must tier. An unguarded site in a shared component attributed to every screen would yield a must edge from screens that never render it, so Claim 3.1 would fail by construction. That none of the readable must edges attribution added fails the witness check is meaningful only where the attribution is narrow. Of the 142 must edges it added, 62 are unread, all on k8sdash, where the scopes are widest.

2. On part of this corpus the attribution is not very narrow. A scope that names three fifths of an application is a weak statement about where a site fires, and the attribution recovery RQ3 reports is bought with scopes of that width. Nothing in the evaluation shows those wide scopes to be wrong, because the rendered-interface check could not test the direction in which they would be wrong. Nothing shows them to be right either. The recovered share of Table 5.20 should be read together with the widths in this table.

Table 5.13: The corpus extracted at each attribution level.

		sites / (site, screen) pairs				
Application	Screens	owned	nesting	rooted	outlet	full
realworld	10	17 / 26	25 / 48	38 / 178	38 / 187	39 / 188
alfresco	26	11 / 33	14 / 37	14 / 37	14 / 37	19 / 117
ghostfolio	84	111 / 121	129 / 155	170 / 3599	182 / 3712	189 / 4104
k8sdash	67	11 / 12	59 / 275	59 / 275	59 / 2250	62 / 2450
bitwarden	98	124 / 134	149 / 389	154 / 879	156 / 1448	167 / 1785
mifos	605	1084 / 1437	1247 / 1765	1307 / 38065	1307 / 41820	1316 / 43347
All	890	1358 / 1763	1623 / 2669	1742 / 43033	1756 / 49454	1792 / 51991

Note. *Sites* counts the source locations a witness cites that the level places on at least one screen; one construct can have more than one, and a location can lie outside the application directory, so site counts are not comparable with the construct counts of Table 5.5. *Pairs* are the (site, screen) attributions it asserts. The levels are nested, and the table is produced only if each one's pairs are a subset of the next one's. *Screens* counts the screens at an address a link can name, so the catch-all screens of alfresco and mifos are left out of it and of the pairs, and the column falls short of the census's total by those screens.

5.3.3 What each mechanism contributes

Table 5.13 extracts the corpus at 5 nested levels, each withholding every mechanism above it, including the stand-ins of Section 3.3; otherwise the lowest level would credit direct ownership with sites that only the stand-ins place.

The levels are not evenly spaced. Containment adds the most sites of any step, the shell rule, which this thesis does not add, is the largest step in (site, screen) pairs and in reference recall, and outlet descent adds mostly width, which a table of counts cannot show to be correct (the next subsection addresses it). The full level adds the member-level call graph that places a service's navigation, its fallback join and the stand-ins for guards and effects, all of which only that level and its per-class variant carry. The call graph's own share is smaller. Against the coarser class-level rule it replaced, it places 7 more sites, 0.4 % of the corpus total, and 331 more pairs. That rule remains as the fallback for a member no caller reaches.

The shell rule is not a mechanism this thesis introduced, so the comparison with earlier work uses two of the levels. Rima et al.'s specification also closes a route's components over those of its ancestor routes, which is what *outlet* adds. Both levels reimplement those mechanisms inside this extractor and are not runs of either tool. The comparison measures what the added mechanisms place on the same inputs, and does not show that the tools themselves would place fewer. Against the nearest of them, *outlet*, this thesis adds the *full* step alone: 36 sites, 2.0 % of the corpus total, every one outside a component, in a service, guard or effect.

Table 5.14: Each attribution level against the hand-read structural reference of each application.

	realworld			alfresco		
Level	Precision	Recall	Off shell	Precision	Recall	Off shell
owned	100.0 %	13.8 %	44.8 %	100.0 %	9.2 %	36.4 %
nesting	100.0 %	25.5 %	82.8 %	100.0 %	11.5 %	43.2 %
rooted	100.0 %	94.7 %	82.8 %	100.0 %	11.5 %	43.2 %
outlet	100.0 %	99.5 %	98.3 %	100.0 %	11.5 %	43.2 %
full	100.0 %	100.0 %	100.0 %	82.0 %	23.6 %	90.9 %
class	95.4 %	100.0 %	100.0 %	92.3 %	13.8 %	52.3 %

Note. Precision and *recall* are agreement with that reference, not with what the running application renders. *Off shell* is recall over the pairs outside the shell, where attribution decides. *class* is the class-level join that the call graph of *full* refines.

5.3.4 Agreement with a structural reference

For realworld the screens each navigation site fires from were read off the sources by hand (Section 4.2): 39 sites and 188 (site, screen) pairs. A second reference was built for alfresco, where navigation inside services occurs; realworld has one such navigation and barely exercises the step only this thesis adds: 16 sites and 174 pairs. Scoring every level against such a reading gives a comparison no other method here can make, because it has both directions rather than one. Table 5.14 reports both.

On realworld the table is produced only if each level's pairs are a subset of the next, and the full level asserts no pair outside the reference, so the levels differ only in recall. Most of the recovery is produced by mechanisms earlier work already had: the shell rule (*rooted*) and the enclosing routes (*outlet*), and the *full* step adds the remaining 1 pair. On alfresco the gain over the levels of earlier work comes from the service join, and the precision order reverses: the full analysis is less precise than both the lower levels and the class join.

- *The reference is not independent of the graph.* The realworld reference was completed in its first version with nine header sites the scorer reported from the graph. Its precision is therefore weakest as evidence where the shell dominates.
- *The shell dominates.* On realworld, 13 of the 39 sites are placed on every screen and supply 130 of the 188 pairs, and every level from *rooted* upward places them trivially. On alfresco, 5 of the sites, 130 of the 174 pairs, sit in a shell that the application takes from a library outside its sources. No level reads it, so every level misses all but at most one of those pairs. The off-shell column is the one where attribution decides.
- *Two repairs were guided by the reference.* One is the call graph, written after the realworld reference exposed the class-level defect. The other is the rule that places an empty-path child on the screen above it (Section 6.3). The 100.0 % precision and the recall of *full* are scored on the same reference, so they are consistent with the two repairs and test neither independently.

- *The class join's excess on realworld* is 9 pairs, all at one site, the `navigate` inside `UserService.logout`. The class join attributes a navigating service to every screen whose components inject it, while its only caller is a button on the settings screen, where the call graph places it alone.

- *The alfresco reference was built without the graph, by an AI assistant.* Its protocol, the same rules as realworld's, was committed before the reference was built or any level was scored (`ground-truth/ALFRESCO-PROTOCOL.txt`). The assistant built it from the sources alone, without the graph, and left 17 sites undetermined, which are not scored.

- *Four lines were excluded after scoring.* The scorer reported them as placed by the graph and absent from the reference; they were read afterwards under the same rules, given only the line numbers and not the graph's placements. None is a navigation site, and they were added as excluded, where the protocol provided only for adding a missed site. No figure depends on it, since unlisted lines are not scored.

- *All of the full level's excess on alfresco is one config-driven site.* The 9 excess pairs are at 1 site, `navigateToSearch`, whose only caller is an effect triggered by a toolbar button declared in the extension configuration, which the analysis does not read, so it places the site on every screen at the may tier. Most of the pairs the full analysis adds over *outlet* come from this placement. The class join's 2 excess pairs are at the same site. The two sites of `navigateBack` are placed exactly on the two search screens their only caller renders.

- *One undetermined site is placed on every screen* by the full and class levels, because no source dispatches its action. It is not scored; scored as firing from nowhere, it would add to the excess of both.

- *The protocol's predictions.* The first (A1), that the full level recalls more than *outlet*, held. The second (A2), that its precision is at least the class join's, did not.

What the two references establish is narrower than soundness. Both apply the analysis's own relations, and neither licenses the corpus-wide widths of Table 5.12 on an application of six hundred screens.

5.3.5 Agreement with the applications' own suites

The end-to-end check of Section 4.2 puts part of the question to the running applications, with a reference that does not pass through the analysis's relations. The second and third validity checks hold on both applications, and the first on mifos, whose uninstrumented run fails the same tests. For realworld only instrumented runs were recorded, none of them with a failure.

Table 5.15 gives the figures for both applications.

Table 5.15: The graph against the pairs of screens the applications' own end-to-end suites navigate between.

	realworld	mifos
Navigations completed	141	62
Distinct pairs	19	22
Pairs in the graph (hits)	19	20
through the source screen or a scope that names it	16	19
only through the shell scope	3	1
Hits without attribution	13	10
Hits at the rooted level	19	17
Pairs redirected (redirect bucket)	0	5
of which hits	0	3
Pairs the expanded graph contains	79	163138
Possible ordered pairs of screens, self-pairs included	100	367236
Misses	0	2
redirect	0	2
base	0	0
dynamic	0	0
other	0	0
Misses that meet the dynamic test in any order	0	2

Note. A redirected navigation is compared on the address it asked for (the preregistered rule). *Rooted* is the level that stands for earlier work's mechanism. The miss classes after *redirect* were defined after the misses had been read, and are tested in the order redirect, base, dynamic, first match winning. All cells are generated figures.

- *realworld.* The agreement is with the transitions the suite takes and not with the graph. The result does not bear on what the levels above *rooted* add, because the *rooted* level already contains every pair.

- *mifos, hits.* The preregistered rule compares a redirected navigation on the address it asked for. The scorer's departure from this rule during development is listed in Table 4.5. None of the matched pairs is a must edge, as predicted. One of the 3 hits of the redirect bucket rests only on the notifications tray, a shell site whose target, `/${entityType}/${entityId}/savings-accounts/${id}` and its siblings, keeps two leading parameters and so reaches every account screen from every screen. Such a hit shows that the target is reachable in the graph and not that the site was resolved. The same site makes the graph wide: it holds 163138 of the 367236 possible pairs, against 28185 before the repairs below, so a hit on mifos discriminates less than it did. No other preregistered prediction is reported here.

- *mifos, misses.* Only the redirect class was preregistered, and its misses are not classified further. The other classes were defined after the misses had been read, before the repairs below. Every remaining miss is in the redirect class (2 of 2): the graph holds the pair on the screen the redirect lands on. For 1 of them a better reader would also hold the address asked for (Section 6.3.1). They also meet the dynamic test, which fires on any unresolved site in the source

component or above it, so that test does not show that a missed navigation is that site. The preregistered five-class scheme and its prediction that misses would be mostly a scope too narrow or a site not read are not scored.

Before the repairs the graph held 12 of the 22 pairs, and the post-hoc classes pointed at places the extractor did not read, not at destinations only the run time knows. Sites in the client, group and savings views open an action screen with a template literal inside a commands array, which the reader treated as a variable although the text is static up to the action name. In the loan view, `relativeTo` was passed through an options variable the reader did not evaluate. The client view's template is rendered with each child tab, and its relative links were resolved against the child's address, where they name nothing mounted; Angular resolves such a link against the route of the component that holds it (Section 3.2.6). These gaps were repaired after the check, each with a test, and every figure that depends on the extractor was re-measured with the repaired one, except the historical groups of Section 4.3.3 (Table 4.5). The repairs follow rules of Angular's router and name no site, but this check found them and scores them, so the mifos figure after them is in-sample like every other figure; only the earlier one, 12, comes from an extractor fixed before the logs were read. No other application with a runnable suite remains to test them.

5.3.6 What scope attribution adds

This subsection gathers the RQ3 evidence in one place. Table 5.16 sets three graphs side by side: without attribution, with the mechanism of earlier work, and with the full analysis. The earlier tools were not run. The *rooted* column reimplements their mechanism inside uigraph, on uigraph's own node extraction (Section 5.3.3). Every figure is in-sample.

Three things follow, with the figures in Table 5.16.

- *Attribution puts more of the suites' transitions into the graph, in a wider graph.* The extractor of the first measurement was fixed before the suites' logs were read; the repairs that followed the check are in-sample (Section 5.3.5).
- *The gain is reached by the mechanisms of earlier work.* On realworld the rooted level holds every suite pair. On mifos it holds 17; the other 3 hits have a child tab as source and a site in the parent view, which the *outlet* level, the enclosing routes of Rima et al.'s specification (Section 5.3.3), places on the child screens. That level holds 20, as many as the full analysis. These pairs were missed before the repairs, so the difference is in-sample (Section 5.3.5). Oshima and Kishi's stated restrictions admit none of these applications, which declare route parameters and nested route trees. Rima et al. state no such restriction, and their specification covers this level (Section 2.4). Neither tool was run, so how they read these applications is not shown. Inside uigraph the level runs on all six because uigraph's route extraction reads parameters and nested trees, with or without attribution.

Table 5.16: The graph without attribution, with the mechanism of earlier work (*rooted*), and with the full analysis.

	no attribution	rooted	full
Suite transitions in the graph, realworld	13 of 19	19 of 19	19 of 19
Suite transitions in the graph, mifos	10 of 22	17 of 22	20 of 22
Graph pairs, realworld	25	78	79
Graph pairs, mifos	5765	159445	163138
Sites placed on at least one screen, corpus	–	1742	1792
Structural-reference recall, realworld	–	94.7 %	100.0 %
Structural-reference recall, alfresco	–	11.5 %	23.6 %
Structural-reference recall off the shell, alfresco	–	43.2 %	90.9 %
Structural-reference precision, alfresco	–	100.0 %	82.0 %
Structural-reference precision, realworld	–	100.0 %	100.0 %
Navigation inside services, constructs lost at baseline and recovered	–	–	20 of 48
Sites placed in services, guards and effects	–	–	36

Note. *Suite transitions* are the distinct pairs of screens the application's own end-to-end suite navigates between (Section 5.3.5); *graph pairs* are those the graph holds. Only the full column of the suite rows was preregistered. The no-attribution graphs and realworld's rooted graph come from existing measurements; the mifos rooted graph was mapped for this comparison after the suites had been scored. A dash marks a figure not measured. All cells are generated figures.

- *The service join places navigation no earlier mechanism can place.* A navigation call inside an injected service belongs to no component, so the levels up to *outlet* cannot place it on any screen, on every application of the corpus (Table 5.5). The member-level call graph is more precise than a class-level join on realworld, but on alfresco the precision order reverses (Table 5.14). The levels above *rooted* find the reference pairs rooted misses, and on the two applications with suites the step above *outlet* adds no transition the suites take.

Of the 454 constructs the loss rules assign to attribution, it recovers 300 (66.1 %), 298 of them at the may tier. The may tier is mostly the applications' own decisions, not the extractor's uncertainty (Table 5.1). The attributed must tier has 0 witness-check failures among 159 edges, with 62 destinations unread, which are not passes (Section 5.5.3).

The scopes are wide where a shell renders a link on every screen: the computed attribution is 1.7 to 18.4 times narrower than assigning every site to every screen (Section 5.3.2). Wide is not the same as too large. Where the scopes were read against a reference, the full analysis asserts no pair outside it on realworld and 9 at one site on alfresco, where it also places an undetermined site on every screen. Both references apply the analysis's own relations, and only two of the six applications were read this way. Section 6.2 discusses both as a limitation of the work. A scope that is too large still yields edges whose witnesses agree, so no witness-based method can find it. The suites test only the missing direction, and only on the transitions they take. Attribution also widens what a source change can be scored against (Section 5.4).

Table 5.17: What the corpus shows can be done without this thesis and with it.

	Without this thesis	With uigraph
Static extraction of the corpus applications	Oshima and Kishi's restrictions admit 0 of 6	extracts all 6
Screens checked without signing in	a browser run renders 30 of 806; the rest redirect to sign-in	every screen the static readers find declared has a node; nothing is run
Dead links found from the source	–	27 candidate sites in bitwarden, mifos, checked against the runtime route oracle (Section 5.6)
Transitions on realworld	its own suite takes 19 pairs	the graph holds 79, an over-approximation
Response to a source change	–	45 of 45 mutants, at constructs the graph represents

Note. The dynamic row is this thesis's own rendered-interface check run without signing in, not a crawler with credentials; the earlier tools were not run; every figure is in-sample. All cells are generated figures.

Table 5.17 puts the comparison in terms of what can be done without this thesis and with it, on this corpus.

5.4 Change sensitivity (RQ4)

Mutation testing requires the application to be extracted again once per mutant, so it was run on three of the six applications: realworld and alfresco, which carry the earlier mutation figures, and bitwarden, whose routes carry many guards. Every figure in this section comes from the attributed graph with harness mutate 1.9.1, a fixed seed and a quota of eight mutants per operator. The corrections made to the mutation harness are listed in Table 4.5.

Table 5.18 gives the sample, the exclusions and the detections on the attributed graph and without attribution; Table 5.19 gives the detection rate per operator and application, and Figure 5.3 the outcomes summed over the applications.

The rename operator is set aside first, because its detection is close to guaranteed by construction: a rename mutant is excluded unless the unmutated graph already holds a node for the segment it renames, and a screen is identified by its full path (Section 3.1). It tests node extraction. The other three operators are constrained in the same way. A deleted `routerLink` is sampled only on a line an edge of the unmutated graph cites, so it is missed only if the extractor misreads its own input. A retarget is generated only for a `navigate` call whose first element is a plain absolute string at a line the graph already cites, and it writes one declared route as a literal. A guard removal is scored only where an edge already carries that guard and enters the guarded screen, so the guard rate is conditional on the graph entering the guarded screen. The exclusions are few on the attributed graph but not on the graph without attribution, the same recovery gap seen from the mutation side. The samples are

Table 5.18: Mutants sampled, excluded, scored and detected, with scope attribution on and off.

	attributed graph	without attribution
Candidate mutations found	272	–
Sampled under the quota	73	73
Excluded	6	22
guard: graph enters no screen the route yields	5	–
guard on a route with no component and no children	1	–
Scored	67	51
Detected	67	50
rename a route path	22 of 22	–
delete a `routerLink`	14 of 14	7 of 7
retarget a `navigate`	20 of 20	–
remove a guard	11 of 11	–
Paired mutants, detected	51 of 51	50 of 51

Note. An excluded mutant carries no information about sensitivity. All exclusions are guard removals: for the first kind no edge of the unmutated graph enters any screen the guarded route yields, and the second guards a route that declares neither a component nor children. *Paired* counts the mutants both graphs can score. A dash means the figure is not reported for that graph. Operator rows are detected over scored.

small, in single digits per operator and application, so the pattern says more than the rate.

Each guard removal is judged by whether some edge loses a guard the removed list names, and all 6 guard exclusions are the condition above failing. One limit of the harness's matching of a class by the file its routing module imports it from shows in the run without attribution. realworld's `editor` route has two children, at the empty path and at `:slug`, that load the same component file, so the harness cannot tell their screens apart. Without attribution no edge enters `/editor`, but one enters `/editor/:slug`, so the guard on the empty-path child is scored and missed rather than excluded. That is the one paired disagreement, and it is a limit of the harness, not of the graph.

The mutation run neither confirms nor contradicts the defect class *guards unreliable in both directions*; Chapter 6 gives the reading. In this sample the graph responded to every scored change in the constructs it represents.

5.5 What the losses were (RQ5)

The defect classes of Section 5.2 are a diagnosis. RQ5 asks what lies behind each class and what that says about the claim, and it is answered by cause analysis on the final version. The cause trace of Section 4.2 defines the three outcomes used below.

Table 5.19: Detection rate per mutation operator on the attributed graph.

Operator	realworld	alfresco	bitwarden
Rename a route path	6/6	8/8	8/8
Delete a `routerLink`	6/6	–	8/8
Retarget a `navigate`	8/8	4/4	8/8
Remove a guard	5/5	3/3	3/3
All operators	25/25	15/15	27/27
Detection rate	100.0%	100.0%	100.0%

Note. Detected over scored mutants. A dash means the operator found no candidate in that application.

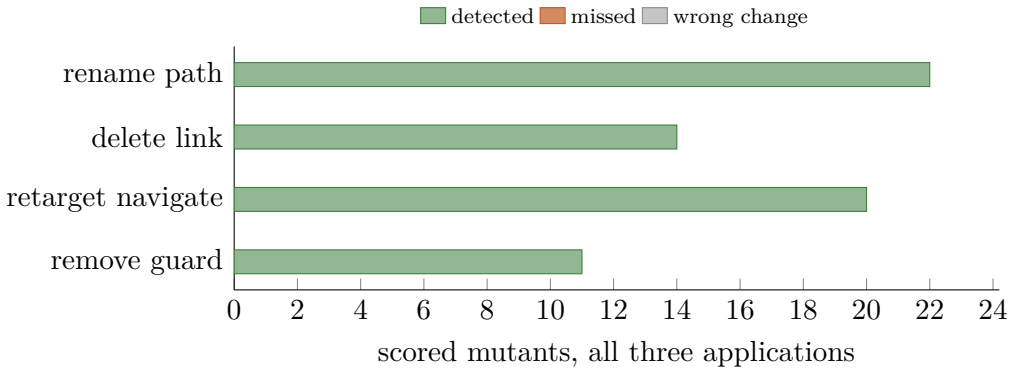


Figure 5.3: Mutation outcomes by operator, summed over realworld, alfresco and bitwarden, on the attributed graph.

Note. Excluded mutants are not shown; 6 of the 73 sampled were excluded here against 22 of 73 on the graph without attribution.

5.5.1 What the losses were

This section keeps two words apart: a *class* is a row of Table 5.4, and a *cause* is what the source review found behind a class.

Read by the final version. The information was in the source, so the loss was the tool's.

- *Lazy module boundary* and *constant-valued route paths*: causes in the extractor, read by the final version, and empty in Table 5.5.
- *Guards*: the two unread forms (a guard on a route a wrapper returns, a `canActivate` element written as a call) are read, and the guard rule is the router's own semantics (Section 3.1).
- The candidate class *node not linked to its component*: empty on the final version.

- Three forms that the end-to-end check exposed: a template literal inside a commands array, `relativeTo` passed through a variable, and the base of a relative link in a template shared with child screens (Section 5.3.5, Table 4.5).

Still not read. The final version does not read these causes.

- *Attribution*: 66.1 % of its constructs are recovered. The remainder is in Table 5.10, including sites that no relation the prototype implements reaches. It mixes gaps of the tool with components that no screen may render, and no measurement separates them.
- *Named router outlets*: a construct the model represents and the extractor does not implement. Its navigations are excluded from the denominator (7 sites) and its screens left out of the runtime route oracle's mounted set (64 route entries).
- ghostfolio's spread of computed routes is a construct the extractor could read and does not. 242 of the 251 mounted addresses without a node sit behind it.

Beyond a static reader of a route table. No reader of a route table can resolve these.

- ghostfolio's custom `UrlMatcher` computes a route's path in a function. No reader of a route table can resolve it, and both tools report it as a blind spot.
- mifos chooses between two route tables by a value injected at deployment, which the runtime method of Section 5.5.3 finds and the sources leave undecided.

The causes the final version reads fall into three groups, traced here in the source.

The route table was not reached. This is the lazy-boundary cause. An NgModule loaded by `loadChildren` keeps its routes in a separate `*-routing.module` that it imports, and a reader that stops at the module file finds nothing. A lazy route may also name a module through a barrel that re-exports a feature module. A reader that follows one hop finds no table and places the screens at the application root (`/breach-report`), where the running router mounts them deeper (`/reports/breach-report`). Both static readers shared this cause, so their agreement could not expose it and the runtime route oracle did. The final extractor follows the chain to the table, bounded at three hops.

The path was named, not written. This is the constant-path cause. In one application every route path is a reference into a constant table, which collapses its graph to a single node when it is not followed.

The destination was named, not written. A target may be a field, a dotted chain into the route table, an element of a commands array, the return of a builder, or a value in a lookup the component fills. A field seeded with a literal is mutable state, not a constant, and the reader does not follow it, because following it would turn a run-time value into a must edge.

Node extraction is therefore not the bottleneck on the final version: every application is complete against the readers' denominator. An analysis that decides where a navigation can fire needs a node for the screen it fires from, and a construct whose source screen has no node is a loss attribution cannot repair.

5.5.2 Construct accounting

Table 5.20 gives the construct accounting in four parts, because a single ratio cannot separate three different things. A site is *recovered* when it becomes a transition with a destination. It is *dynamic* when the extractor records the destination as unresolved: a value from the backend or a history entry, but also a field it treats as mutable although the route table initialises it, or a template literal inside a commands array (Section 6.3). The graph then records an edge into that screen's `u_` sink with modality unknown, rather than dropping or guessing. It is *unplaced* when the destination is readable but the screen it fires from is not, and the edge is anchored at an `o_` origin node that says so. What is left carries a recorded reason but no transition. The check enforces that a reason is recorded, not that it is correct.

43 sites leave the denominator, on three grounds decided from the construct's own text before the graph is consulted. The set is therefore the same in both runs, and the figures are not produced if the grounds do not sum to the total. 35 pass an empty commands array and so change only the query of the current route, and 1 names no destination. Neither is a transition between screens. 7 navigate a secondary outlet. These are transitions that the model represents as a second mount (Section 3.2.3) and the extractor does not implement, so they are a declared recall limitation. Excluding them raises alfresco's ratios. Both runs use the same ledger, so the two columns of Table 5.7 are ratios over the same 1775 sites. The comparison refuses to print if any construct changes its ground between them.

k8s dashboard shows why recovered and dynamic must be reported separately: a low recovered fraction can coexist with many sites whose destination depends on runtime data. Its dynamic sites are spread over 42 files, 37 of which contribute a single site, so the column reports a habit of the codebase and not one place to fix. mifos supplies 75.7 % of the in-scope sites on its own and pulls the corpus figures towards its own: without it, recovered is 50.5 % and accounted 97.9 %. Changes to the route denominator are recorded in Section 4.3.2.

The named-outlet restriction is enforced on the route side as well: a route that declares a named `outlet` is reached as `(name:path)` and never at the primary address, so it contributes no screen, and the restriction reaches a named outlet's *subtree* too. alfresco's viewer routes were minted at the primary address, which Angular reaches only as `(viewer:...)`. Both tools read the subtree the same way, so the coverage figure could not expose it, and the runtime route oracle of Section 5.5.3

Table 5.20: Construct accounting after the loss list was worked.

Application	Sites	Excl.	In scope	Recovered	Dynamic	Unplaced
realworld	40	1	39	100.0 %	0.0 %	0.0 %
alfresco	34	7	27	25.9 %	25.9 %	44.4 %
ghostfolio	198	12	186	47.8 %	43.5 %	7.5 %
k8s dashboard	72	4	68	17.6 %	70.6 %	8.8 %
bitwarden	131	19	112	63.4 %	14.3 %	18.8 %
mifos	1343	0	1343	90.5 %	6.4 %	2.5 %
All	1818	43	1775	80.7 %	13.4 %	4.8 %

Note. Percentages are of the in-scope sites.

was needed.

5.5.3 The final tier and the node set

The final tier. The attributed must tier has 0 witness-check failures among 159 edges, with 62 destinations unread, which are neither passes nor failures. The unread destinations are the toolbar edges of one application, whose source side rests on the scope of its chrome component and on the route-order assumption. The witness check compares the destination a construct names with the destination its edge names; it does not ask whether the construct runs every time or whether the router lets the navigation through, which the demotion rules cover. Neither the source reading of the must tier, made once by the AI assistant with the extractor's decisions in view, nor the mechanical check is an independent proof of Claim 3.1.

The whole graph. Over the whole graph the check says much less. The 15653 edges cite 17801 locations, and only 6.1 % are clean, because 89.0 % are over-approximations for which the check claims no target agreement. 34.8 % of all edges (5454) come from one call site in mifos, `view-transaction.component.ts:135`, a `navigateByUrl` to a template literal with an empty static prefix. Of the 1437 locations the check judges, 76.1 % are clean, and none of the rest is cited by a must edge. A precision figure for the must tier is not one for the graph.

The node set. Across the corpus the compared graph declares 0 screens at addresses the framework mounts nothing at. Every address the extractor claims is an address the framework's own configuration mounts, which is external evidence about these six applications that the two static readers could not produce, since an address both place at the wrong prefix is one they agree on. The evidence is a precondition of the must-tier property and not part of it, and it is bounded by what a census keyed on addresses can represent: entries on a named outlet or decided by a `UrlMatcher` (65 route entries) are counted in neither column.

The runtime route oracle. The oracle loads each application, built in its development configuration, in a headless browser, reads the live router's route configuration after expanding every lazily loaded subtree, and counts an address as mounted when the configuration places a component there on the primary outlet. No route file is parsed. Against the 1146 addresses it mounts, the node set is 892 of 1146. The oracle ran on an earlier graph (29–30 October) with the same node total, one corrected using earlier findings of this check. The oracle's classification was itself checked by navigation and agrees on 41 of 41 addresses; the 251 unminted addresses are not individually checked.

5.6 Dead links, found from the source

A site whose target the application mounts nothing at is a dead link. The method finds candidates for such links at no extra cost. The candidates are found statically, and each is then checked against the route configuration the runtime route oracle recorded before it is reported. The result is a candidate list and not a verdict: no candidate was navigated to in a running application, so that a link is reached at run time is not established.

The clearest case is mifos. A transactions tab links to a journal entry at the address `/journal-entry/view/:id`. The application declares that screen at `/accounting/journal-entry/view/:id`, inside the accounting module. The link omits the parent prefix, so the URL it builds is one no route matches. mifos declares a catch-all route to a not-found component, so the link does not fail invisibly. It renders the not-found screen, which a user can mistake for missing data rather than a broken link. The same shape repeats across the application: links to `/loans-accounts/...` and `/savings-accounts/...` name subtrees declared only under an owner, and further calls name segments the application mounts under other names. A test that never opens the tabs holding these links would not find them, and a crawler would find them only if it reached those screens. Reading the route configuration finds them without running anything.

The list is not one defect class, and the split matters for what each part is worth.

- 18 of the candidates are an absolute link written against a path the application does not mount, usually the name the server uses rather than the one the router does. Those are the application's.
- 5 come from a *relative* `routerLink`, such as `[routerLink]="'edit'"` or `[routerLink]="['../../transactions']"`. Their target depends on the route the link resolves against, that of the component holding it (Section 3.2.6), so a wrong base is the analysis's question before it is the application's defect.
- 4 are neither, because the result files do not record enough of the construct to say which it is. They are counted and not classified, rather than assigned to the half that would flatter the finding.

The last two groups are reported as candidates and weighted lower. Counted as places in the source rather than as targets, the three groups are 18, 3 and 4; the difference is one relative link in a shared template that resolves against three owning screens.

A further 69 sites are not published at all, because they sit outside the directory the measurement is scoped to, in ../../libs, ../aca-shared, ../browser, in the repositories of 2 applications. Most are the source of a *sibling front end*, a browser extension whose own router declares the same addresses a census would have called dead, so publishing them would report one application's valid links as another's defects. They are counted here and left out of the census.

Chapter 6

Discussion

This chapter interprets the results of Chapter 5; it does not repeat them. The thread running through the chapter is that the extractor loses transitions in two places, at the nodes and at the edges, and that each level limits what the other can achieve.

6.1 Key findings

Two places where extraction fails. Static extraction of a transition graph from Angular source fails in two places, and the first turned out to be the tool's rather than the method's. Node extraction is complete on all six applications against the readers of the route configuration (Table 5.3). The runtime route oracle observes no node placed at an address where the framework mounts nothing. That shows that no such error was observed, not that none can occur. The runtime route oracle also bounds the first claim, and node completeness has two readings.

- Against the readers of the route configuration, node extraction is complete on all six applications. The loss classification is computed against this weaker reading.
- Against the mounted addresses, the route configuration the oracle reads holds 251 addresses with no node, 242 of them in ghostfolio (Section 5.5.3). Only 2 of the six are complete in this stronger sense of a node for every mounted address, which is what a user of the graph would want.

The node defects (constant paths, lazy boundaries) are traced in Section 5.2. The final version reads their causes (Section 5.5), and those losses are empty.

What remains is lost at the edge. The navigation sites exist in the source and the screens they would connect exist in the graph, but the extractor cannot decide which screens a site belongs to. The exceptions are sites that the diagnosis labels as having a target screen with no node. The label also covers targets held in a field. Most of these sites are in ghostfolio (Table 5.10). Scope attribution addresses this loss at the edge. Under the order in which the loss rules are applied it is the largest loss category in every application (Table 5.5). That order, however, is part of the measure (Section 6.2). The ranking is a fact about an extractor, not a limit of the approach. The two levels are not independent. Attribution cannot draw an edge

into a screen that has no node. A complete set of nodes yields few transitions while the sites outside the component a route renders stay unattributed. Reporting only one level would have described a tool that works better than it does, which is why Chapter 5 reports node coverage beside must conversion.

RQ3 and what the loss rules can delimit. A lost construct is assigned to the first loss category that applies (Table 4.3). A construct inside a shared component that appears only in screens without nodes is therefore counted as an attribution loss, although attribution alone cannot recover it. The categories are a diagnostic partition of the losses, not a bound on what attribution recovers.

The demotion rules decide whether the edges attribution added at k8sdash (its toolbar edges, Section 5.5.3) occur on every input. 125 of them are kept at must. The demotion rules were written during development, in part in response to objections from red-team panels run with the AI assistant. The witness check finds 0 failures among the 159 must edges. It cannot read the destination of 62, all of them k8sdash's toolbar edges. So only about half of that application's must edges are read.

RQ4 is conditional, and the condition is RQ1. Leave aside the rename operator, whose detection is close to guaranteed by construction. The other three operators are each generated or scored only at constructs the graph already represents in the simplest form the extractor reads. They are detected in 45 of 45 scored mutants on the attributed graph (Table 5.19), split as follows.

- Deleted `routerLink`: every one is detected.
- Retargeted calls: 20 of 20.
- Removed guards: 11 of 11.

The mutation harness was corrected during development after its misses had been read (Section 4.3.2), so the clean result is not an independent test. It does not show that the remaining operators would survive a larger or differently drawn sample.

For guards, what separates a detected mutant from an unobservable one is whether any edge enters a screen the guarded route yields, which is the exclusion rule (Section 5.4). The class "guards unreliable in both directions" of Chapter 5 rests instead on what Section 5.5 measured. These are a guard carried by a route a wrapper function returns, and a `canActivate` element written as a call. Source review in red-team panels found those defects, not the witness check. The mutation run neither confirms nor refutes the class.

Detection is also conditional on recovery. The conclusion is therefore as follows. Given that a construct is represented in the graph, every scored change to it was visible. The probability that it is represented at all is what RQ1 measures. For change impact analysis this form of claim is appropriate, because an impact query is only asked about the part of the application that the model represents. The graph states which part through its witnesses.

Two uses of the graph. The first, with measured support, is change impact analysis. Every edge carries the file and line of the construct that produced it.

Changed lines therefore map to the edges whose witnesses fall inside them. From there they map to the screens those edges leave and enter. This is the screen-level counterpart of Chianti [61], in the impact analysis tradition of Arnold and Bohner [4, 12, 43]. RQ4 shows that the mapping reacts to the injected edits for the constructs the graph represents. That is a property such an analysis needs, not a measure of its accuracy, which would ask whether every affected screen is named and how many are named in excess.

The second, not itself measured, is checking in continuous integration at no execution cost. The graph is recomputed from source on each commit, deterministically and without installing, serving or crawling anything (Section 4.3). Properties such as "a screen still has an inbound deterministic transition" can then be asserted like a type check. A guard assertion is the member of that list the graph cannot yet carry, and for a screen no edge enters there is nothing to read. A dynamically built model cannot be used this way at the same cost, because it needs a running application, test data and an exploration budget [50, 51, 36].

Where both uses stop. Both stop at runtime dynamic routing: a target computed from a value, a route table assembled at startup from configuration, and an overlay no route describes. There the static graph is silent or over-approximates into the may tier, which carries no soundness claim. Under the assumptions of Claim 3.1 the must tier is a lower bound on the navigation an application can perform; the may tier is not. A query that must not miss a transition therefore cannot be answered from the graph alone. The graph's witnesses and declared approximations tell a consumer which queries those are. Authentication bounds the deterministic tier in the same way. A guard decides on state the source does not fix, so a reader that keeps Claim 3.1 demotes it. The small must tier is in part this fact and not a defect of the extractor. Taking every guard to let the navigation through raises the tier from 159 to 203 edges (Section 5.5).

6.2 Limitations and threats to validity

The results are measurements of one tool, on one corpus, with five methods and an end-to-end check on two applications, performed by the person who wrote the tool. Each of those four choices limits what the figures support. Table 6.1 lists each threat with its effect on the results, what was done about it and what remains open.

Table 6.1: Threats to validity: effect on the results, mitigation applied, and what remains open.

Threat	Effect on the results	What was done	What remains open
Internal validity			

Threat	Effect on the results	What was done	What remains open
Tool developed with the evaluation, on the same corpus	uigraph was built for this thesis and corrected while it was evaluated (Section 4.3.2). Every figure measured on the six applications is in-sample and fitted to this corpus; the extractor, the independent route parser, the mutation harness and the structural reference were corrected against the applications they are scored on. The structural reference guided two repairs of the extractor, and the end-to-end check three.	The end-to-end check came closest to an out-of-sample test: the extractor was fixed before the suites' logs were read, and held 12 of 22 mifos pairs. The repairs it exposed make the final figure in-sample like the rest. Corrections are reported in full.	The archive cannot corroborate that order, since the preregistration and the logs were committed together. The corpus figures should not be read as estimates for other applications.
Dependencies not installed	Extraction runs on source trees whose dependencies are not installed, so the TypeScript compiler cannot resolve types from the framework or third-party packages. Type-directed steps such as narrowing an injected receiver are weaker.	Reported as a property of the configuration.	The figures are properties of that configuration only.
Mutation experiment	Small samples per operator, in single digits for several operator and application pairs. The exclusion rule uses the graph itself, which couples the sample to the tool's own recovery.	Mutants were drawn with a seeded quota, so the pattern of which operators are detected is more trustworthy than the rates [57]. The excluded mutants are reported.	The sample is not independent of the artefact under test.
Test-source cross-check	Measures node existence and nothing finer; covers one application.	Used as the only method that sees past the syntactic proxy (below).	It reads the specifications without running them, so a URL there is named and not observed.
Scope attribution			

Threat	Effect on the results	What was done	What remains open
Scope compared in both directions on two applications only, against the analysis's own relations	Every other method of Chapter 4 checks a node, an edge or a witness; none can be asked whether the set of screens attribution computes for a site is the set it fires from. Three methods bear on it, each partly (next three rows).	Three partial methods, below.	A site attributed to too *many* screens produces edges whose witnesses all pass, because the construct really is there and really names that target. Outside the two references the rendered-interface check cannot convict over-attribution, and the direction is unrefuted on the other four applications.
Hand-read structural reference (Section 5.3.4)	Answers in both directions, but for two applications of six and only as agreement with the analysis's own relations. On alfresco the full analysis recalls 90.9 % of the pairs off the shell (23.6 % overall) at 82.0 % precision, below the class join's 92.3 %. On realworld it finds 100.0 % of its (site, screen) pairs at 100.0 % precision. The *class* level, the per-class rule the call graph replaced, asserts 9 pairs outside it at one site.	The call graph was written after that finding.	It is scored on the same ground truth, so the clean figure is consistent with the repair but does not test it independently.
End-to-end check (Section 5.3.5)	Asks the same half of the question of the navigations two applications' own suites perform. On realworld it does not discriminate: the shell-rule level, which stands for earlier work, already contains all 19 observed pairs (19). On mifos the same level holds 17 of 22 and the enclosing-route level 20, as many as the full analysis (Section 5.3.6), in a graph that holds 163138 of 367236 possible pairs.	Hits and misses are reported in Section 5.3.5.	Two applications only; the mifos figure after the repairs is in-sample.

Threat	Effect on the results	What was done	What remains open
Rendered-interface check (Section 5.3.1)	Covers 5 of the six applications and answers half of the question; ran on 30 of 806 screens.	Ran on an earlier graph than the final call graph.	A predicted link missing from the DOM is equally a link rendered only to a signed-in user, so it cannot convict over-attribution.
Construct validity			
Denominator of must conversion (Section 4.2)	Over-counts, since it contains constructs in code the application never mounts, and under-counts, as the independent route parser recognises navigation by four syntactic patterns only: a `routerLink` attribute or binding, and a call to `navigate` or `navigateByUrl` on a receiver whose name contains *router*. Helper-wrapped navigation, assignments to `location.href`, server-driven redirects and navigation inside third-party components count as neither a loss nor a success. The figure is not a bound in either direction.	The ratio is read as recall against an enumerable syntactic proxy. The test-source cross-check is the only method that sees past it, because a URL the application's own end-to-end specifications name is an address its authors expect to exist.	The completeness of the proxy is not itself measured.
Denominator of node coverage	Its denominator is the screens the independent route parser can address, so a screen whose path neither tool can read is outside the ratio and not counted against it. Refusing to read more raises reported coverage.	The runtime route oracle supplies a second reading (Section 5.5.3).	The stronger threat, in the opposite direction to the one above.
Witness check	A construct counts towards must conversion only on evidence the check has read. It is mechanical and re-parses the source at the claimed location.	Judges every location an edge cites.	It checks the witness and not the address: an edge whose target node sits at a path the application does not mount satisfies it.

Threat	Effect on the results	What was done	What remains open
Precedence of the loss rules	The order of the rules is part of the measure; testing the target first would make the target category larger than attribution on 2 applications (alfresco and k8s dashboard).	Category sizes are compared only within one application and under one order, which is what Chapter 5 claims.	Cross-order comparison is not supported.
External validity			
Corpus and selection	Six open-source Angular applications pinned to single commits; proprietary applications may differ in size, routing style and navigation mechanisms [34, 35]. The selection is tool-driven: `SAP/spartacus` was rejected because its routing is configuration-driven in a way the extractor does not support (Section 4.1). Figures are likely optimistic for Angular applications at large.	The six selection criteria were fixed before any results, and rejected candidates are reported with their reasons.	One rejection was on tool support, a ground outside the criteria. Two retained applications do not meet the test-suite criterion (Section 4.1).
Uneven outside evidence	URLs named in end-to-end test sources (read and not run) come from alfresco alone. The runtime router configuration was read on all 6 applications. Rendered links were observed, on an earlier graph and in part, on 5. The end-to-end suites were observed on realworld and mifos, covering 22 distinct transitions of an application with $\geq$909 route entries.	–	The findings about scale rest mainly on the runtime configuration and the extractor's own reading.
Angular only	The results are Angular results; the graph model and the lattice do not mention Angular.	Section 6.4 argues about other routers.	That argument is not a measurement.
Oracles and experimenter			

Threat	Effect on the results	What was done	What remains open
Oracle independence	The independent route parser shares no code with the extractor, which makes it independent and says nothing about whether it is right. For the route-path forms added during development (a table behind a freezing call, a path assembled from constants, a module reached through a workspace alias) the two tools apply the same rules (Table 4.5), so their agreement is weaker evidence.	Each disagreement was resolved by reading the source.	It was resolved by the person who wrote both implementations; a differential test only shows that two implementations disagree, never which is right.
Experimenter bias	I wrote the extractor, designed the evaluation, selected the corpus and defined the defect classes.	Commits and harness version are pinned, and the result tables are generated from result files (Section 4.3); the two tables compiled by hand are the defect-class table and the table of the witness check and the test-source cross-check. The protocol corrections are reported in full, including those that made the figures worse (Section 4.3.2).	No blinding, no second annotator for the loss classification, and no independent audit of the residual constructs inspected by hand; the defect classes were defined after the first observations, not in advance.
AI-assistant reading of the final must tier	The source-level reading of the final must tier (its 25 constructs) was made by the artificial intelligence (AI) assistant declared at the front, which also helped write the extractor and the independent route parser. It was made with the extractor's decisions in view.	–	No human second reader checked it. Its agreement with the extractor shows consistency with the stated rules, not independent correctness.

Threat	Effect on the results	What was done	What remains open
Two lines of evidence per defect class	This requirement is the strongest control on the fitting threat and the reason one class is still a candidate, but it is weaker than it appears: Table 5.4 shows that four of the five accepted classes rest on one named method, the rest of their evidence being observations.	Requirement of two lines of evidence per class.	For those four the control is one method plus my own judgements.
The runtime route oracle (Section 5.5.3)			
Oracle selection effect	The oracle removes the threat that mattered most, that two readers of the same route files agree on a wrong address and the agreement is read as a result. The method needs an application that builds and boots, and a corpus chosen partly for being buildable is not chosen at random.	–	The selection is not random.
Configuration, not rendered page	It reads the route configuration, so a route behind a guard that always refuses, or whose component throws on render, counts as mounted.	–	Makes the framework's set larger than what a visitor can reach, which makes containment easier to satisfy.
Development build	The build was a development build in each case, because Angular removes the debug surface the method reads in production mode.	–	A production configuration can differ in ways a route table does not show; it can add routes as well as drop them, so the effect has no fixed direction.
Workspace scope	Where a workspace application composes the library a measurement is scoped to, the framework's set is a superset of the library's, and part of any difference is scope and not reading. Two of alfresco's three differences are of this kind.	–	Also makes the framework's set larger, which makes containment easier to satisfy.

The oracle threats do not make a mounted address look unmounted; they weaken

the claim in the other direction. The claim is that an address the extractor mints is one the framework mounts, tested by finding the extractor's set inside the framework's. Both results are statements about the development router configuration that was recorded, under the abstraction of Section 5.5.3. The compared graph mints 0 such addresses, which is the absence of a failure on that configuration and not a proof of its impossibility.

6.3 Declared approximations

This section lists the constructs the analysis does not decide, so that a consumer of the graph can tell which queries it may ask (Table 6.2). The list separates constructs read exactly, where a failure is a defect that Chapter 5 counts, from constructs approximated, where no completeness is claimed. Constructs of the approximated group still count in the construct denominator of Table 4.2, where they appear as dynamic, unplaced or attribution losses. Before the repairs of Table 4.5, the end-to-end check found one such cost on mifos, the base of a relative link in a template shared with child tabs; no remaining miss is of this kind (Section 5.3.5). The first two rows were found by the runtime route oracle of Section 5.5.3 and not by reading the analysis. Both are places where the analysis is silent rather than approximate.

Table 6.2: Declared approximations and exact-read constructs, and what the graph loses for each.

Construct	How it is treated	Consequence (tier, missing edges, note)
Found by the runtime route oracle		
Route declared outside the language	alfresco's about page is registered by a JSON manifest that names a path and an identifier, which an extension registry binds to a component class at run time. No reader of the application's TypeScript can see it; covering it would require reading the framework's plugin format as a second source language, which is not attempted.	The route is absent from the graph and no note is raised, because nothing in the code is unreadable.

Construct	How it is treated	Consequence (tier, missing edges, note)
Table chosen outside the sources	mifos holds two route tables for its users module, one per identity provider, and selects between them with a constant assembled from `window.env`, injected at deployment. Where the sources fix the condition (a flag in an environment file) the extractor reads the arm it selects; where they do not, as here, it declares the subtree unread. The value read is the one the checked-out sources carry, and a production build with another environment file may select a different arm; the oracle reads a development build for the same reason, so the two agree about which configuration is measured.	Declared unread subtree, 5 screens wide.
Read exactly (the analysis's responsibility)		
Literal routes, links, calls, nesting, outlet, guards	Read exactly: route entries with a string-literal path and the component they render; `routerLink` bindings whose value is a literal or an array of literals; `Router.navigate` and `Router.navigateByUrl` calls with a literal target; the nesting relation from element selectors resolved within the importing file's scope; the primary `<router-outlet>` with the shell rule that follows from it; guards named on a route, as labels on the edges entering that route's subtree.	A failure is a defect that Chapter 5 counts (constant route paths, lazy boundaries, unlinked nodes, Section 5.2): bugs with known causes, which the final version reads (Section 5.5), not approximations.
Approximated (no completeness claimed)		
Components created programmatically (dialogs, overlays)	An overlay is a state no route declares, so it has no screen node, and the component implementing it is reached from no template, so its sites have the empty scope. With attribution each such navigating component gets an unplaced origin carrying its file, and its sites become edges from that origin at *may* (Section 3.1).	Missing: the overlay state, the transitions into and out of it, and the screens from which its exits fire, not the destinations of those exits. A navigation in a service that only such a component injects is recorded only as a note.

Construct	How it is treated	Consequence (tier, missing edges, note)
Named outlets and `{outlets: ...}` targets	An auxiliary route adds a second active route and mount point, encoded as `(name:path)` [27]. The model of Chapter 3 represents it as a second mount; the adapter does not implement it. A route naming a non-primary outlet and every route beneath it mint no node; so does a route whose outlet the reader cannot fix to a string. In `navigateByUrl` an auxiliary group is dropped and the edge to the primary path is held at *may*, since the navigation fails when the auxiliary route does not match and that route is not read.	The last route kind is counted in the note that marks the route set incomplete. A commands array with an `{outlets: ...}` element yields no edge and no note; a group that also carries the primary outlet yields nothing. A later string element of a `navigate` command is one segment whatever it holds, so an auxiliary URL written there reaches whichever route matches it as one segment; the test-source cross-check shows such URLs among its misses.
Navigation relative to the active route (`relativeTo`)	The target depends on the route of the component holding the site. A template `routerLink`, and a `Router.navigate` call whose options are an object literal with a `relativeTo` key (written out or held in a `const`), are resolved against the route that renders that component on the chain above the screen the edge fires from (Section 3.2.6), applying a leading run of `.` and `..` segments; a call without `relativeTo`, and `navigateByUrl`, are resolved against the root.	Held at *may* where no route on that chain renders the component (a component nested in another's template), where `relativeTo` is written in shorthand, passed through any other variable or given another route, where a link binds its own `[relativeTo]` input, and where the edge leaves the shell. Such an edge can name the wrong screen, which the may tier admits and the must tier does not.
Templates passed as values (`TemplateRef`)	The template value is not followed from declaration to instantiation. Links inside an `<ng-template>` are read from the declaring template and attributed to the declaring component's screens.	Every such link is held at *may*, since nothing says whether or where the template is instantiated. When the value is carried to a component on other screens, the edges stay with the declaring screens. Recovering the place of instantiation is a dataflow problem, the same obstacle as for JavaScript XML (JSX; Section 6.4).
Dispatch through NgRx effects and router event subscriptions	The dispatch relation is specified in Chapter 3 and not implemented, so the screen that dispatched an action is never recovered. At the levels that keep the call graph or its class join (*full*, *class*) every navigation call in a file inside the application directory that contains a `createEffect` call is given $\top$ and offered from the shell at *may*, or into the shell's `u_` sink when its target is fully dynamic. At the lower levels the stand-in is not applied.	Sound and coarse, since an effect can fire on any screen. An effect outside the application directory, or declared with the older `@Effect` decorator, is treated as any service call.

Construct	How it is treated	Consequence (tier, missing edges, note)
Navigation inside signal effects and observable subscriptions	The firing condition is a data dependence in a graph the runtime builds, not a control-flow condition in the program text (Sections 2.7.5 and 2.8). The model gives such edges the modality *unknown* with the read set as symbolic guard; the extractor implements neither. A navigation inside an `effect` body, a `subscribe` callback or any other function something else calls is attributed to the scope of its class (Section 5.5).	Held at *may*, because it runs only when that function is called. A fully dynamic target goes to the `u_` sink. The graph records that the screen can be left for that destination, not under what condition.
Computed targets, no static part	A target built at run time from values cannot be enumerated statically in general. A target with no static part, such as a `routerLink` whose value is an interpolation among them, is an edge into the `u_` sink of the node it fires from.	Modality *unknown*, confidence 0.
Computed targets, static leading part	A static leading part (the head of a template literal, the left operand of a concatenation, or a commands array that ends in variable elements, read up to them, where a variable element before them, or an interpolation of a template literal that opens the array, stands for any one segment) gets one edge to every declared route that extends that prefix by at least one segment. An empty prefix connects the site to every declared route except the root.	Over-approximated into the *may* tier, and incomplete: a static prefix segment matches only an identical route segment, so a route with a parameter in that position is not a candidate.
Computed targets, commands array with trailing constants	An array whose variable element is followed by constant elements is read as a pattern and drawn as a single edge to the first declared route that matches it.	*May* tier. The variable element matches only a parameter segment, so a route whose segment in that position is a constant is not reached, and no note records the omission.
Injection tokens and provider bindings	The call graph of Section 3.3 resolves a call on a field through its declared type. A binding is taken application-wide and not from the injector that resolves the token, so where two components provide a token differently both implementations are joined. A factory or value provider, and a provider array a function returns from outside the application, bind nothing the pass can follow; such a call falls back to the join over every component that injects its class, or to no scope at all if none does.	Joining widens a scope and never narrows it; an unfollowable token contributes no edge. Over the corpus the pass resolved 5574 calls on application types to the class the type names, 4937 through a provider binding and 18 to subclasses, and left 1111 unresolved, in 2 of the six applications.

Construct	How it is treated	Consequence (tier, missing edges, note)
Configuration-driven route registration	A table passed as an array to `RouterModule.forRoot`, `forChild` or `provideRouter` is read wherever the registration runs, including inside a provider factory the application calls. A route that exists only once configuration has been evaluated at startup is not recovered. Where a route object is declared and its `path` is supplied by configuration, the path is counted as unreadable and no node is minted.	The route set is declared a lower bound in that case. Where the table is computed by a function and passed to a registration call, provided through the `ROUTES` token, installed with `Router.resetConfig`, or declared in a JSON extension manifest (Section 5.5.3), the routes are absent and no note is raised, so the route set is not declared a lower bound on their account. The independent route parser behaves identically, so the two readers agree on routes neither can see. The same approximation is why one candidate application was rejected (Section 4.1).
Guards that would let the navigation through	A guard is a function of the application's state and is not evaluated. It is identified by the text of its expression, so two guards that compute the same thing are two guards. Content projected through `<ng-content>` is read as its host template places it. Only `mat-menu` and `mat-tab` are read as rendering content lazily. A route component behind a conditional outlet is conditional, but a component nested in it is judged by its own placement, and the application root's outlet is not read.	A guard the navigation runs holds the edge at *may*, even when the guard would always pass. A link projected into a component that renders it only under a condition is read as unconditional.
Navigations a later one may cancel	Part of the execution model of Claim 3.1, which holds only on executions this row and the modifier-key row do not describe. A navigation is demoted when code that runs after it, before the event is handled, may start another (Section 3.1). Whether that code may navigate is read through calls resolvable to a body in the application: a member of the class, a function of the file, or a method the checker resolves on a typed receiver.	A call into a library, through a receiver the checker cannot type, or into code loaded at run time is taken not to navigate, so a navigation followed by it can be published as must although the call cancels it. A navigation scheduled for later, and the other listeners one event reaches, are not followed.
Children of a route a function returns, at the empty path	The first empty-path child under a screen, and the first below that one, render on the screen's own address and their links are drawn from it. Where the empty-path route is written inside a function that returns it, as `withShell([...])` does, the walk does not place the component the function's children name under the screen above.	Its links stay on an unplaced origin at *may*. The graph loses those edges and asserts nothing false about them.

Construct	How it is treated	Consequence (tier, missing edges, note)
Activation with a modifier key	A `routerLink` on an anchor leaves a click made with a modifier key held, or with a non-primary button, to the browser, which opens the address in a new browsing context; so does an anchor whose `target` names another context. The model reads every activation of a link as a navigation of the application.	A must edge from an anchor holds for a plain activation and not for these.
Order in which Angular matches routes	The router takes the first route, in declaration order, that matches an address; it prefers an exact path to a parameter route and consults a redirect only when no route matches. Route tables an application registers apart are joined in the order their providers are registered, which the extractor does not read; it takes a catch-all in one such table to be matched after every route of the others, and names each catch-all it dismissed this way in a note.	Wherever two declared routes, or a route and a redirect, can answer an address, the edge is held at *may* and may name the route Angular does not take. The one exception, the catch-all assumption, rests on the unread registration order; the must edges that rest on it are counted with Claim 3.1 in Section 3.1.
Ambiguous selectors in the nesting relation	When a selector belongs to more than one component, the candidates are narrowed to those the rendering file imports; where none is imported, the analysis joins over all of them.	A note is recorded; a site can be attributed to screens on which its component is never rendered.

Note. "May" is the tier without a soundness claim.

6.3.1 Why the may tier is incomplete

The may tier is read as an upper bound (Section 3.1), so a transition the application takes and the graph lacks is its defect, while an excess may edge is not. Table 6.3 sorts every transition still missing in the two suites (Section 5.3.5) and the two structural references (Section 5.3.4) by where its destination is fixed. *Inside* the static boundary, the frontend source, a library under `node_modules` or configuration committed to the repository fixes it, so a better reader recovers it: these rows are known limitations of this tool, not of static extraction. *Outside* it, the backend supplies the target or the route, configuration is fetched at run time, or user input changes the destination screen. A third class, *held*, is a suite pair the graph holds on the screen a redirect lands on, which the preregistered rule scores on the address asked for: no reader turns it into a hit, so it counts toward neither side. On alfresco every missing (site, screen) pair of the reference lies inside (133 of 133); on mifos 1 of the 2 missing suite pairs lies inside and 1 is held; realworld misses nothing in either measure. A reader that recovered every inside miss would hold 100.0 % of alfresco's reference pairs (now 23.6 %) and 95.5 % of mifos's suite pairs (now 90.9 %). This is the static ceiling on these four measures, and on the references it is weak evidence:

Table 6.3: Why the graph still misses transitions: every remaining miss, grouped by gap.

Gap	Example	Boundary	Fix	Pairs
Shell from a library (alfresco)	`src/lib/components/sidenav/components/sidenav-header.component.html:20`	inside	read the shell route that @alfresco/adf-core/shell provides under node_modules and the extension JSON in the repository that fills it	129
Routed component nested in another screen's template (alfresco)	`src/lib/components/files/files.component.ts:143`	inside	attribute its sites to the screens of the component that nests it as well as to its own routes	4
Redirected navigation (mifos)	`src/app/login/login.component.ts:121`	held	none in the reader: the graph holds the pair on the screen the redirect lands on, and the rule scores the address asked for	1
Default commands of a service navigation (mifos)	`src/app/clients/clients-view/client-actions/client-action-notifier.service.ts:34`	inside	read a parameter's default value and the route each caller passes as relativeTo	1

Note. Covers the two suites and the two structural references. *Pairs* counts missing suite pairs and missing (site, screen) pairs of a reference.

both apply the analysis's own relations (Section 5.3.4), so none of their pairs can lie outside the boundary, and realworld's was partly completed from the graph's output. Only the suites can show an outside miss, and on two applications they show none. The ceiling does not reach causes these measures do not exercise, such as the two Section 5.5.1 places beyond any static reader of a route table. It bounds the missing direction only: the excess of the may tier is not scored here, and the mifos graph holds 163138 of the 367236 possible pairs (Section 5.3.5). The classes are the author's reading of each miss, recorded with its site in the repository; the script that joins them to the scores refuses a miss without a class and a class without a miss.

6.4 Transfer to other frameworks

All measurements in this thesis concern Angular. This section argues how much of the approach would carry over to React with React Router and to Vue with Vue Router [60, 80]. It is a qualitative analysis of the routers' documented behaviour, not a measurement; showing that the approach performs as well on React or Vue would require a corpus for each framework.

The analysis has a framework-independent part and a framework-specific part. The graph model, the lattice of scopes, the fixpoint, anchored edges and the expansion

query do not mention Angular. What is specific to Angular are the four input relations of Section 3.2: component nesting, router outlets, the call graph with dependency injection, and dispatch relations. Moving to another framework means deriving the same four relations from that framework's constructs.

Outlets transfer directly: in all three routers a parent route renders a placeholder where the matched child is inserted. The components above the placeholder form a shell that is present on every child screen, so the scopes $\top$ and $\mathit{subtree}(r)$ have the same meaning.

The call graph is easier in React and Vue, because a hook or composable gives every navigation site an owning component and no injector needs to be resolved. Code that imports the router instance directly is the analogue of Angular's service case and can be resolved by ordinary import resolution, as in the JavaScript call graphs of Feldthaus et al. [23].

Redirects do not transfer, and the asymmetry was found by porting the model's reading. Angular's validator refuses a route that declares `redirectTo` beside `children` or a `component`. A redirecting Angular route therefore has neither a layout nor a component to lose, and the model's step from "redirects" to "renders nothing" costs nothing. Vue Router applies a record's redirect only when that record is the last one matched, so a redirecting parent with children does render its component for a matching child. Carrying the Angular reading into the Vue extractor dropped those components. The extractor now keeps the component of a redirecting record that has children. It declares one residue as a note at the parent rather than modelling it as an edge: navigating to the parent's own address lands on the redirect target with that component unmounted. The same porting exposed a second Vue-only hazard. A table that mixes the Vue 2 catch-all `'*'` with the Vue 3 form `'/:pathMatch(.*)*'` folds both to one address. The extractor keeps the record declared first. Where that is the Vue 2 form, which Vue Router 4 rejects, the surviving component is the wrong one. The collision is declared, but which record to keep is not something the table alone determines.

Guards move to different scopes. A React Router loader redirect plays the role of `canActivate`, so its scope is the join of the origins of the navigations that may run the loader, as the relation T gives for entry guards, and not $\mathit{subtree}(r)$. Vue's global `beforeEach` guard corresponds most closely to a `canActivateChild` on a root route with no component, anchored at $\top$.

Component nesting is harder in React. Angular and Vue keep templates in a separate language, so the nesting relation can be read from element names. In React, components are created in JSX, which is ordinary JavaScript. A component can be stored in a variable, passed as a property or as `children`, and rendered under any condition the language can express. Recovering the nesting relation then becomes a data flow problem closer to points-to analysis than to reading a template. This is the main place where a React version would need new work.

The lattice, the fixpoint and the anchored representation transfer unchanged. Of the relations, outlets and guards transfer with small changes, the call graph becomes simpler, and component nesting becomes harder in React. These are expectations derived from the routers' documented behaviour, and testing them on

real applications is left for future work (Section 6.5).

6.5 Future work

This section looks forward; the development versions and the corrections between them are listed in Section 4.3.2. The topics are listed in the order of how much each would change what the thesis can claim.

An oracle for the screen set on a large application. This is the cheapest addition that would strengthen the thesis most. The hand-built ground truth of Section 5.3.4 covers only the smallest application. Reading every site by hand is not practical on an application of hundreds of screens, where most of the corpus's (site, screen) attributions are made (Table 5.12). Two routes lead there: driving such an application into a signed-in state and reading its rendered links, or annotating a stratified sample of its sites instead of all of them.

The unimplemented part of the model. Chapter 3 specifies two things the prototype leaves out.

- The dispatch relations that carry NgRx effects and router event subscriptions.
- The injector hierarchy that the implemented call graph reads past, since it takes every provider binding as applying application-wide (Section 6.3). Reading bindings relative to the injector that resolves them would narrow the scopes, and the fixpoint already covers it. The call graph follows class hierarchy analysis in the style the framework-modelling literature recommends [17, 67, 3].

Guards that return a `UrlTree` belong to the same group, since a redirecting guard is itself a navigation site and not only a label. The wider treatment of guards in Chapter 3, where the guard of an edge is the conjunction of the conditions on the containment path with the guards the router runs, is the more substantial piece of work. Its test is the guard-removal operator, which on this corpus reaches only guards the graph already records on an edge (Section 5.4).

A runtime tier. The runtime tier is the complement to the whole static approach and not an extension of it. The model already admits observation identifiers as witnesses beside source locations. Setting that tier against the dynamic lineage, where the state flow graph comes from firing events on a running application [50, 51, 74], would answer the question this thesis cannot. The question is how much of what a crawler finds the static graph already contains, and how much of what the static graph contains a crawler never reaches. The Android work combines static analysis with runtime traces in the same way [45, 39].

For applications used signed in, such a combination is also the way to complete the graph: a signed-in run against a seeded backend would show which may edges occur, and an observed transition would gain an observation witness beside its source witness. That is evidence that the may edge exists, not a promotion to must, since a

must edge asserts the transition on every execution the claim ranges over and one run shows it on one. A candidate never observed stays may. The combination can therefore add evidence without weakening Claim 3.1. This thesis neither builds nor measures it.

A larger and broader corpus. An automated search over a larger pool of repositories, keeping applications the extractor handles badly [34, 35], would weaken the tool-driven selection bias of Section 6.2. A case study on a proprietary industrial application, reporting only aggregate figures, is the most direct answer to the external validity threat.

The use that motivated the thesis. That use is still unmeasured. Consider a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge. Whether such a graph improves mobile testing agents or coding agents (Chapter 1), or improves automated test generation, is a controlled experiment that this thesis does not attempt. The graph and the harness are the artefacts such an experiment would need, and the defects quantified in Chapter 5 are what it would have to control for.

6.6 Summary

The transfer argument is a hypothesis, not a measurement. The most valuable next steps are an oracle for the screen set on a large application and a runtime tier that completes the graph for signed-in use.

Chapter 7

Conclusion

This thesis measured how much of the navigation structure of an Angular application can be recovered from its source code without running it. It explained where the recovery fails, and it proposed an analysis for the largest loss. Table 7.1 summarises the answers to the five research questions; the contributions and what remains open follow.

Table 7.1: Answers to the research questions, with the figures they rest on and their main limit.

RQ	Answer	Key figures	Main limit
RQ1: how much is recovered	Nodes: every screen the static readers find declared has a node. Transitions: almost no navigation construct becomes a deterministic transition, and the tier is small mostly because of the applications.	Nodes in all 6 applications; 2 of six complete against the mounted routes. 25 constructs yield 159 must edges, 125 from one toolbar. 1582 of 1607 may-only constructs sit behind application guards, branches, callbacks or template conditions; 25 are held at may by the extractor alone (Table 5.1).	Most missing addresses lie behind one construct the extractor declares unread. Most must edges rest on an unread route-order assumption (126 edges; 33 without it). The toolbar's scope is untested. The witness check cannot read the destination of 62 edges.
RQ2: where extraction fails	Five defect classes and one candidate class (Table 5.4). Under the rules' order, attribution is the largest loss category in every application.	Each lost construct goes to the first loss category whose rule matches. Testing the target before the owner makes the target category larger on alfresco and k8s dashboard (Section 5.2).	The ranking depends on the order in which the rules are applied.

RQ	Answer	Key figures	Main limit
RQ3: what scope attribution adds	It recovers constructs, almost all as possible rather than certain transitions, and loses none the baseline recovered. Only it places navigation inside services.	300 of 454 (66.1 %); 37.7 % without mifos; 4 of six recover under half. Realworld: 100.0 % of the reference pairs, none outside, against 99.5 % for earlier work's mechanisms; services 20 of 48, with none of 9 class-join pairs asserted. Alfresco: non-shell recall 43.2 % to 90.9 % (overall 11.5 % to 23.6 %) at 82.0 % precision, below the class join's 92.3 %; 9 pairs lie outside the reference at one site whose trigger is declared in configuration. End-to-end on mifos: 20 of 22 (10 without attribution; 17 and 20 at earlier work's two levels), against 12 before the repairs the check exposed; every remaining miss is in the preregistered redirect class (Section 5.3.5).	Most recovery is on mifos. The realworld reference was partly completed from the graph's output and guided two repairs. A library shell no level reads holds most alfresco pairs. On realworld the end-to-end check does not discriminate, and on mifos it discriminates little: the graph holds 163138 of 367236 possible pairs. Only two applications have such suites.
RQ4: whether the graph responds to change	On three applications the graph detects injected link deletions, retargets and guard removals. It shows that the graph responds to change, not how well it would select tests.	45 of 45 detected. Of 51 mutants both graphs can score, attributed 51, baseline 50.	Operators are scored only at constructs the graph already represents, in the simplest form read, so detection is close to guaranteed. The harness was corrected after its misses were read; the final sample shares 46 of its 73 mutants with that run, so the rate is not an independent test. The samples are small, and without attribution fewer changes can be scored.

RQ	Answer	Key figures	Main limit
RQ5: whose the losses are	Almost every cause is a construct a static reader can read: the information was in the source, the loss was the tool's, and the final version reads most of them.	Every site that still yields no transition carries a recorded label; 2 say that no reason was found (Table 5.5, Table 5.10).	A statement about classes, not every lost construct: destinations only the run time knows are counted apart. Still unread: named router outlets (modelled as a second mount, not implemented) and one application's spread of computed routes (declared unread). The target forms the end-to-end check found are read since its repairs. The unrecovered attribution remainder mixes tool gaps with components no screen may render, and no measurement separates them. Two causes lie beyond any static reader of a route table: a path computed in a custom `UrlMatcher`, and a choice between two route tables made at deployment. A check enforces that a label is recorded, not that it is correct.

Contributions. The thesis contributes four things. The first is the extractor, uigraph, built in this work. It accepts nested and parameterised route trees, which the stated restrictions of an earlier Angular extraction exclude. Those restrictions admit 0 of the 6 corpus applications (Section 5.3.6). uigraph extracts all of them. The second is the measurement itself: a corpus of six pinned applications, five evaluation methods with different blind spots, four further checks of the scopes, and a classification of every lost construct. The third is the apparatus that makes it checkable. The result tables are generated from the harness's result files. The methods that read the sources re-run from pinned commits. The runtime route oracle re-runs only in part. The realworld end-to-end check depends on an unpinned public backend. Superseded figures cannot be regenerated. The fourth is scope attribution, specified as a dataflow analysis over screen sets and implemented over three of its four input relations (Section 3.2). It contains the mechanisms of earlier work as levels and adds one they lack, the placement of navigation inside services (Section 5.3.6).

What remains open. The dispatch relation is specified but not implemented, the guard relation is replaced by a stand-in, and the call graph ignores the injector hierarchy. The screen sets were checked in both directions on two applications only, against references that apply the analysis's own relations. Further, uigraph was developed on the same corpus it is measured on. The end-to-end check on mifos found transitions lost to target forms the extractor did not read and to a relative link resolved against the child tab's address rather than the route that holds it. They were repaired after the check, so its final mifos figure is in-sample (Section 5.3.5). The readings of the must tier were made by the AI assistant, and two readings by one model measure consistency rather than correctness. The rendered-interface check judged only the screens that are reachable without signing in. Driving the applications into a signed-in state is the work that would test the rest (Section 6.5). Finally, whether such a graph helps the consumers that motivated the work, such as agents that drive a user interface or automated test generation, is not measured here.

What the measurements establish is that a declared route table is a usable starting point for static navigation extraction, but not a sufficient one.

Bibliography

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Effects. `https://angular.dev/guide/signals/effect`, 2026. Accessed November 2026.

[27] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[28] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[29] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[30] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[32] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[33] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[36] Iva Kertusha, Gebremariam Assres, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry. *Science of Computer Programming*, 253:103473, 2026.

[37] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[38] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[39] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[40] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[41] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[42] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[43] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[44] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[45] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[46] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[47] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[48] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[49] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[50] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[51] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[52] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[54] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[55] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[56] Naito Oshima and Tomoji Kishi. Model checking method for SPA page transition based on component-based framework. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)*. KSI Research, 2018.

[57] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[58] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[59] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[60] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[61] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[62] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[63] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[64] Bachar Rima, Anas Shatnawi, and Abdelhak-Djamel Seriai. E2E test generation over a source-code-derived workflow space: An Angular case study. In *Software Engineering and Advanced Applications*, Lecture Notes in Computer Science, pages 334–351, Cham, 2026. Springer Nature Switzerland. Published online 27 August 2026; print volume dated 2027. The method is described in this thesis from the publisher abstract and the replication package; the full text was not available.

[65] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[66] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[67] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[68] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[69] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[70] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. *Proceedings of the ACM on Software Engineering*, 2(FSE):1001–1022, 2025.

[71] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[72] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[73] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[74] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 49(3):1086–1101, 2023.

[75] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[76] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[77] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[78] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[79] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[80] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[81] Tao Zhang, Fuzhong Hao, Yunfan Wang, Bo Zhang, and Guangwei Xie. VeriFlow: A framework for the static verification of web application access control via policy-graph consistency. *Electronics*, 14(18):3742, 2025.

[82] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.